

Construction of Highway

高谷悠太

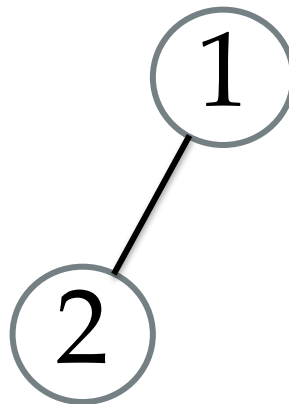
問題概要

- 木の頂点を根に近い順に追加していく。
- 木の頂点には活気という値があり、頂点を追加するたびに、活気の列の inversion を求める。
- そして、活気の値を書き換える。

サンプル

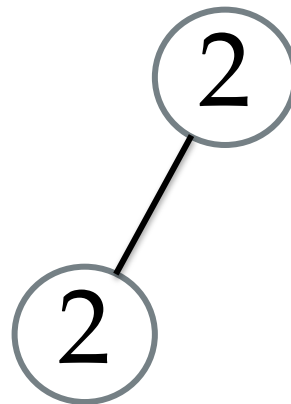
①

サンプル

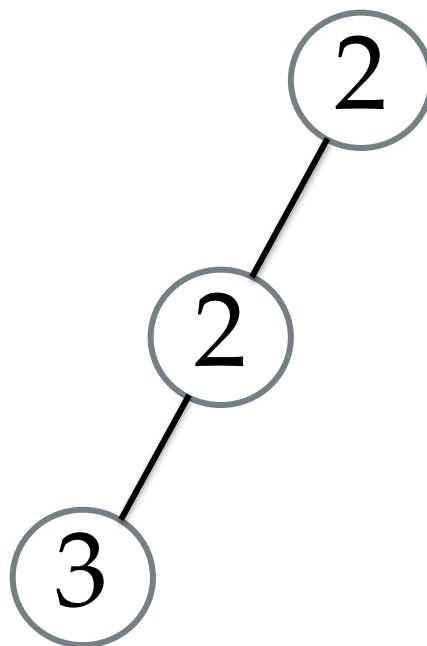


1 なので、
inversion は 0

サンプル

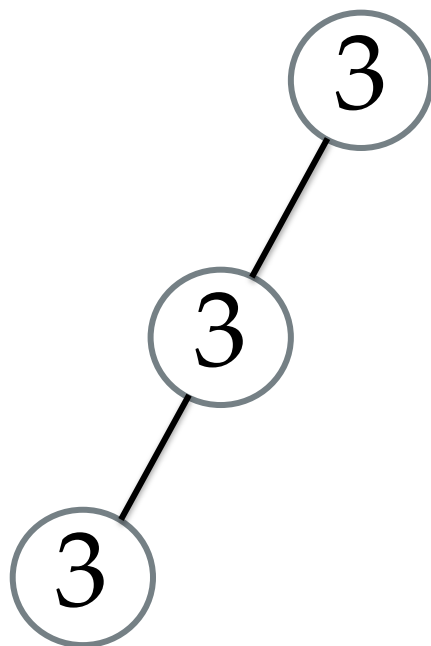


サンプル

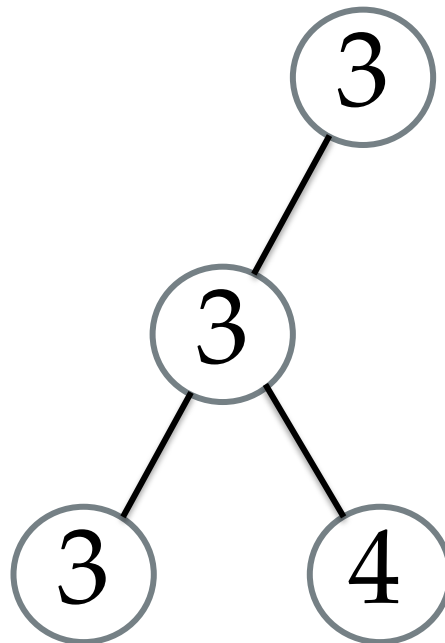


2,2 なので、
inversion は 0

サンプル

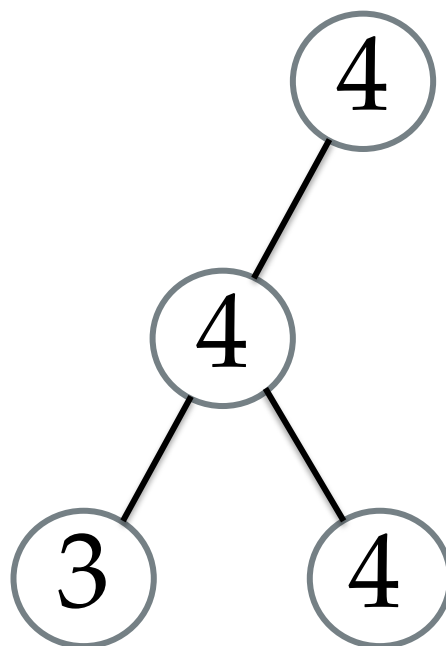


サンプル

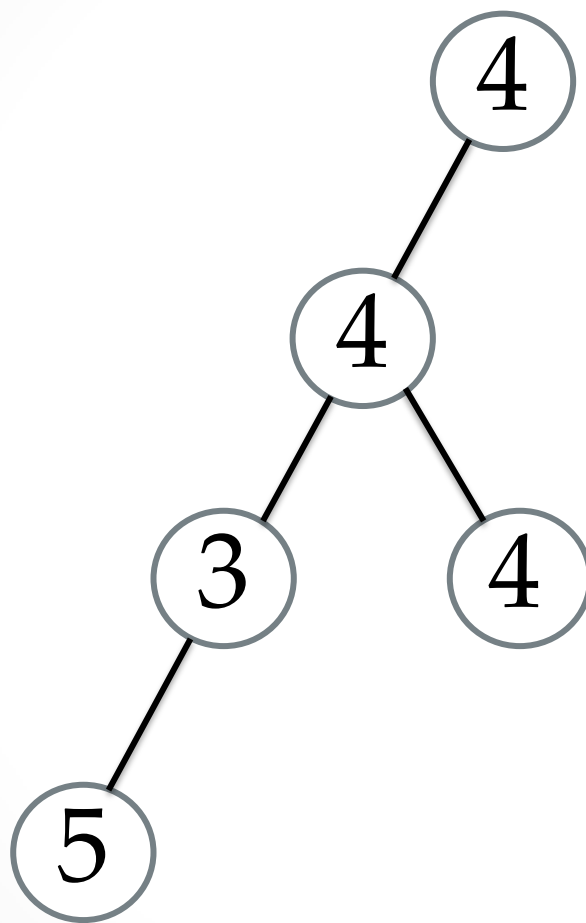


3,3 なので、
inversion は 0

サンプル

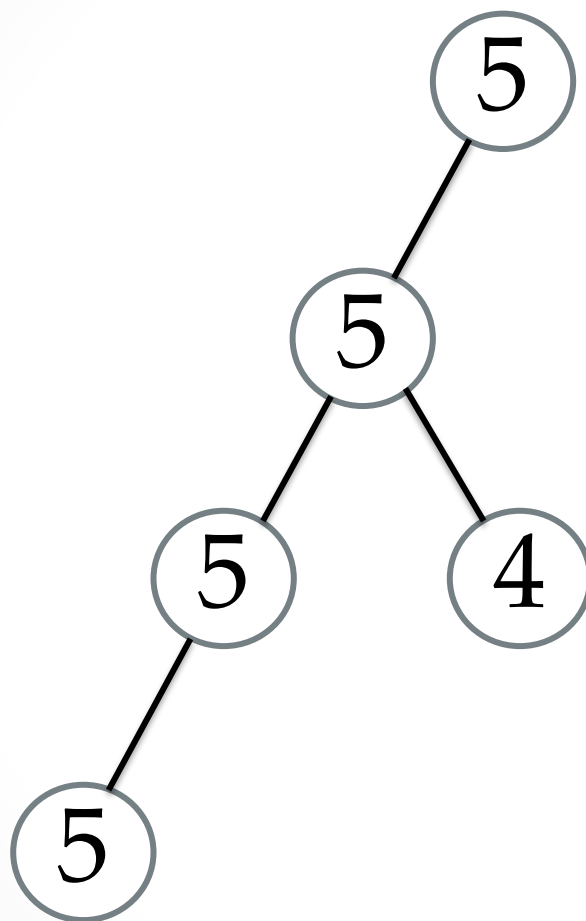


サンプル



4,4,3 なので、
inversion は 2

サンプル



制約

- 小課題1 : $1 \leq N \leq 500$
- 小課題2 : $1 \leq N \leq 4000$
- 小課題3 : $1 \leq N \leq 100000$

制約

- 小課題1 : $1 \leq N \leq 500 \rightarrow O(N^3)$
- 小課題2 : $1 \leq N \leq 4000 \rightarrow O(N^2)$
- 小課題3 : $1 \leq N \leq 100000 \rightarrow O(N * \log N)$
- ぐらい??

小課題1

- $N - 1$ 回それぞれで $O(N^2)$ かけてよいから、根までのパスを調べることができる
- 更新も簡単

小課題1

- $N - 1$ 回それぞれで $O(N^2)$ かけてよいから、根までのパスを調べることができる
- 更新も簡単
- inversion の計算を $O(N^2)$ でできればよい

小課題1

- $N - 1$ 回それぞれで $O(N^2)$ かけてよいから、根までのパスを調べることができる
- 更新も簡単
- inversion の計算を $O(N^2)$ でできればよい
- 愚直で大丈夫

小課題1

- $N - 1$ 回それぞれで $O(N^2)$ かけてよいから、根までのパスを調べることができる
- 更新も簡単
- inversion の計算を $O(N^2)$ でできればよい
- 愚直で大丈夫

7てん

小課題2

- $N - 1$ 回それぞれで $O(N)$ かけてよいから、根までのパスを調べることができる
- 更新も簡単

小課題2

- $N - 1$ 回それぞれで $O(N)$ かけてよいから、根までのパスを調べることができる
- 更新も簡単
- inversion の計算を $O(N)$ ぐらいでできればよい

小課題2

- $N - 1$ 回それぞれで $O(N)$ かけてよいから、根までのパスを調べることができる
- 更新も簡単
- inversion の計算を $O(N)$ ぐらいでできればよい
- BIT で大丈夫

1 6 て ん

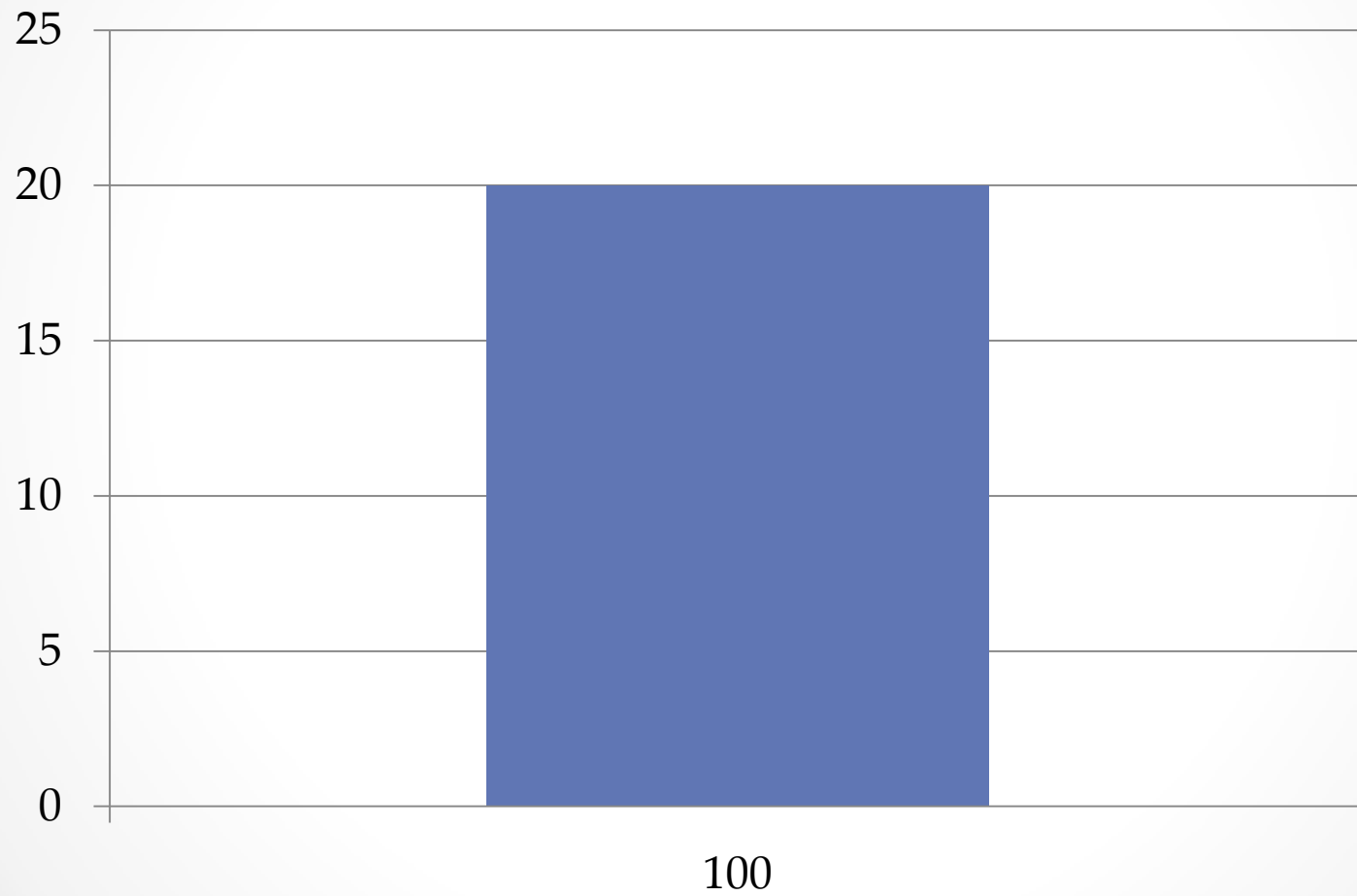
満点解

- $N - 1$ 回それぞれで $O(1)$ かけてよいから、根までのパスを調べることができる
- 更新も簡単
- inversion の計算を $O(1)$ でできればよい

満点解

- $N - 1$ 回それぞれで $O(1)$ かけてよいから、根までのパスを調べることができる
- 更新も簡単
- inversion の計算を $O(1)$ でできればよい

得点分布



満点解

- $N - 1$ 回それぞれで $O(1)$ かけてよいから、根までのパスを調べることができる
- 更新も簡単
- inversion の計算を $O(1)$ でできればよい
- ???????

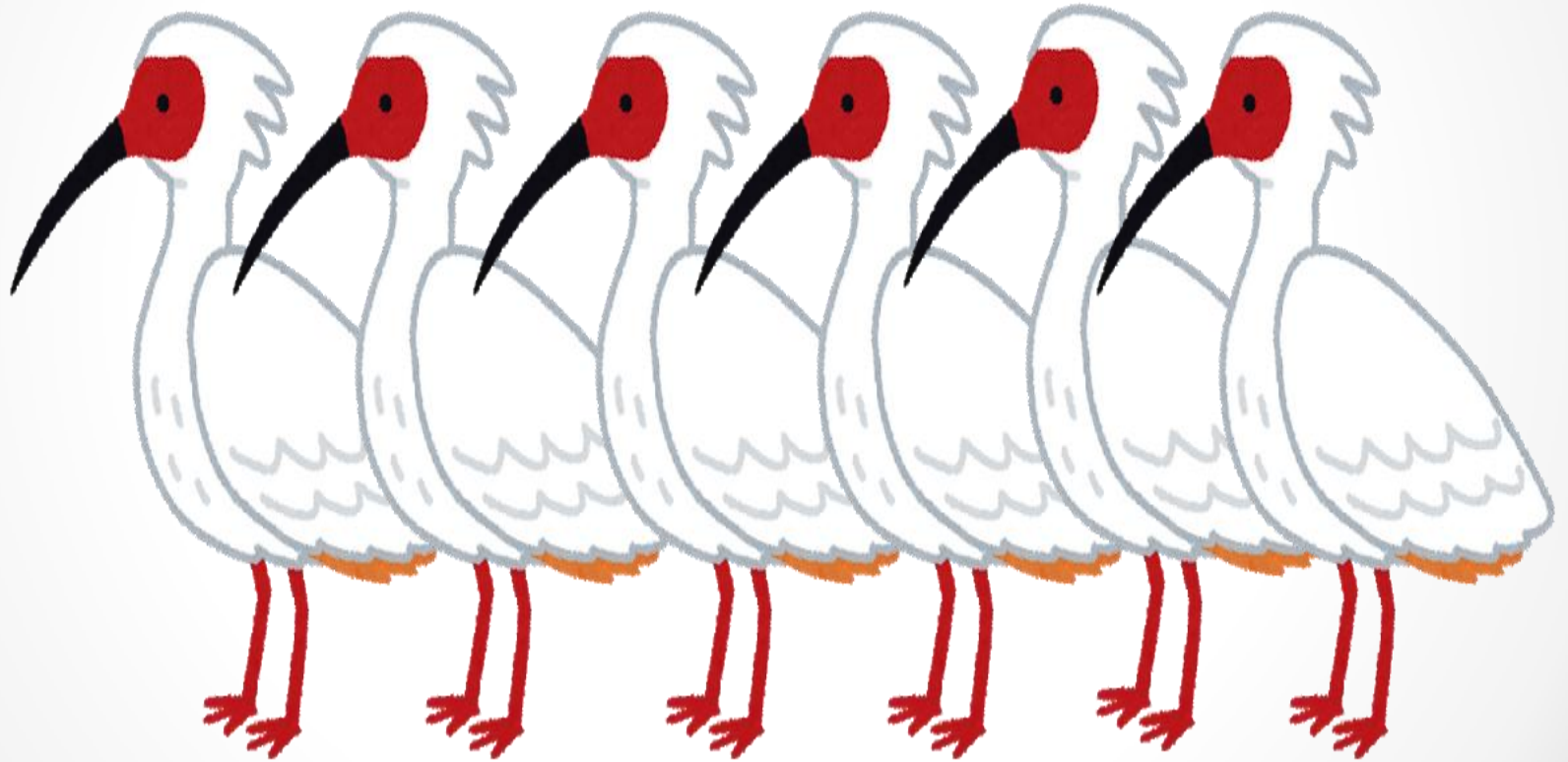
よく考えよう

- 今までの小課題では全く問題の性質を使っていない
- N 回ぐらい inversion の計算をしてるだけ

よく考えよう

- 今までの小課題では全く問題の性質を使っていない
- N 回ぐらい inversion の計算をしてるだけ
- 具体的な例で考えてみよう！！

列のトキ



列のとき

④

列のとき



4 なので、
inversion は 0

列のとき



列のとき



3,3 なので、
inversion は 0

列のとき



列のとき



2,2,2 なので、
inversion は 0

列のとき



列のとき



列のとき（拡張）

- 列を作るのは自明なケースに帰着されてしまう

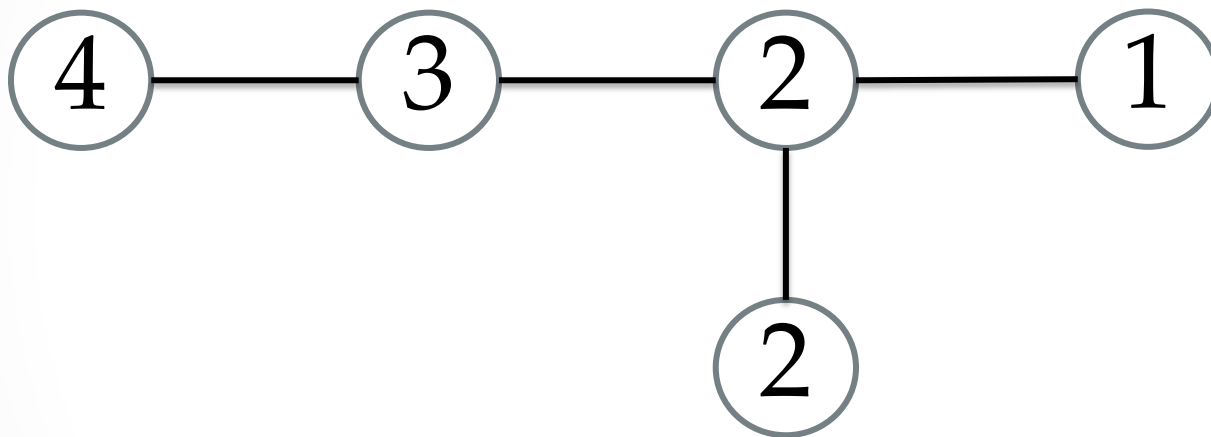
列のとき（拡張）

- 列を作るのは自明なケースに帰着されてしまう
- 列がある状態からその列を更新するのは非自明っぽい

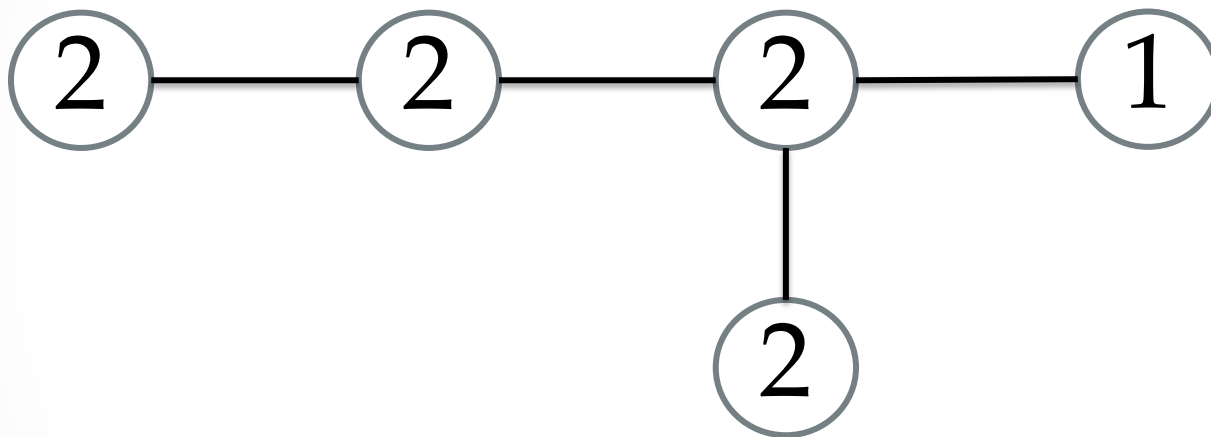
列のとき（拡張）



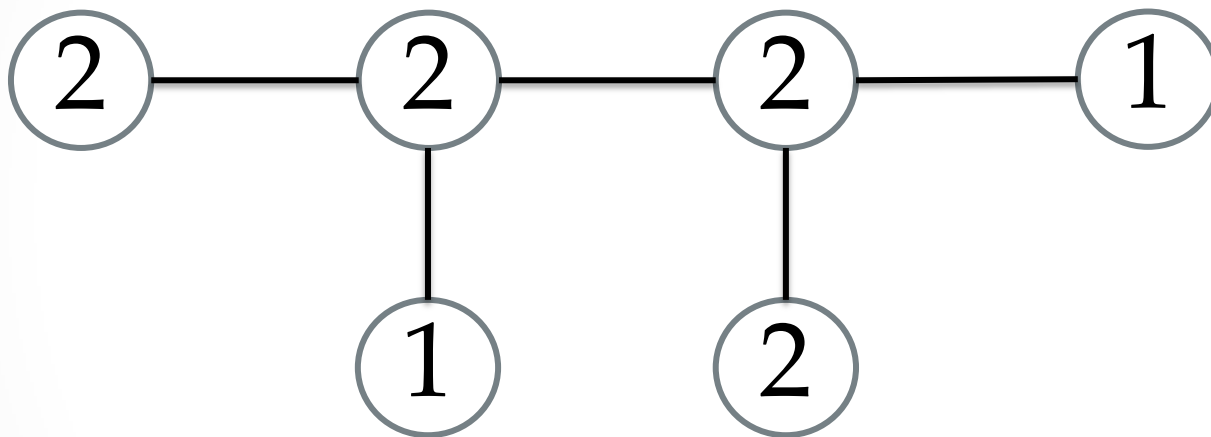
列のとき（拡張）



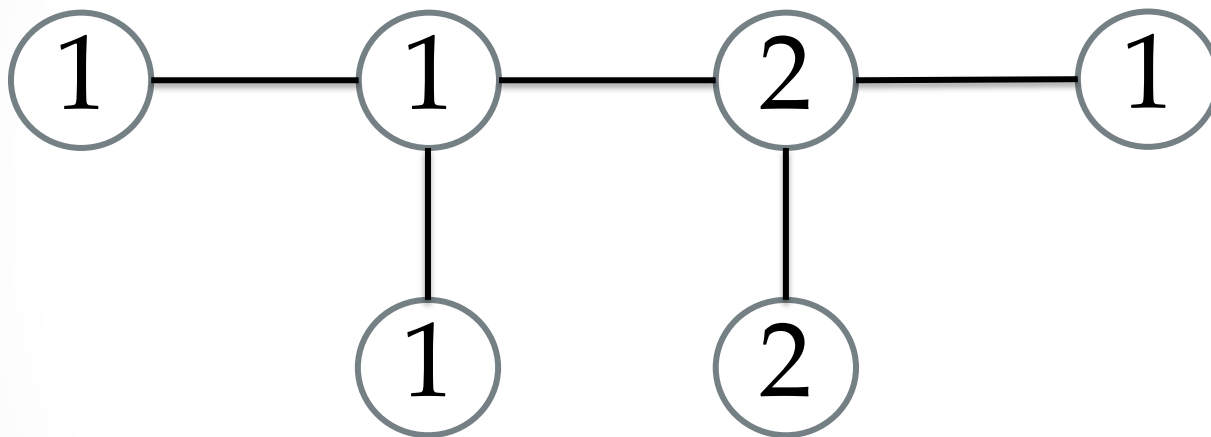
列のとき（拡張）



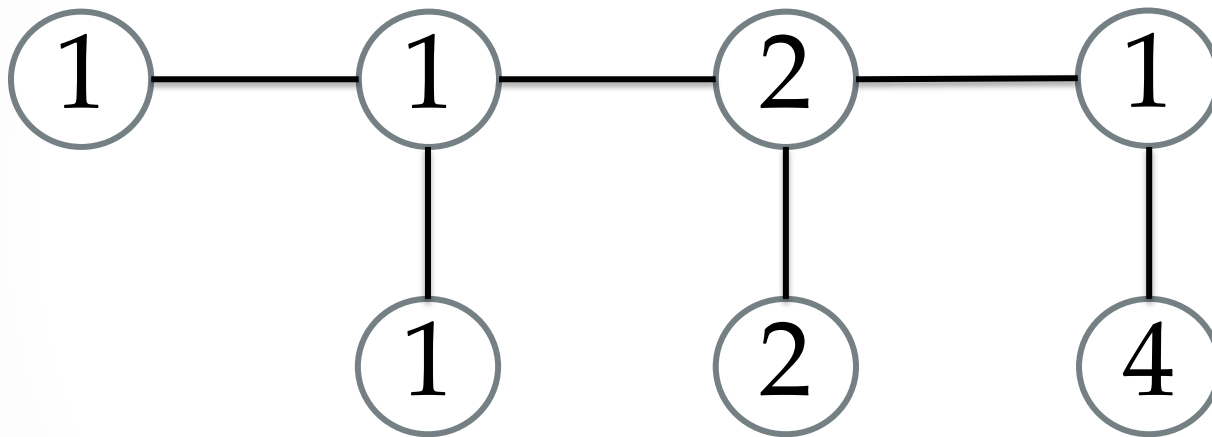
列のとき（拡張）



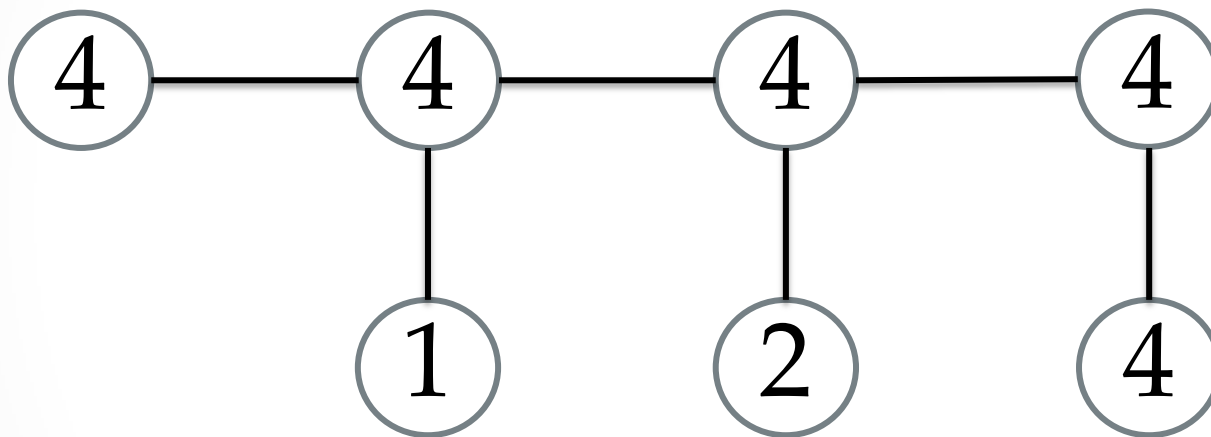
列のとき（拡張）



列のとき（拡張）



列のとき（拡張）



列のとき（拡張）

- 他の部分を無視して、今考えてる列にだけ注目する
- 先頭 k 文字を x にする、という変更が起こる

列のとき（拡張）

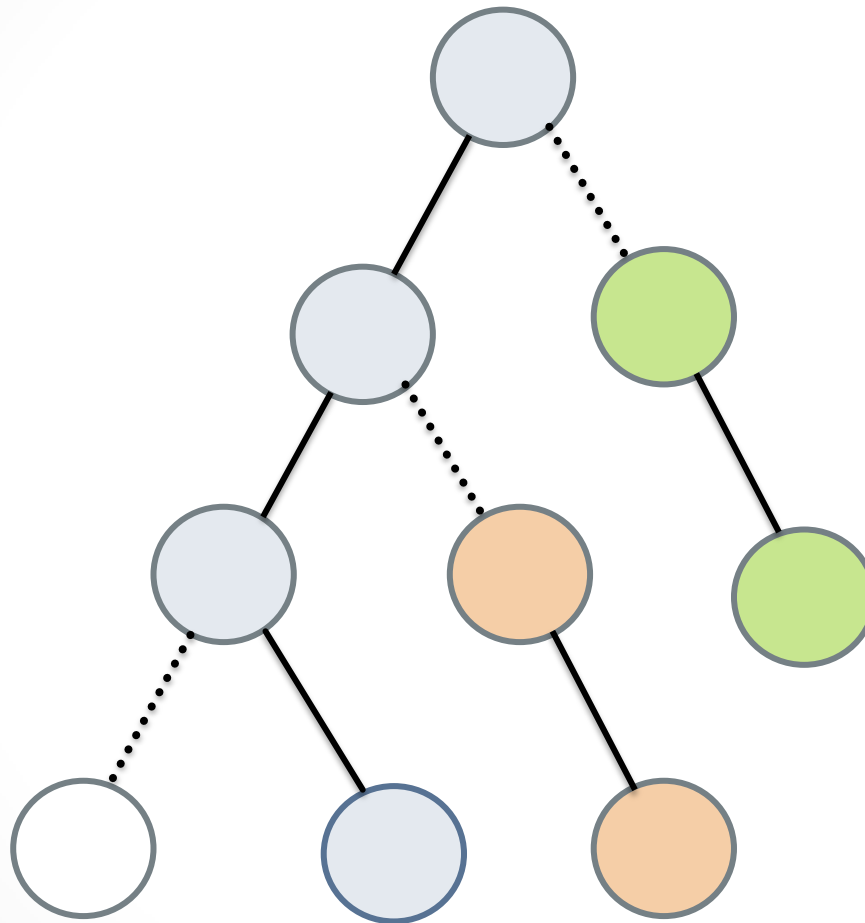
- 他の部分を無視して、今考えてる列にだけ注目する
- 先頭 k 文字を x にする、という変更が起こる
- 変更クエリは列を取り出すと扱いやすい??

Heavy Light Decomposition

HL 分解

- 根付き木を列に分解するアルゴリズム
- 各頂点 v から根までのパスは高々 $\log N$ 個の列にしか属さない

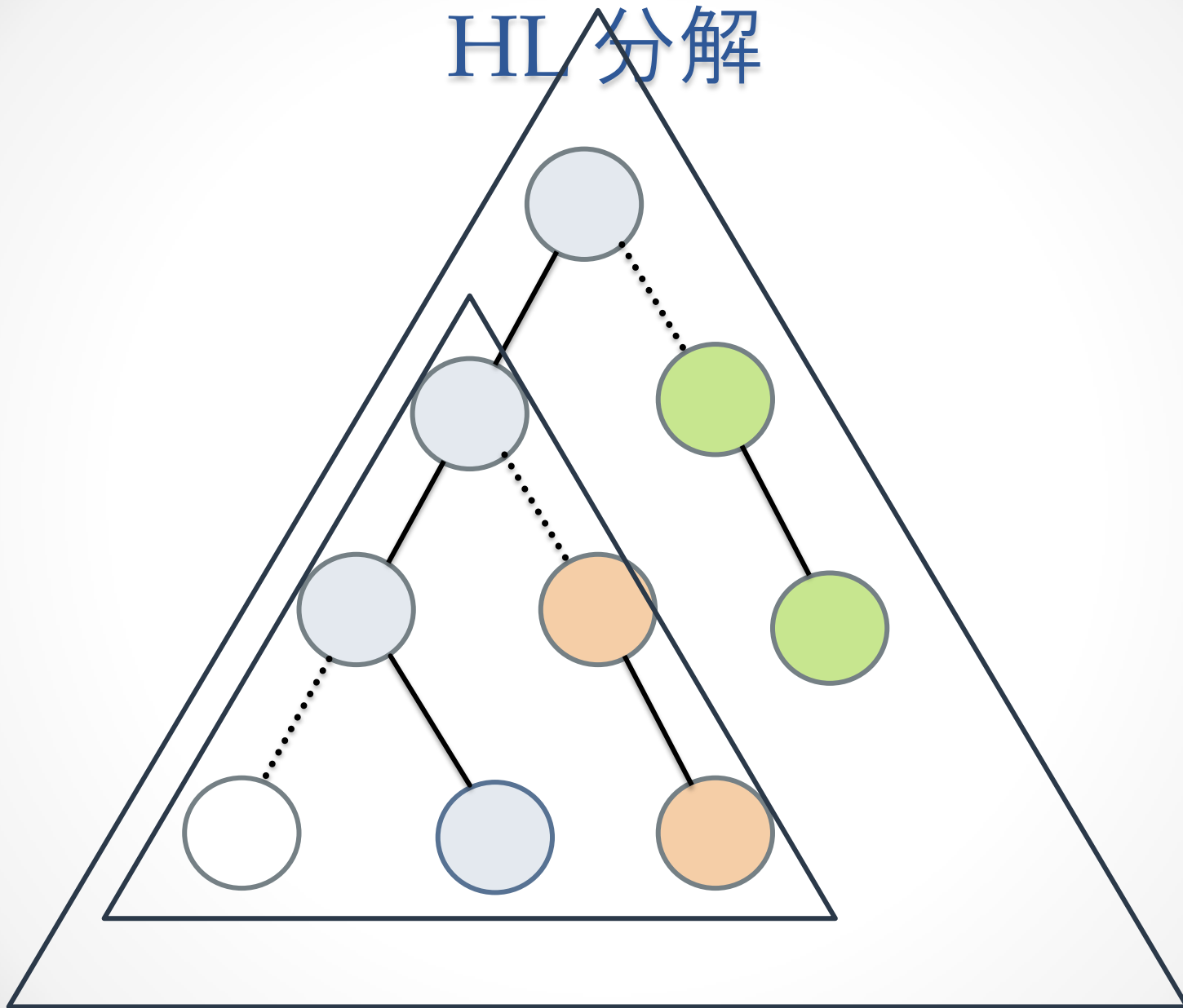
HL 分解



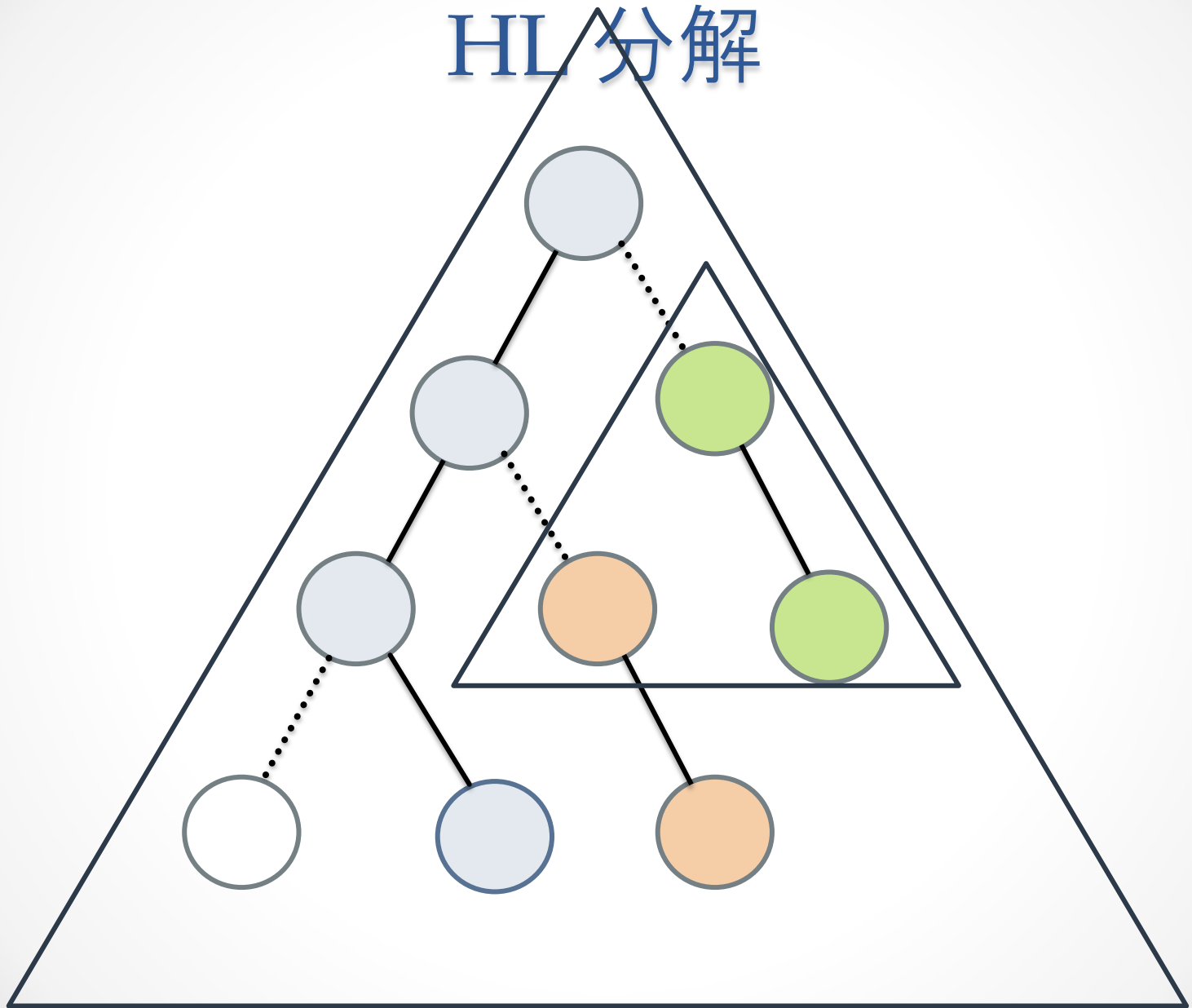
HL 分解 – 作り方

- 各頂点 v に対して、子供への辺を高々一つ決めればよい
- v の部分木サイズを S_v として、 $S_v \leq 2 * S_u$ なる子供 u は高々一つ
- そのような u への辺を採用する

HL 分解



HL 分解



HL 分解 – 解析

- 「各頂点 v から根までのパスは高々 $\log N$ 個の列にしか属さない」という条件は成り立ってる??

HL 分解 – 解析

- 「各頂点 v から根までのパスは高々 $\log N$ 個の列にしか属さない」という条件は成り立ってる??
- 言い換えると、各頂点 v から根までのパスに、列に含まれない辺が高々 $\log N$ 個しか存在しないということ

HL 分解 – 解析

- 「各頂点 v から根までのパスは高々 $\log N$ 個の列にしか属さない」という条件は成り立ってる??
- 言い換えると、各頂点 v から根までのパスに、列に含まれない辺が高々 $\log N$ 個しか存在しないということ
- 先の式を見れば、列に含まれないパスを子供から親へ上ると、部分木サイズが 2 倍になるから、確かにそうなる

ところで

- クエリごとに頂点を追加するけど、動的に HL 分解を管理することなんてできるの???

ところで

- クエリごとに頂点を追加するけど、動的に HL 分解を管理することなんてできるの???
- 最終的にできる木は先読みできるから、そこで HL 分解を作ればよい

今までの振り返り

- 0 が根の根付き木とその HL 分解が求まっている
- そこで、以下の操作を行う
- ある頂点 v から根までのパスまでの数列を得て、その inversion を求める
- そして、パス上の値をすべて一定の値に書き換える

あとは簡単

- 列の時の考察から、各列の更新は愚直にできる
(ただし、同じ値は圧縮する必要がある)
- 各列での更新ができるなら、各列の更新前も分かる
- 圧縮した値を持っても、inversion は計算できる

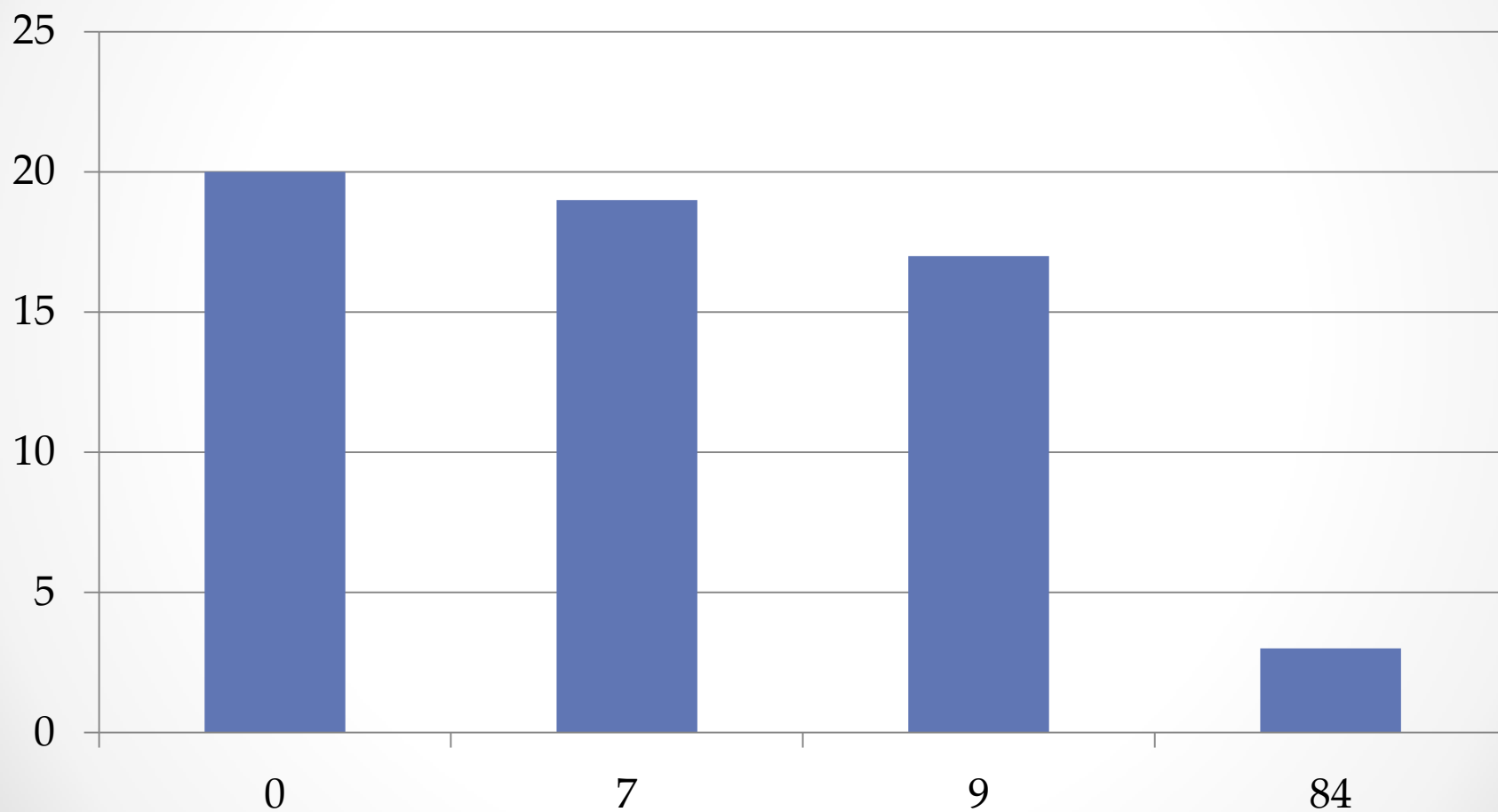
具体的に

- HL 分解を行い、各列ごとに「 x が k 個連続している」というように圧縮して、値を管理しておく
- 頂点 v について操作を行うとき、 v から根までのパスに属する列をすべて調べる
- 列ごとに、愚直に更新する
- 圧縮した列で inversion を求める

計算量解析

- 頂点 v を追加したとき、見る列の個数は $\log N$ 個
- それぞれの列で新しい値は 1 個入るだけだから、値が消えるのは全部合わせても $N \log N$ 回だけ
- inversion の計算は $O((\text{消える個数}) \log(\text{消える個数}))$ だから、すべて合わせると $O(N \log^2 N)$
- 間に合う！！

得点分布

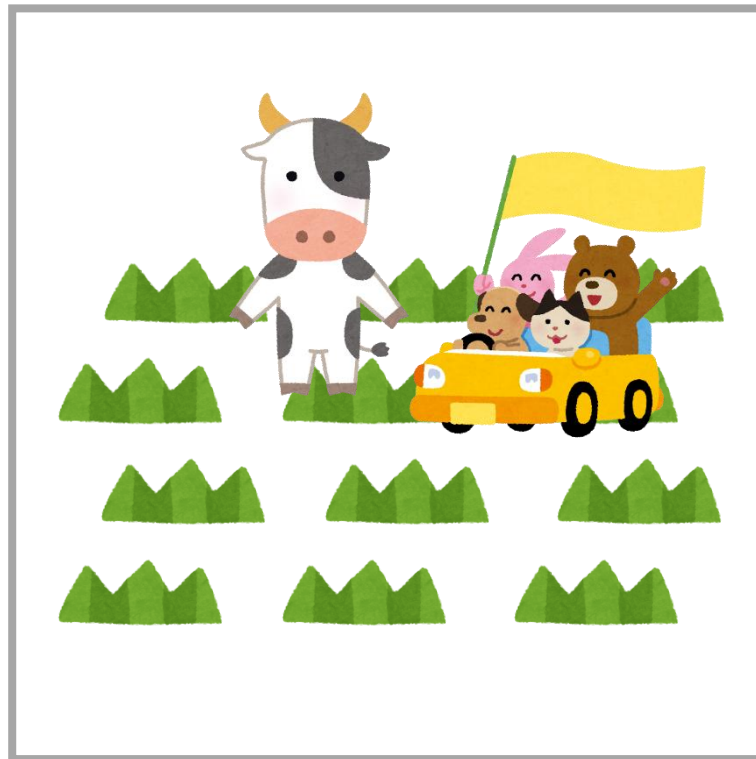


Fences 解説

解説：村井

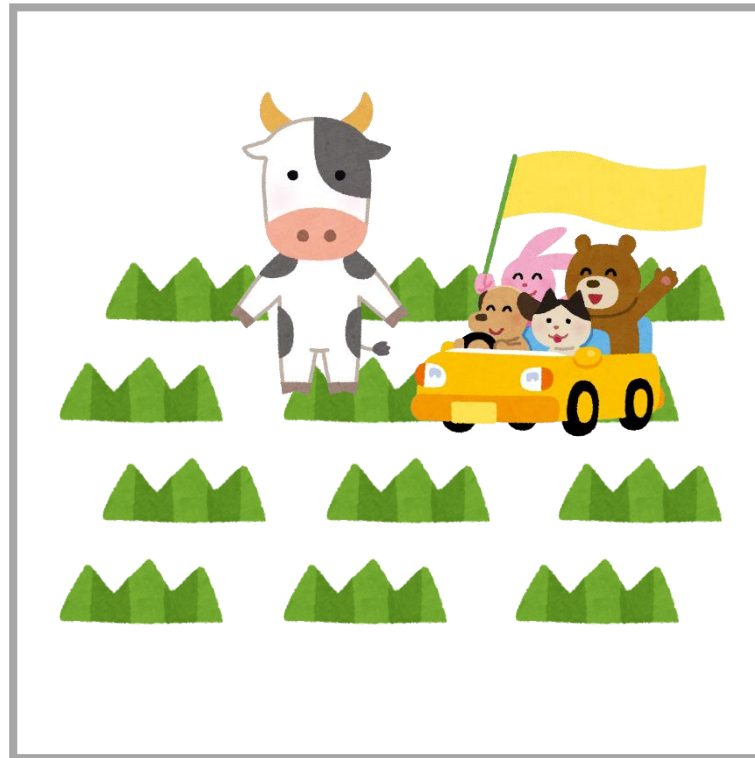
問題概要

牧草地がある



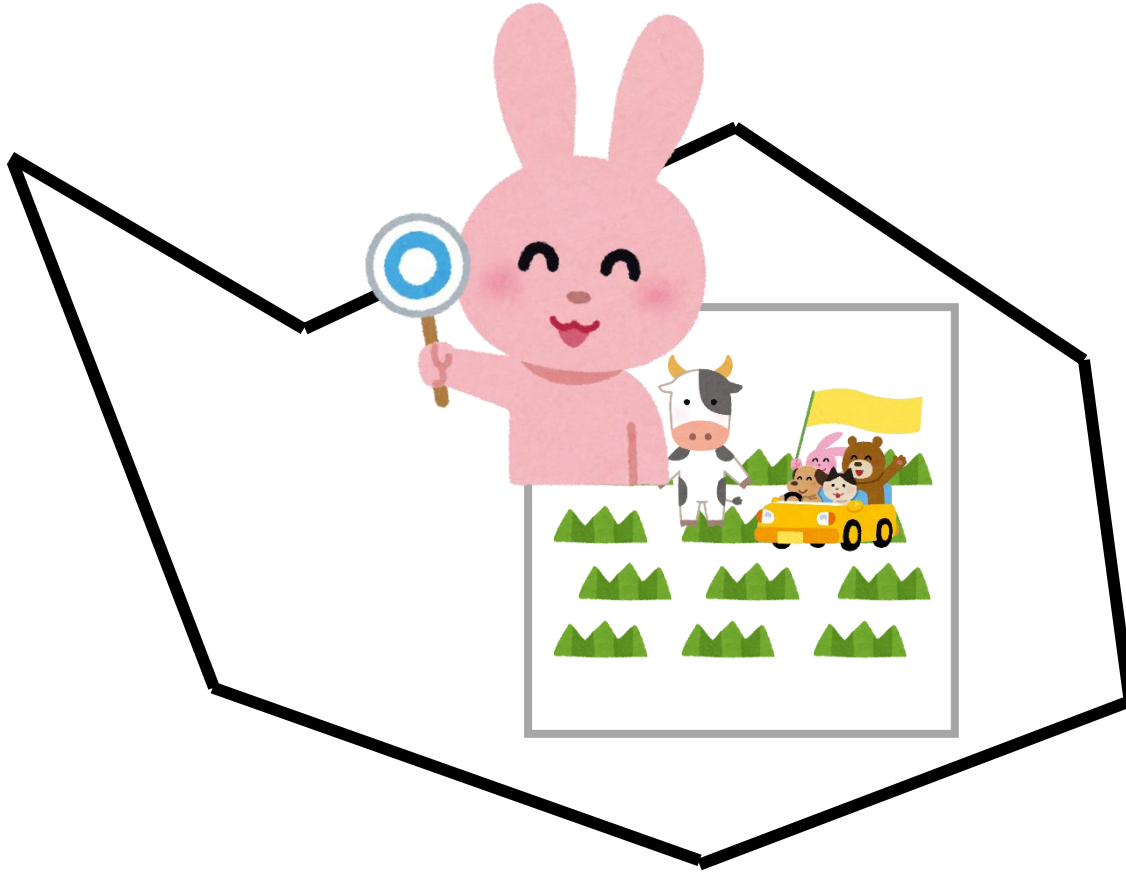
問題概要

牛が逃げ出さないように，牧草地を柵で囲いたい



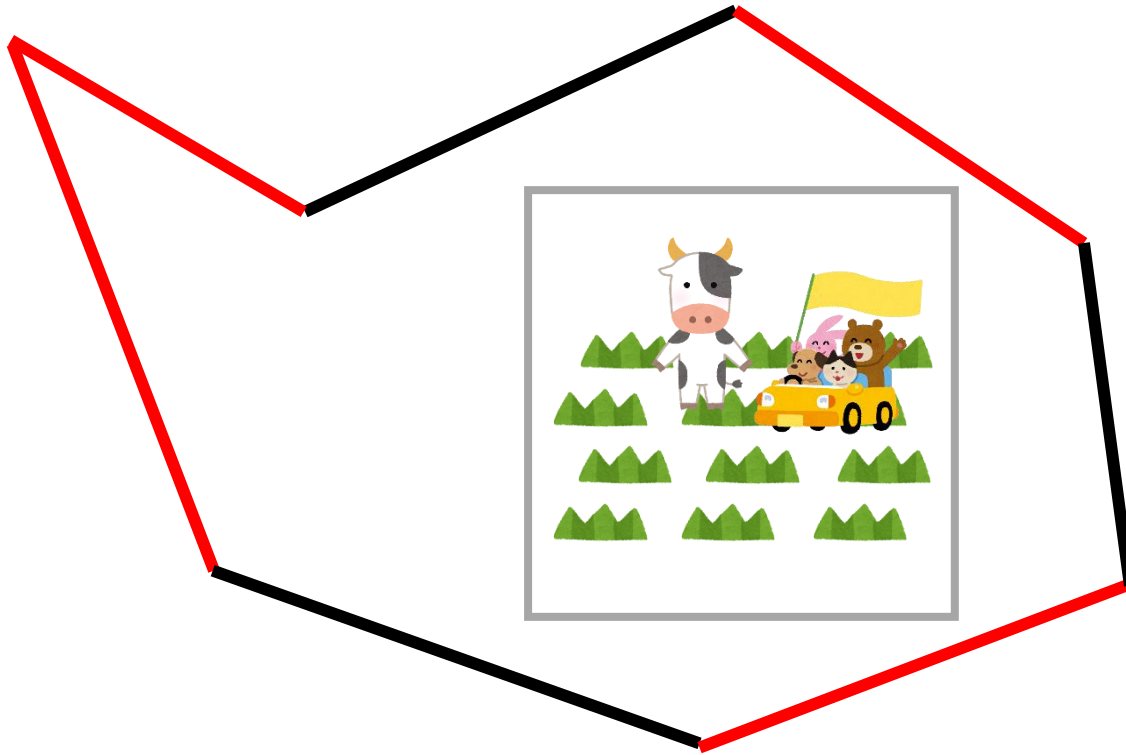
問題概要

牛が逃げ出さないように，牧草地を柵で囲いたい



問題概要

新しく設置する柵の長さの合計を最小化したい



はじめに

- この問題は「幾何」問題です
- 幾何問題を解くにあたっては、必要な知識がいくつかあるので、去年の講義スライドなどを参考にしてください

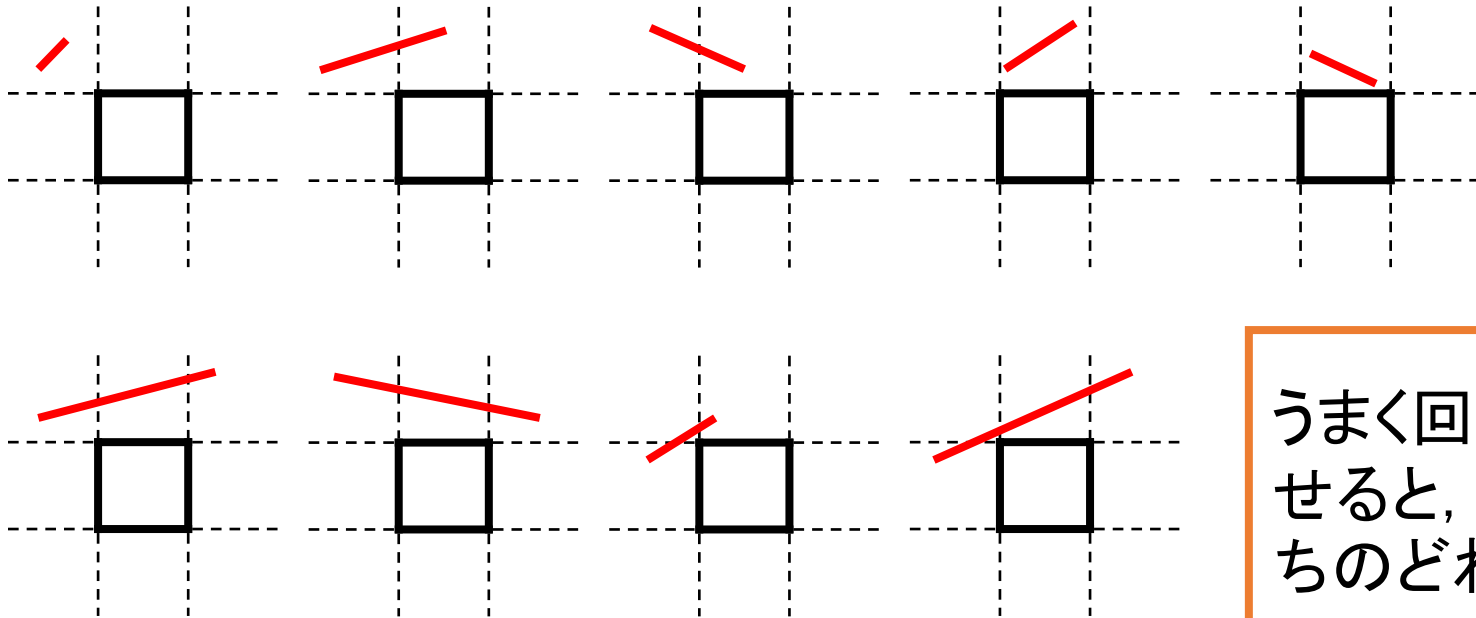
https://www.ioi-jp.org/camp/2017/2017-sp_camp-hide.pdf

小課題 1

- $N = 1$
- 最初に柵が 1 個しかない

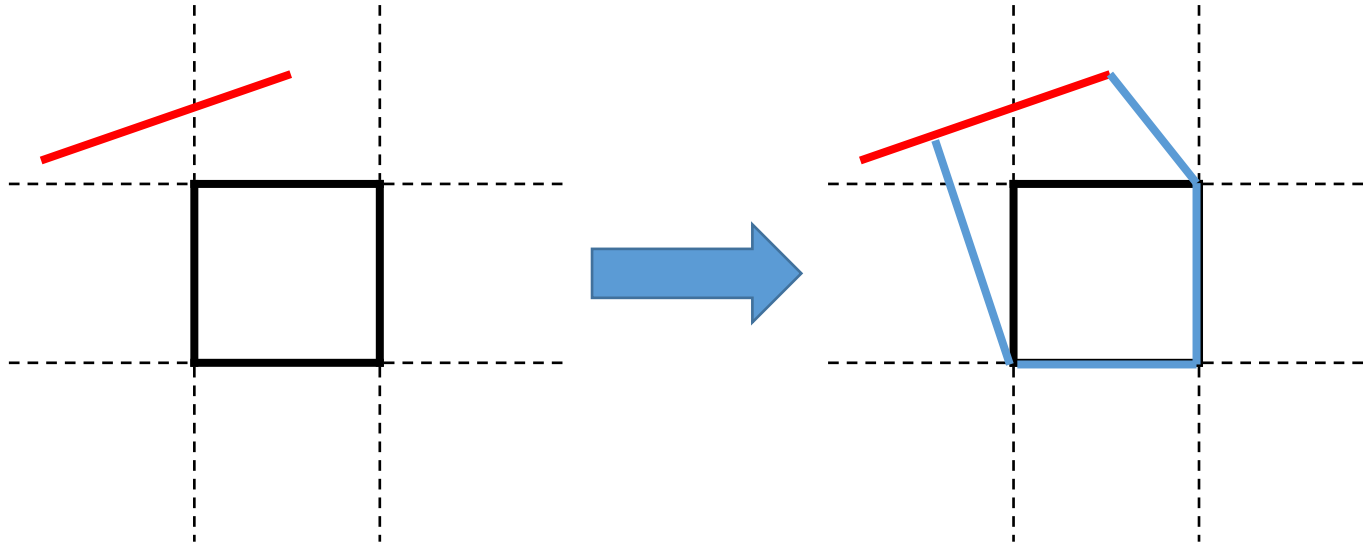
柵の置かれ方のパターン

本質的にはそんなに多くないので全部場合分けする



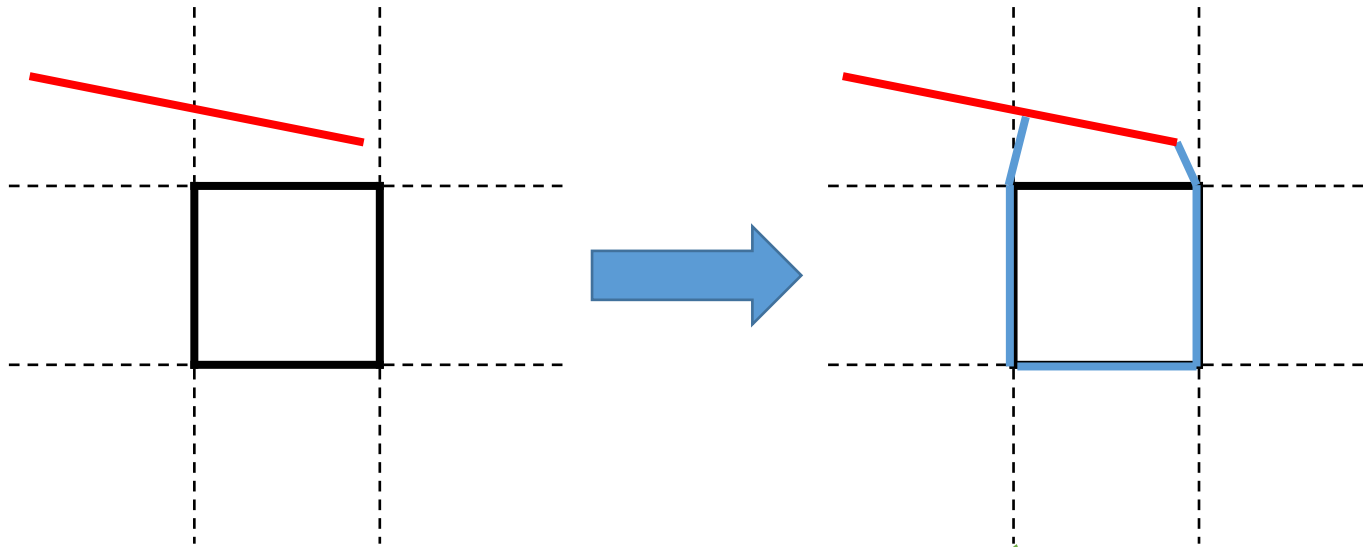
うまく回転, 反転させると, これらのうちのどれかになる

場合分けの例



牧草地の左上の角は無視して、
左下 -> 柵 -> 右上 という感じで結ぶ

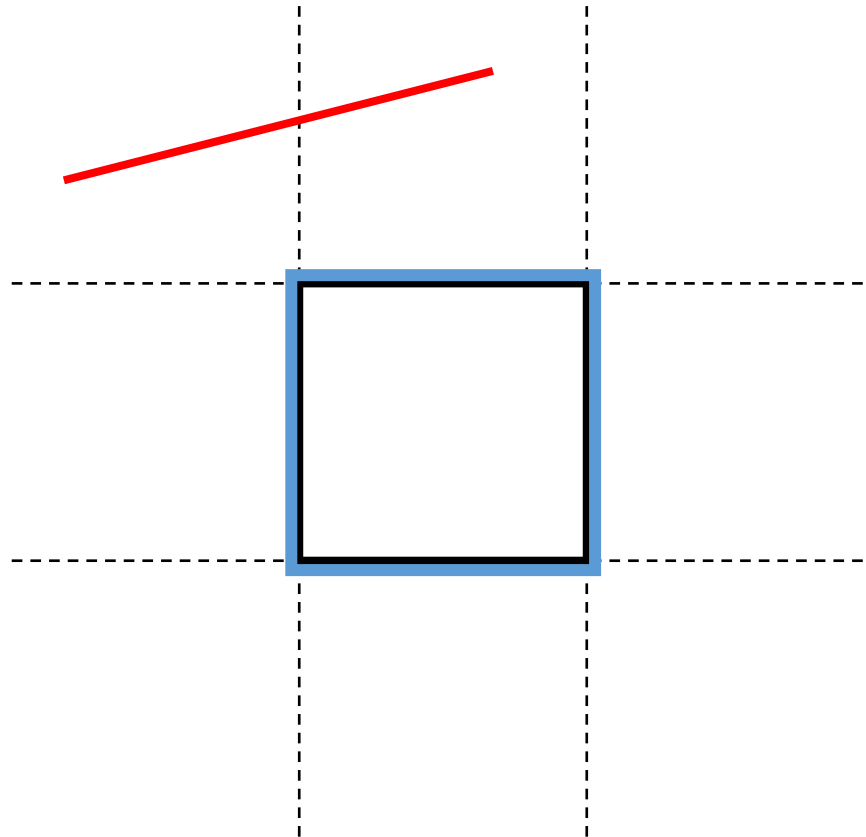
場合分けの例



左上 -> 柵 -> 右上 -> ...

例外的？なパターン

既存の柵を無視したほうがいい場合に注意



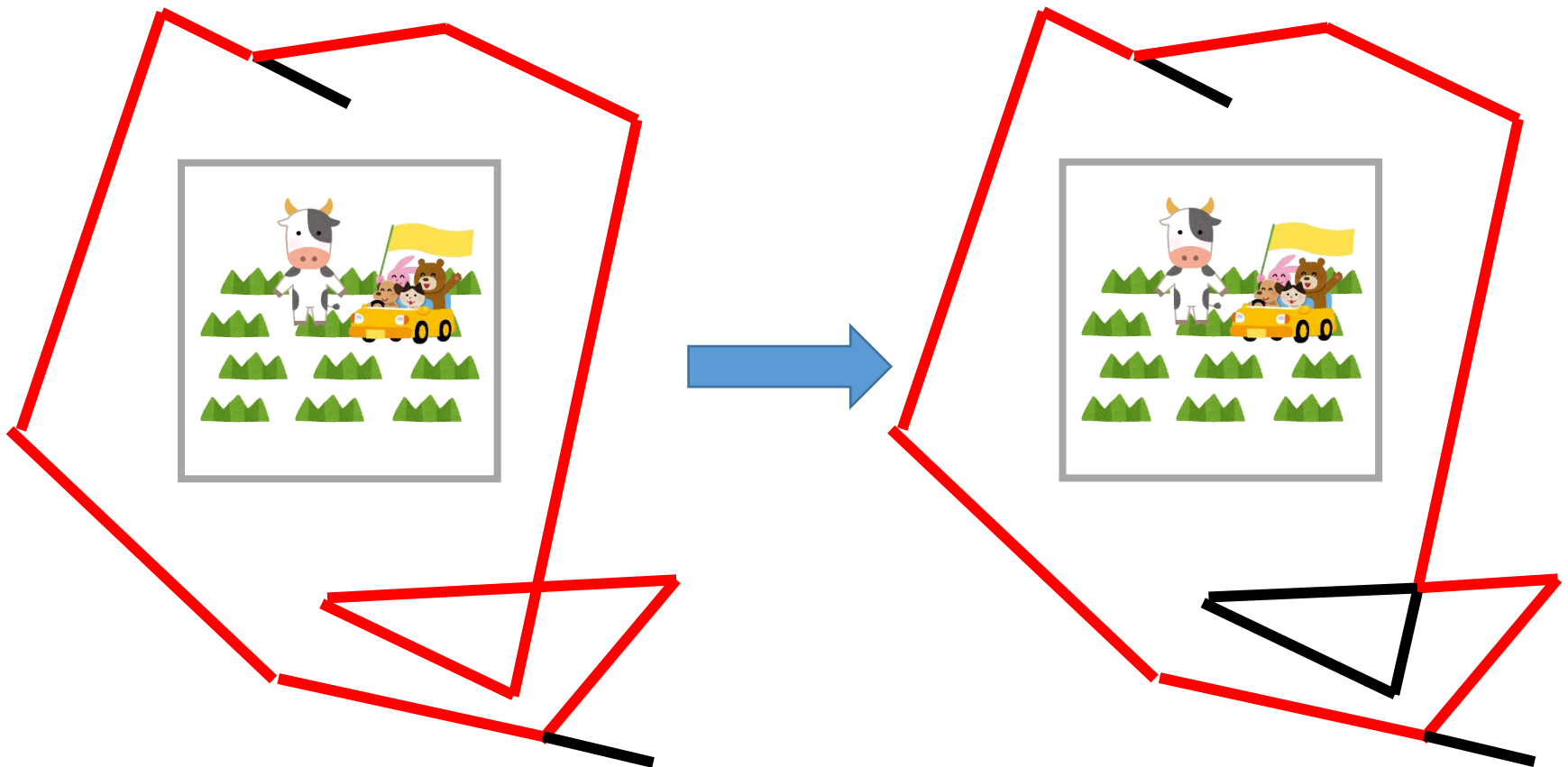
小課題 2

- $N \leq 6$
- 場合分けはとてもやる気が起きない

そもそも「牧草地を柵で囲う」ってなんだ？

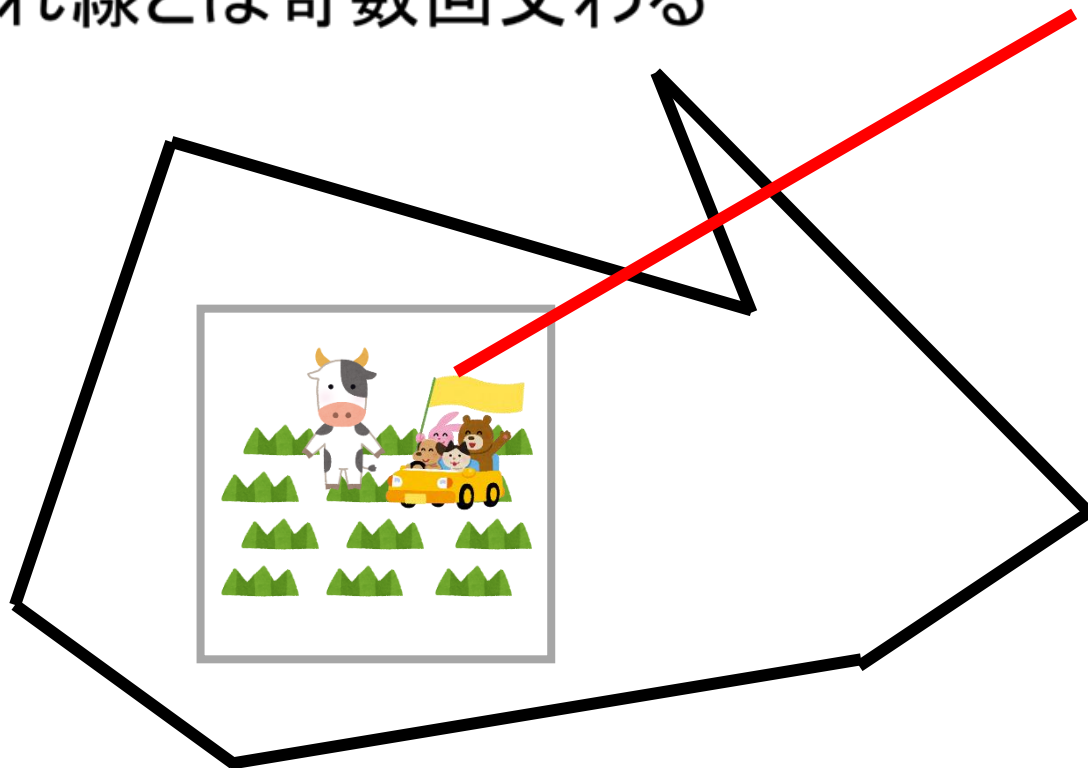
牧草地の囲い方

囲っている柵の部分に自己交差はないと思える



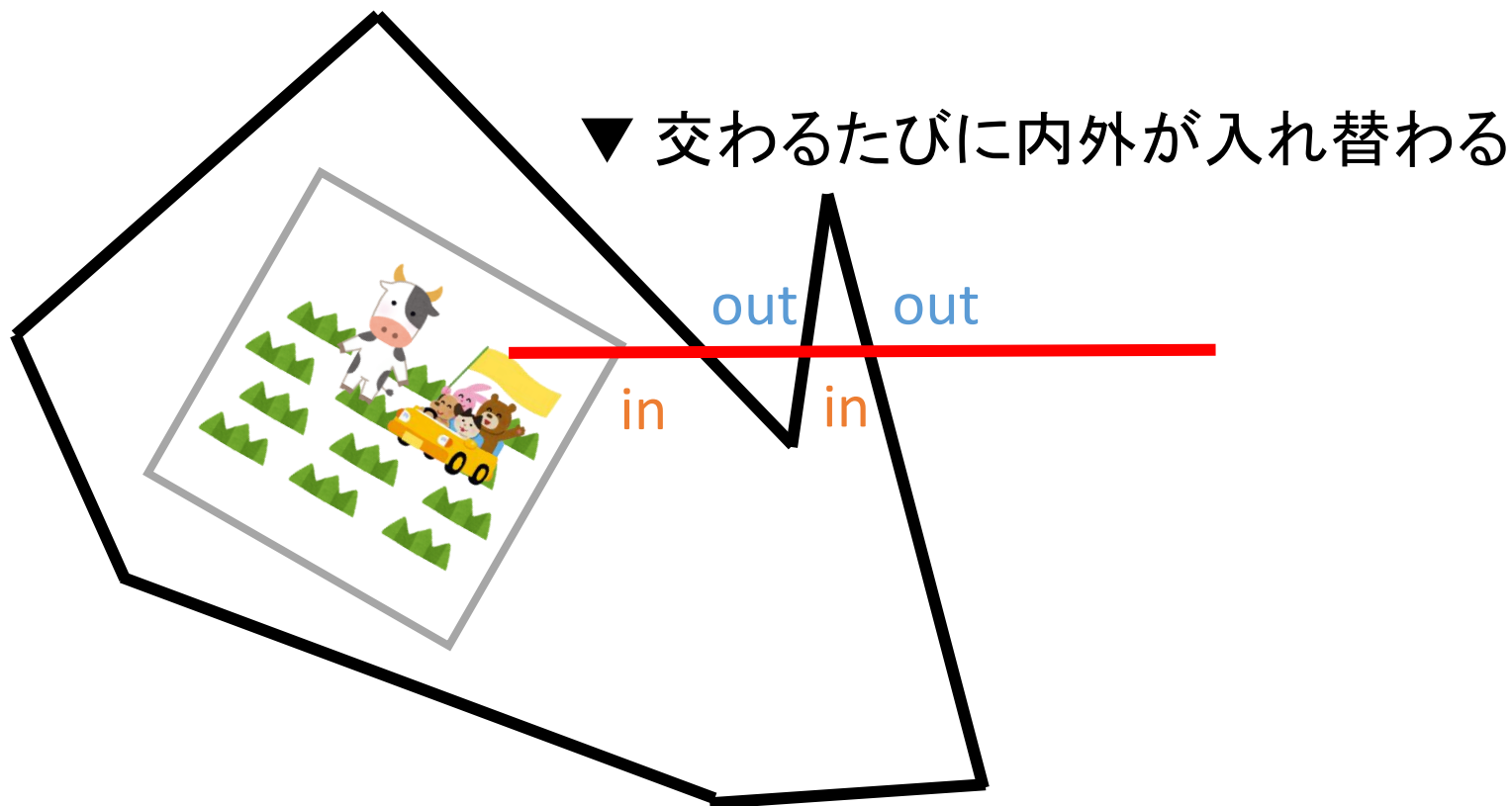
牧草地を囲う閉じた折れ線

牧草地の内部から、外へ至る適当な半直線 m を考えると、この折れ線とは奇数回交わる



牧草地を囲う閉じた折れ線

牧草地の内部から、外へ至る適当な半直線 m を考えると、この折れ線とは奇数回交わる

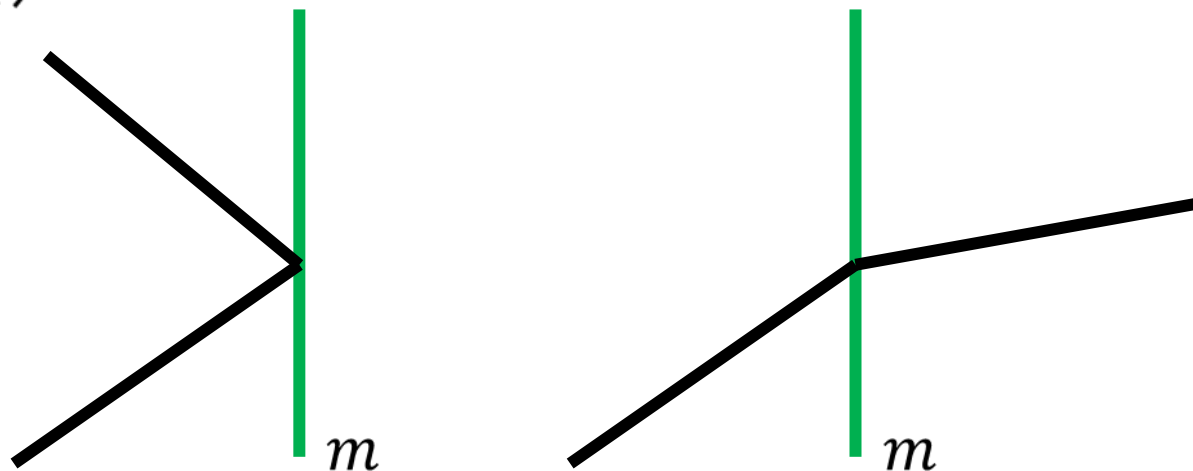


問題の言い換え

つまり,「牧草地を囲う」というのは次のように言い換えられる

牧草地の内部から, 外部へ至る半直線 m と奇数回交差するような, 閉じた折れ線を描く

(厳密には, m と折れ線が「接する」ような場合に注意が必要)



折れ線の描き方

既存の柵を使う順番を決める

-> どこで m を横切るかも決める

-> 柵と柵の間は「最短」経路で結べばよい？

最短経路？

いろんな条件がある

- m と交わる回数の偶奇ごとに考える
- 牧草地の内部に入らない

どうやって求める？

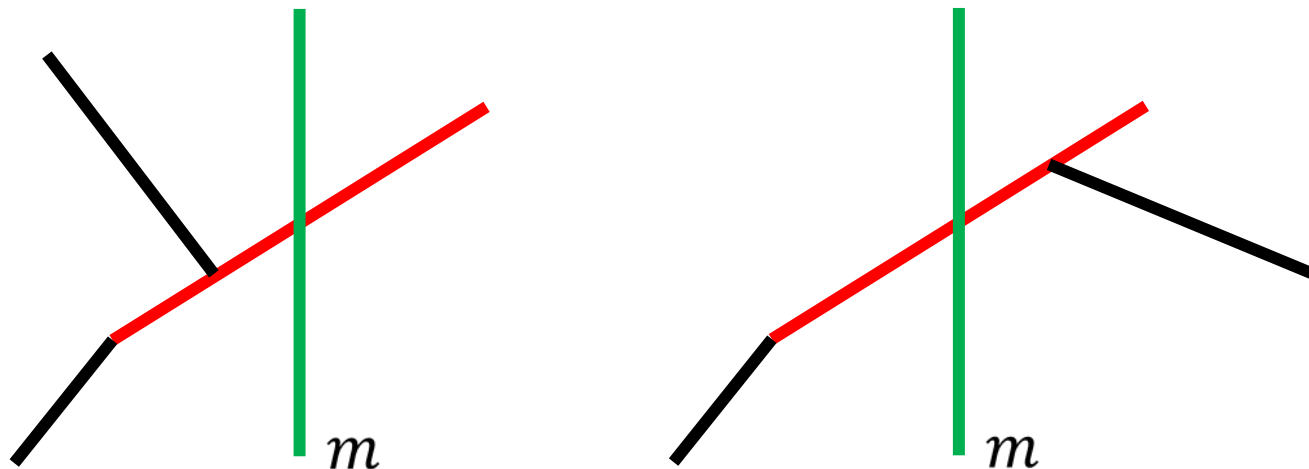
m と交わる回数の偶奇？

既存の柵上で m と交わるとちょっと困る

- 新たな柵をどこからつなげるかによって、この偶奇は変わってしまう

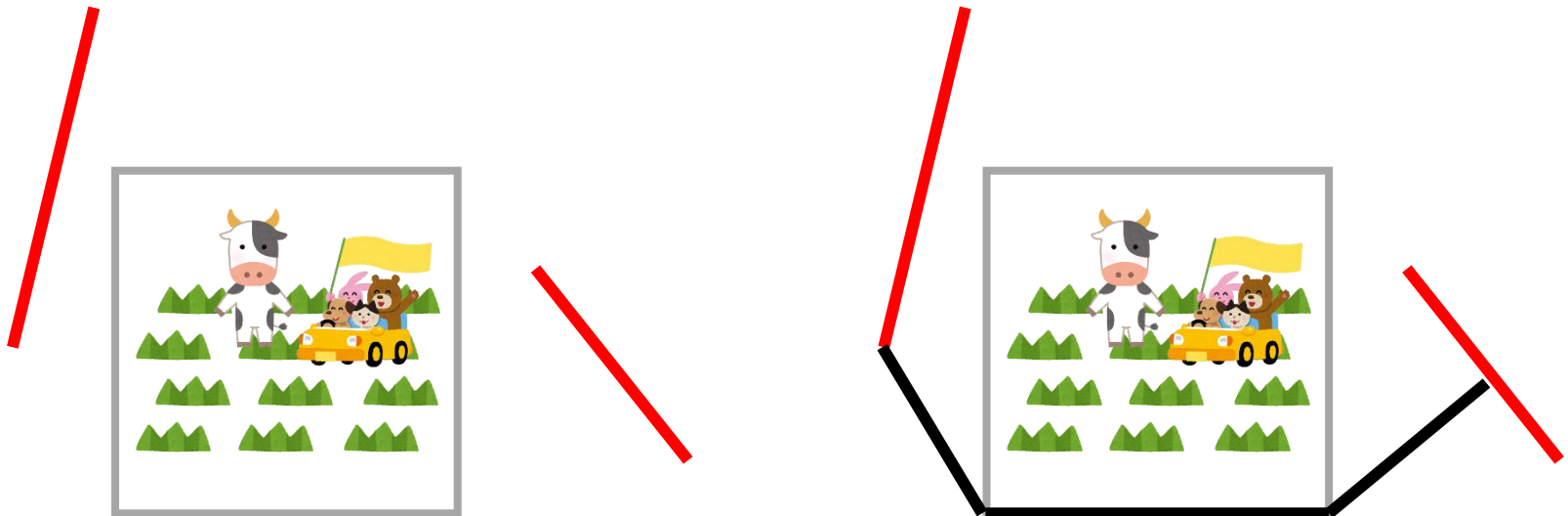
柵 $\rightarrow$ 柵ではなく、端点 $\rightarrow$ 端点で考えるとよい

- 柵ごとに、どちらの端点を基準にするかを固定



最短経路で考えるべき点

最短経路は折れ線になるが、その途中で折れる点としては「牧草地の四隅」以外は考えなくてよい



最短経路に含まれる経路

- 柵と柵(線分と線分)の最短経路
- 柵(線分)と、牧草地の隅(点)の最短経路
- 牧草地の隅同士の最短経路

これらを求めておいて、牧草地の内部に入るものは除外する.

また、各経路が m と交わるかどうか覚えておく.

柵と柵の間の最短距離

- 通る点の順番を全探索で試せば十分
- m との交差回数を求めつつ長さを求める
- 交差回数の偶奇のそれぞれについて、最短距離が求まる
- 最短路問題にして Dijkstra などをしてよい

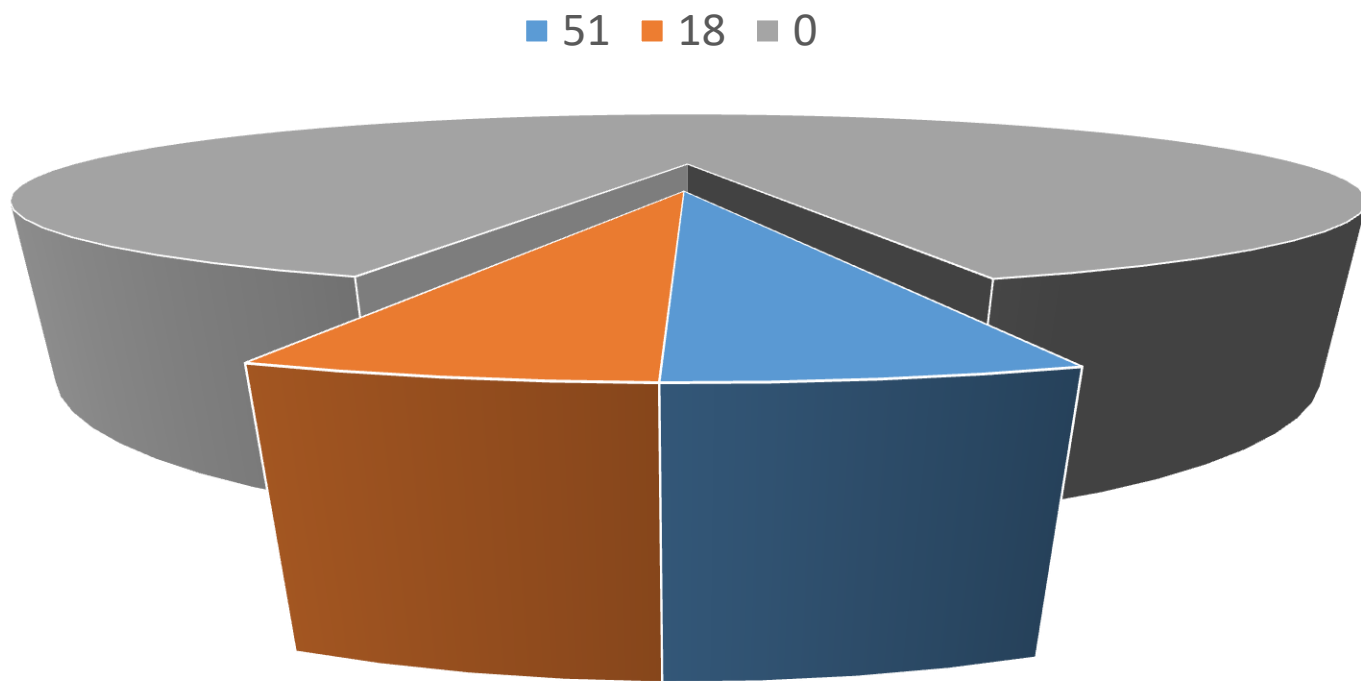
牧草地を囲う経路

- 柵のペア, m との交差回数の偶奇ごとに, 最短距離が求まった
- 柵を使う順番, 各移動での交差回数の偶奇を決めて全探索できる
 - $O(n! \cdot 2^n)$

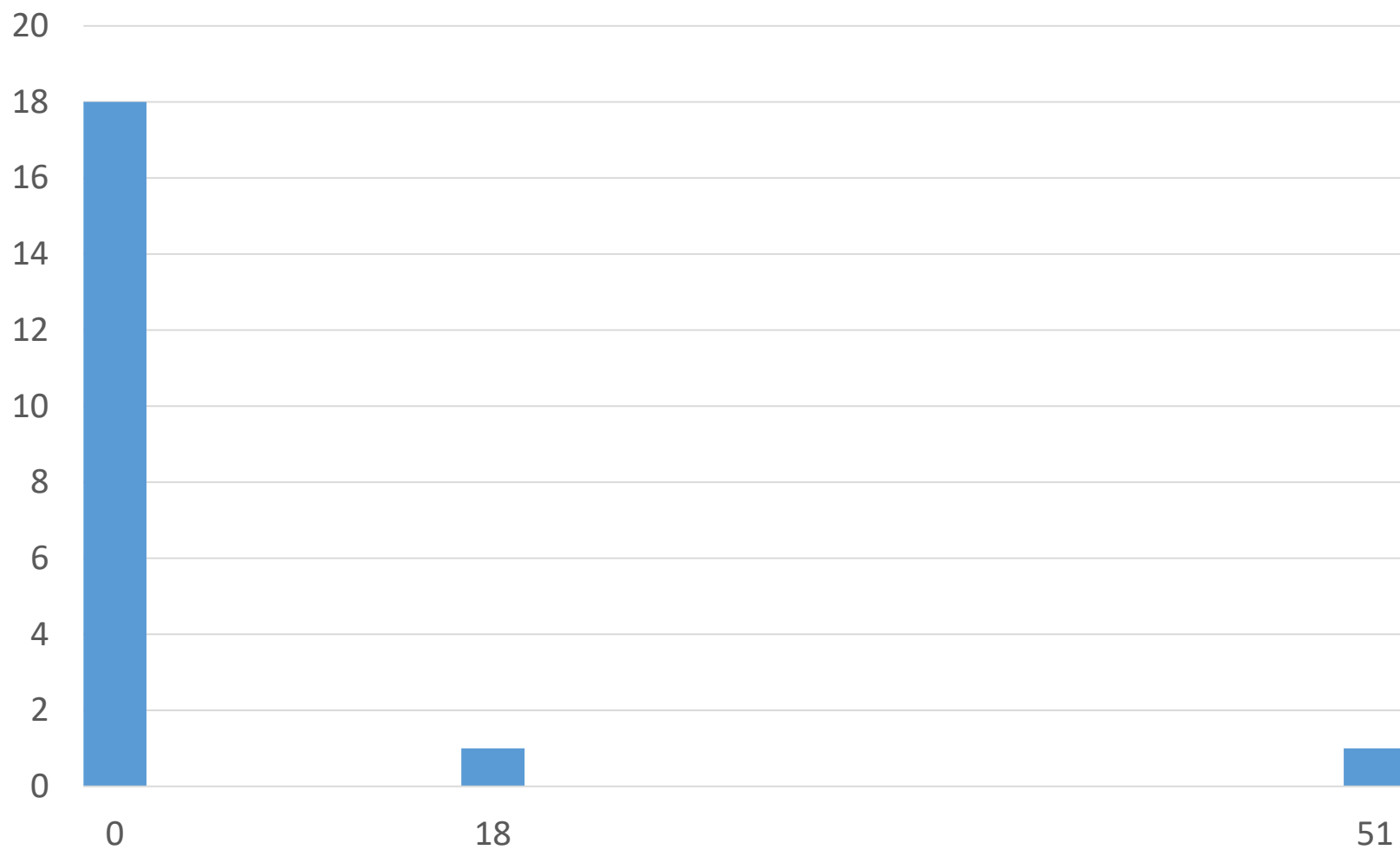
小課題 3

- $n \leq 100$
- 全探索 $O(n! \cdot 2^n)$ とかは間に合わない
- 通る柵を 1 つ決めると, そこから「 m と奇数回交わって同じ柵に戻ってくる」パスを求めればよいことがわかる
- (柵, 交差回数の偶奇) でグラフを作って最短路
- Warshal-Floyd など $O(n^3)$

得点分布

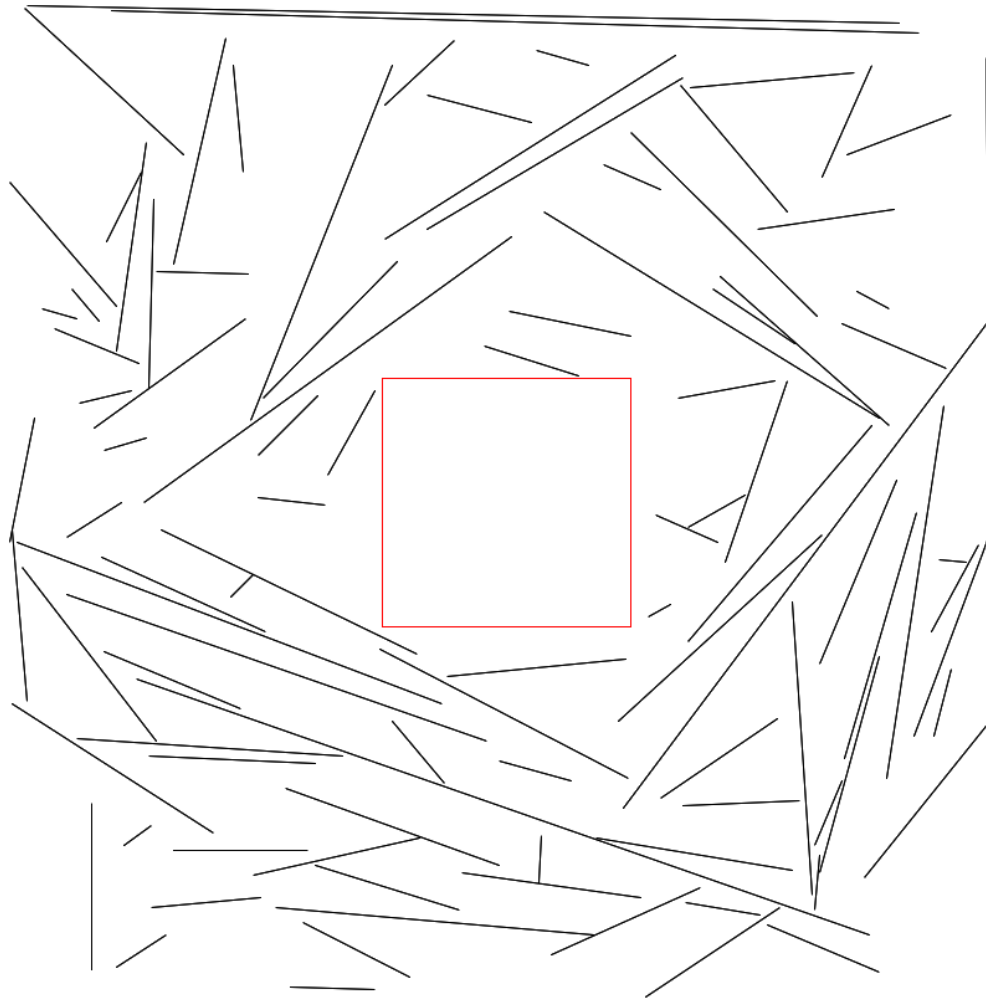


得点分布



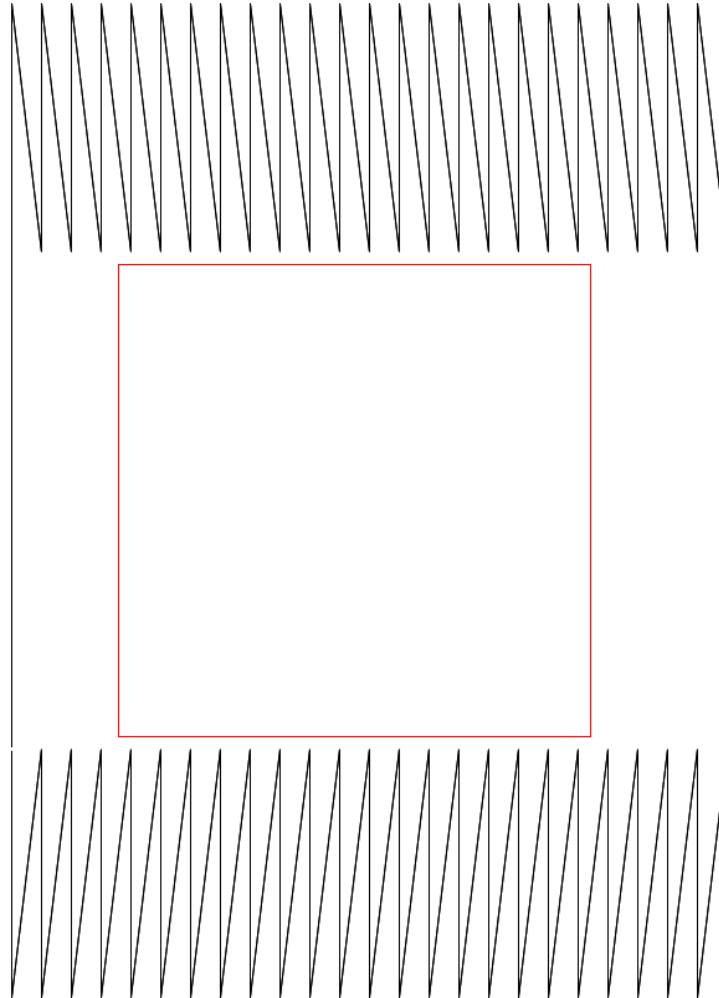
おまけ

- 03-02



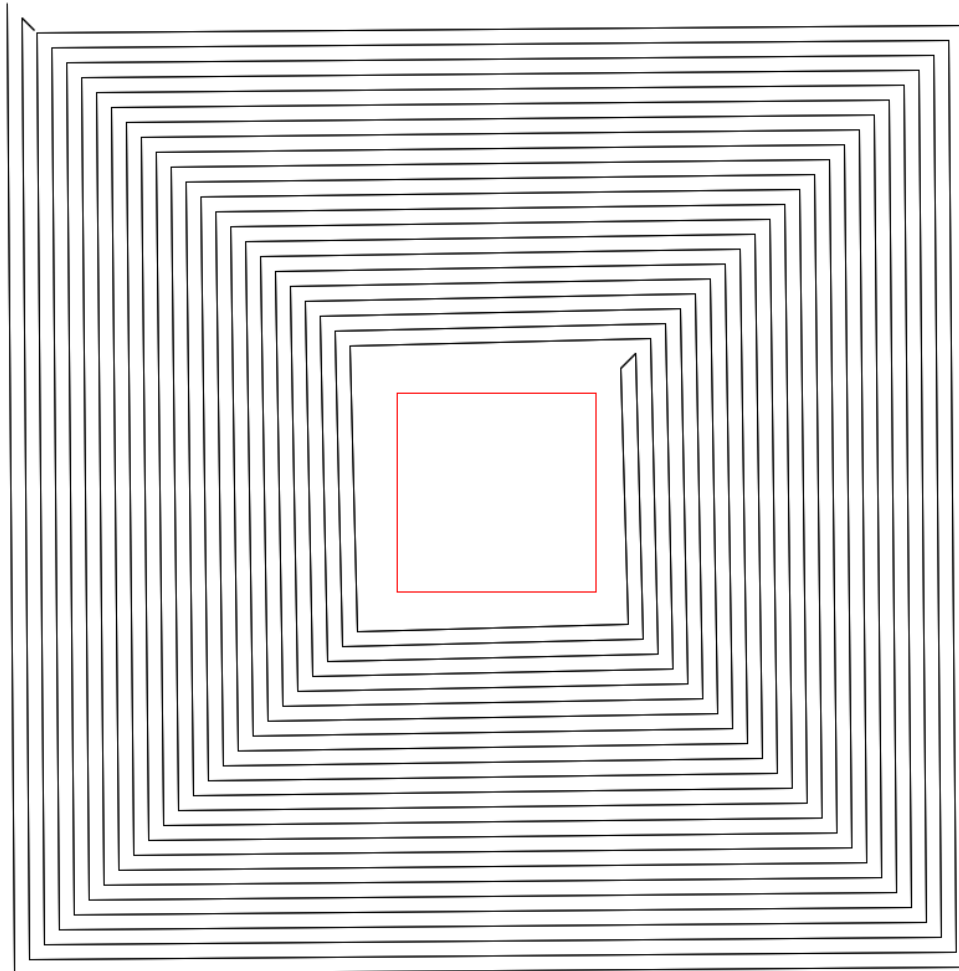
おまけ

- 03-21



おまけ

• 03-24



JOI 2017/2018 春合宿 Tents 解説

三谷庸

問題概要

- $H \times W$ のグリッドに何張かのテントを設営する
- テントは 4 方向のどれかに出入り口を持つ
- 同じ行・同じ列の 2 張のテントの出入り口は向かい合っていないといけない
- テントを 1 張以上設営する方法の個数を求めよ (mod $10^9 + 7$)
 - 小課題 1: $H, W \leq 300$
 - 小課題 2: $H, W \leq 3\,000$

記法

- テントを出入り口の向きによって ↓、↑、←、→ で表す

基本的な考察

各行は

- テントが 1 張以下
- テントが 2 張あり、左にあるものは→、右にあるものは←
のいずれか

列についても同様

解法

- 数学？ → 小課題 1
- DP？ →
 - 下手な DP → 小課題 1
 - うまい DP → 小課題 1, 2(満点)

数学っぽい解法 ($O(N^3)$, 48 点)

設営方法が

- $\leftarrow$ 、 $\rightarrow$ を両方置く行は i 行ある
 - $\uparrow$ 、 $\downarrow$ を両方置く列は j 列ある
 - テントをひとつだけ置く行、列は k 行 k 列ある
- という条件を満たすときの設営方法の個数が求まる
(適当に前計算しておくで $O(1)$ 時間、詳細略)
- i, j, k をすべて試すことで $O(N^3)$ 時間

DP による解法 ($O(N^3)$, 48 点)

- 上の行からテントの置き方を決めていく
- 現在の行に置けるテントは
 - テントが置かれていない列については、任意
 - ↓のテントが置かれている列については、↑のみ
 - それ以外の列については、何も置けない
- DP のキーとして、
 - 空いている行数
 - 空いている列数
 - ↓のテントが置かれている列数をもっておけばよい

DP による解法 ($O(N^3)$, 48 点)

$dp[i][j][k] =$

空行の個数 i , 空列の個数 j , $\downarrow$ だけ置かれた列の個数 k のとき
今の行に

- 何も置かない: $dp[i - 1][j][k]$
- $\downarrow$ を置く: $j \times dp[i - 1][j - 1][k + 1]$
- $\uparrow$ を置く: $j \times dp[i - 1][j - 1][k] + k \times dp[i - 1][j][k - 1]$
- $\leftarrow$ 、 $\rightarrow$ を 1 つ置く: $2j \times dp[i - 1][j - 1][k]$
- $\leftarrow$ 、 $\rightarrow$ を 2 つ置く: $\binom{j}{2} \times dp[i - 1][j - 2][k]$

DP による解法 ($O(N^2)$, 100 点)

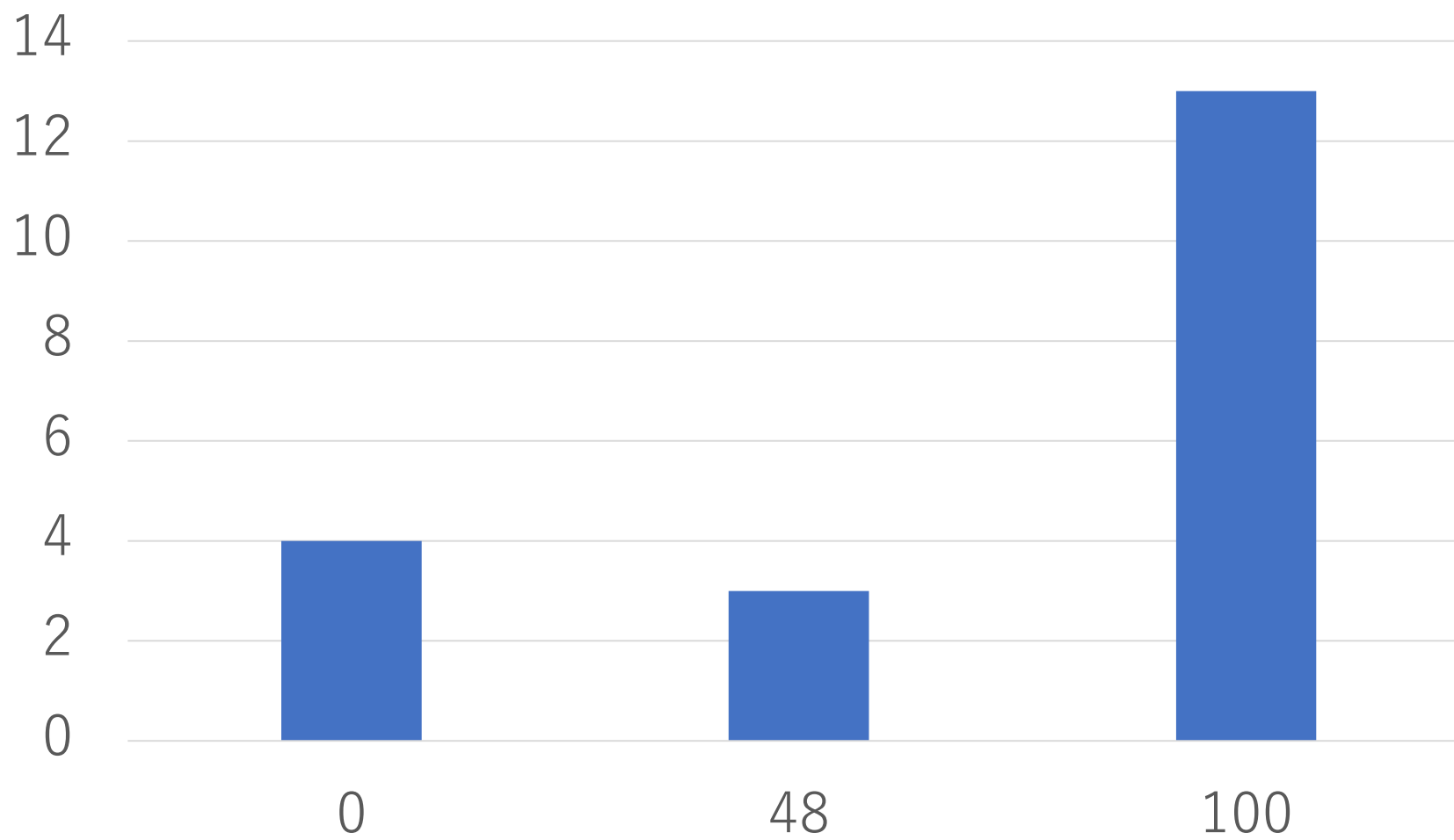
- $O(N^3)$ の DP を高速化する
- ↓だけ置かれた列の個数の情報を持たずに済ませたい
- ↓を置いた列は、
 - あとで↑を置く
 - 他に何も置かないのいずれか
- ↓を置くとき、同じ列に↑を置くかどうかあわせて考える

DP による解法 ($O(N^2)$, 100 点)

$dp[i][j]$ = 空行の個数 i , 空列の個数 j のとき
今の行に

- 何も置かない: $dp[i - 1][j]$
- テントを 1 張置き、それと同じ行、列には何も置かない
 $4j \times dp[i - 1][j - 1]$
- $\downarrow$ を置き、同じ列に $\uparrow$ を置く: $(i - 1)j \times dp[i - 2][j - 1]$
- $\leftarrow$ 、 $\rightarrow$ を両方置く: $\binom{j}{2} \times dp[i - 1][j - 2]$

得点分布



Asceticism 解説

解説: tozangezan



問題概要

- $N!$ 通りの長さ N の順列を考える
- 可能な限り少ない連続する増加列に切り分けると、 K 個の連続する増加列になるものは何通り？

とりあえず、言い換え

- 要は、
- 「 $a_i > a_{i+1}$ 」となるのが $K-1$ 箇所ある順列は何通りあるか求めればよい
- 小課題1: $N!$ 通りの順列全てに関して $a_i > a_{i+1}$ となる箇所を数えて判定
- 以下、 $a_i > a_{i+1}$ という関係性を「 $\leftarrow$ 」という記号で表します

とりあえず典型的な解決を

- DP で解けるか？
- 解けそう
 - 「←の個数が i 個以上」となる順列の個数は、簡単に求められそう
 - $dp[i][j] :=$ (左から i 要素で、←でないといけな場所がもう j 個登場したときの場合の数)
 - これがわかれば包除原理で計算できそう
 - 慣れてないとややこしい
 - 「←の個数が i 個以上」から「←の個数がちょうど j 個」($i < j$)となるものの個数に ${}_jC_i$ をかけたものを引いていけばよい
 - なんで ${}_jC_i$ をかける？ → 上のDPでは「←の個数がちょうど j 個」が ${}_jC_i$ 回重複して登場しているから

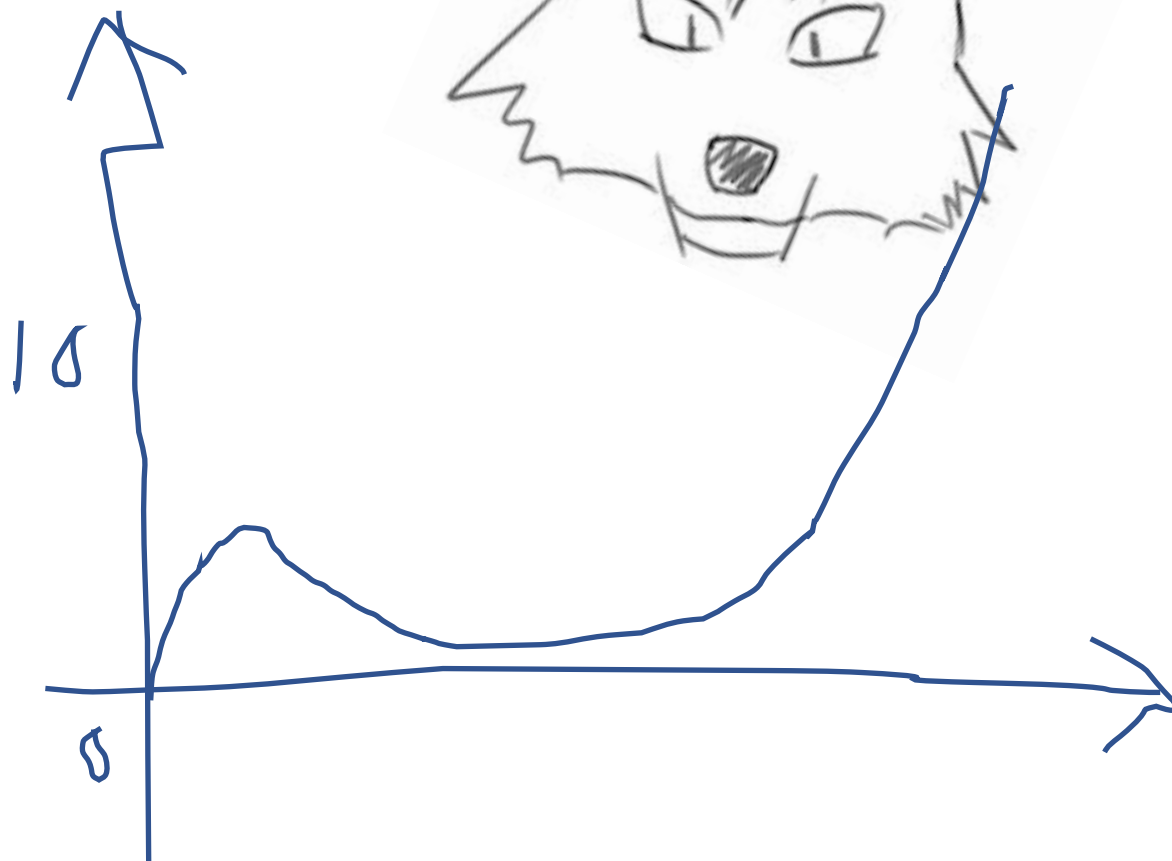
計算量は？

- 本当は $O(N^2)$ にしたいが、DPの更新がきつい
 - 今見ているところから、 $\leftarrow$ でないものを1個追加 ($dp[i+1][j]$ に $dp[i][j]$ を足す)、もしくは連続する k 個の $\leftarrow$ を追加 ($dp[i+k+1][j+k]$ に $dp[i][j]/(k+1)!$ を足す) という更新になる。
 - 係数が複雑なので、複数の更新をまとめることはできない
- 仕方なく $O(N^3)$
 - 小課題 2

かしこいDP

- 挿入 DP がかなりうまくいく
- $dp[i][j]$: $N = i, K = j$ のときの問題の答え
- $N = i, K = j$ となる順列に整数 $i+1$ を追加すると、
 - 追加できる $i+1$ 箇所のうち、 j 箇所($\leftarrow$ の真ん中、最後)に追加すると K は増えない
 - 残りの $i-j+1$ 箇所に追加すると K は 1 増える
- $dp[i][j] = dp[i-1][j] * j + dp[i-1][j-1] * (i-j+1)$
- $O(N^2)$ なので、これなら小課題 3 も解ける

得点分布



スライドショーの最後です。クリックすると終了します。



목  란

$N = 100,000$

DPの限界

- DPで状態数を $O(N^2)$ より少なくできそうに見えますか？
- $dp[i][j]$: i 個の整数、 j 個の←
- …無理そう

DPから離れよう！

DPから離れよう！

+

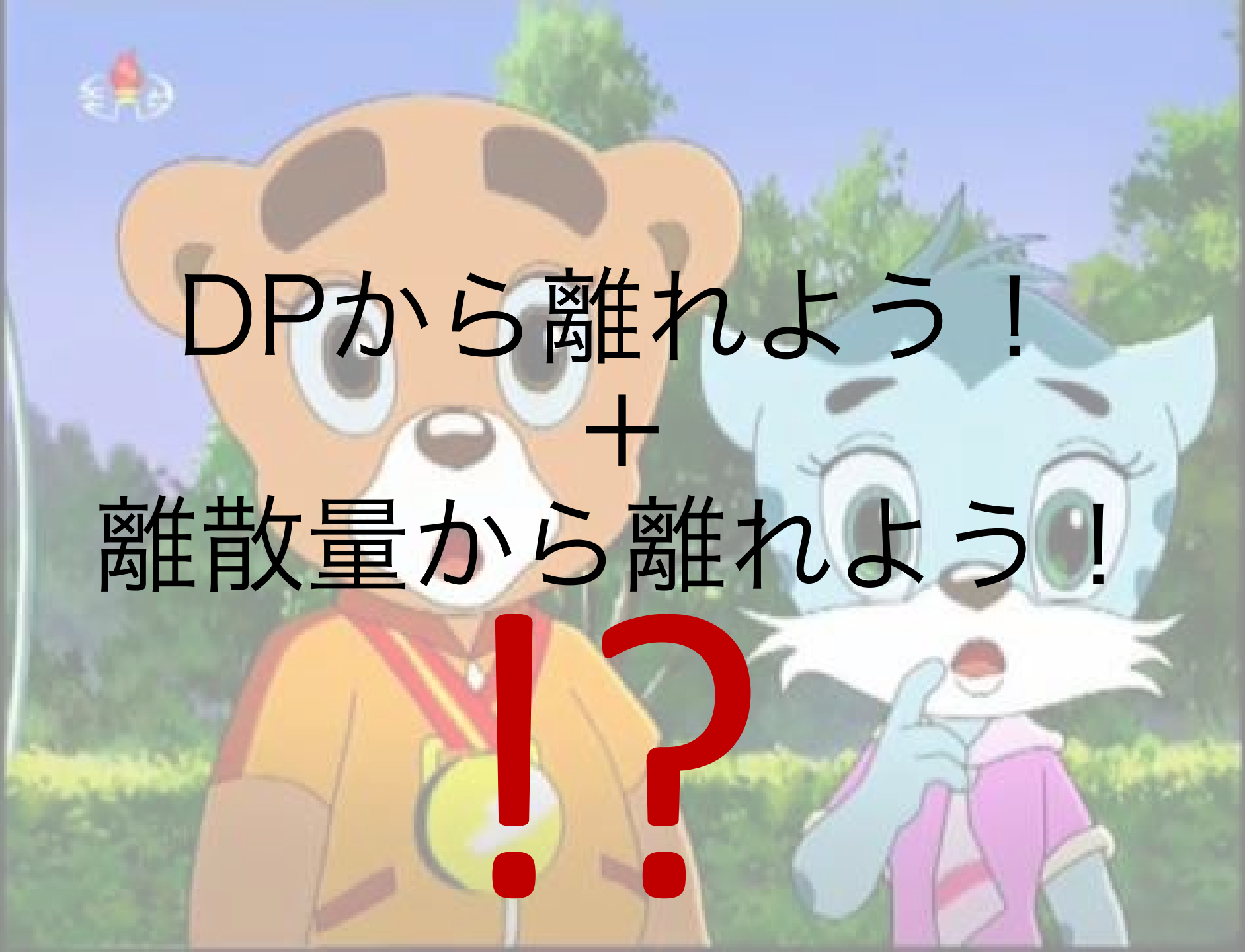
DPから離れよう！

+

離散量から離れよう！



DPから離れよう！
＋
離散量から離れよう！
！！



数え上げと確率

- 「全部の～のうち、～なものは何通りか？」という問題と「ランダムに～を一つ取ってきた。～な確率はいくらか？」という問題は実質同じ
- 掛け算すればよい
- 「ありうる全部の S について、 $f(S)$ の合計を求めよ」みたいなのも「期待値はいくらか？」という問題と同じ

問題例

- New Year Contest 2015

J- ランダムウォーク

実行時間制限: 4 sec / メモリ制限: 512 MB

Problem Statement

無限に広い二次元のマス目がある。このマス目の座標は二つの整数 i, j を用いて (i, j) と表せる。

すぬけ君は、 $(0, 0)$ から出発し、 N 歩ランダムウォークを行う。 (i, j) にいるとき、一歩進むとそれぞれ $1/4$ ずつの確率で $(i - 1, j), (i, j - 1), (i, j + 1), (i + 1, j)$ に進む。

すぬけ君が N 歩ランダムウォークを行ったとき、一回以上訪れたマス目の個数の期待値を E とする。 $E \times 4^N$ を M で割ったあまりを求めよ。ただし、 $(0, 0)$ は常に一回以上訪れたマス目に含まれるものとする。

問題例

• ARC061 by



F - 3人でカードゲーム / Card Game for Three [問題を編集](#)

実行時間制限: 3 sec / メモリ制限: 256 MB

配点: 1100 点

問題文

A さん、B さん、C さんの 3 人が以下のようなカードゲームをプレイしています。

- 最初、3 人はそれぞれ '[a](#)'、'[b](#)'、'[c](#)' いずれかの文字が書かれたカードを、A さんは N 枚、B さんは M 枚、C さんは K 枚持っている。3 人は、持っているカードを並べ替えることはできない。
- A さんのターンから始まる。
- 現在自分のターンである人がカードを 1 枚以上持っているならば、そのうち先頭のカードを捨てる。その後、捨てられたカードに書かれているアルファベットと同じ名前の人 (例えば、カードに '[a](#)' と書かれていたならば A さん) のターンとなる。
- 現在自分のターンである人がカードを 1 枚も持っていないならば、その人がゲームの勝者となり、ゲームは終了する。

3 人が最初に配られるカードに書いてある文字の並びは、全部で 3^{N+M+K} 通りの組み合わせがあります。このうち、A さんが勝者となってゲームが終了するのが何通りあるかを求めてください。

答えは大きくなる可能性があるので、 $1\,000\,000\,007 (= 10^9 + 7)$ で割った余りを出力してください。

ただし、

- さっきの2問は別にこの言い換えが本質ではない

ただし、

- さっきの2問は別にこの言い換えが本質ではない

- 今回は**本質**

確率の問題になった

- ランダムに1個順列を取ってきたときに、 $a_i > a_{i+1}$ となる個数が $K-1$ 個である確率は？

上と下は同じ

- $[0,1)$ の乱数を N 個取ってきたときに、 $a_i > a_{i+1}$ となる個数が $K-1$ 個である確率は？
- 以下、この $[0,1)$ の乱数列を $x_1, \dots, x_N$ とよぶことにする

式変形

- $x_1, \dots, x_N$ のうち $K-1$ 箇所が $x_i > x_{i+1}$
- $x_1, \dots, x_N$ が別の $[0, 1)$ の実数列 $y_1, \dots, y_N$ の累積和の小数部分だと思ふ
- $\{x_i\}$ と $\{y_i\}$ は1対1対応している
- $x_i > x_{i+1}$ のときは整数部分が繰り上がっている
- $x_i \leq x_{i+1}$ のときは整数部分が繰り上がっていない
- と考えると、元の条件は、 x_N の(隠された)整数部分が $K-1$ であることと同値
- $\rightarrow K-1 \leq \sum y_i < K$

あとはもう簡単

- これ



あとはもう簡単

- これ



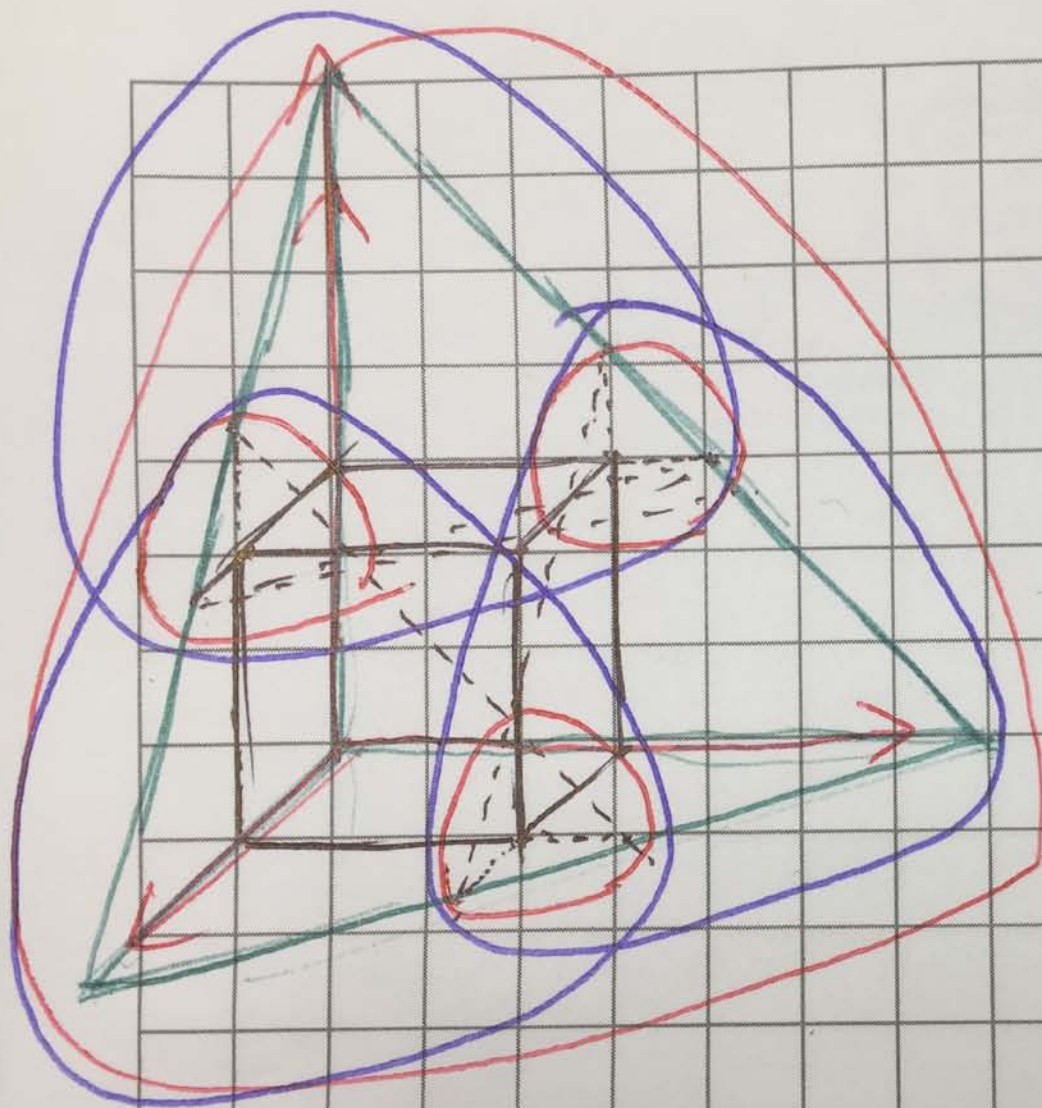
あとはもう簡単

- $y_1, \dots, y_N$ が $[0,1)$ を動くとき、 $K-1 \leq \sum y_i < K$ となる確率は？
- $\sum y_i < K$ となる確率から $\sum y_i < K-1$ となる確率を引けばよい

あとはもう簡単

- $y_1, \dots, y_N$ が $[0,1)$ を動くとき、 $\sum y_i < K$ となる確率は？
- 包除原理で求められる
 - $y_1, \dots, y_N$ のうち i 個が1以上で $\sum y_i < K$ となる範囲の大きさは？
 - $\sum t_i < K-i$ となる $\{t_i\}$ ($t_i \geq 0$) が動ける範囲の大きさ
 - 上で求めたものを使って、適当な係数 $(-1)^i ({}_N C_i)$ をかけて引いていく

イメージ図



三角錐と立方体の
共通部分の体積

○ : +1倍

○ : -1倍

やっとゴールだ

- $\sum t_i < K-i$ となる $\{t_i\}$ ($t_i \geq 0$) が動ける範囲の大きさはいくつ？
 - $(K-i)^N / (N!)$ ($K-i > 0$ のとき)
- なぜ？
 - 1. 多重積分する (シラバス外ですが…)
 - 2. 数直線の $[0, K-i)$ 上に N 点置くイメージ
 - t_i は $i-1$ 番目の点の座標と i 番目の点の座標との差に対応
 - $t_i \geq 0$ なので、置いた点が左から順に並んでいる必要がある
 - ということは $(K-i)^N / (N!)$

まとめ

- 何ができればいいのか？
 - 階乗 (前計算 $O(N)$, 本計算 $O(1)$)
 - 階乗の逆元 (前計算 $O(N)$, 本計算 $O(1)$)
 - コンビネーション (階乗と階乗の逆元でOK)
 - 累乗 (毎回 $O(\log N)$)

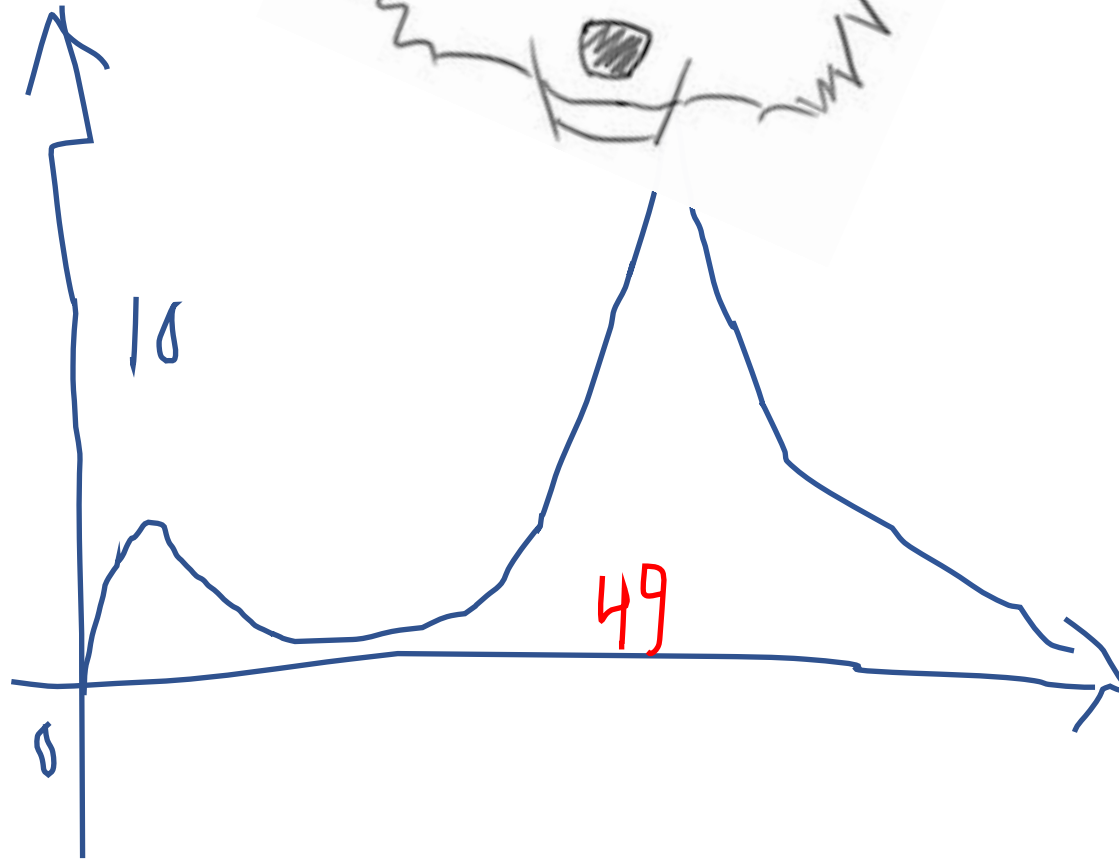
まとめ

- 何ができるばいいか？
 - 階乗 (前計算 $O(N)$, 本計算 $O(1)$)
 - 階乗の逆元 (前計算 $O(N)$, 本計算 $O(1)$)
 - コンビネーション (階乗と階乗の逆元でOK)
 - 累乗 (毎回 $O(\log N)$)
- あわせて $O(N \log N)$
- 100点

得てほしいもの

- 問題の種類によっては、数え上げ問題と確率問題は表裏一体
- 一対一対応による変換が、数え上げ・確率問題の解決手段としてはかなり有用 (AGC013 Eとか)
- 一対一対応の一つとしては、累積和・差分は有名 (AOJ2617: Air Pollutionとか)

得点分布



余談 (oeis.org)

A008292 Triangle of Eulerian numbers $T(n,k)$ ($n \geq 1, 1 \leq k \leq n$) read by rows. 275

1, 1, 1, 1, 4, 1, 1, 11, 11, 1, 1, 26, 66, 26, 1, 1, 57, 302, 302, 57, 1, 1, 120, 1191, 2416, 1191, 120, 1, 1, 247, 4293, 15619, 15619, 4293, 247, 1, 1, 502, 14608, 88234, 156190, 88234, 14608, 502, 1, 1, 1013, 47840, 455192, 1310354, 1310354, 455192, 47840, 1013 ([list](#); [table](#); [graph](#); [refs](#); [listen](#); [history](#); [text](#); [internal](#))

エ●カちゃん:

日頃からOEISを丸暗記していれば、こんな問題余裕だピ。

これこそ真のasceticismだピね。

Pranab Kumar Khundunr (pranabk@MIT.edu), Jul 07 2004

$T(n,k)/n!$ also represents the n -dimensional volume of the portion of the n -dimensional hypercube cut by the $(n-1)$ -dimensional hyperplanes $x_1 + x_2 + \dots + x_n = k$, $x_1 + x_2 + \dots + x_n = k-1$; or, equivalently, it represents the probability that the sum of n independent random variables with uniform distribution between 0 and 1 is between $k-1$ and k . - Stefano Zunino, Oct 25 2006

2018年春合宿 Day2 Road Service

チューター 劉

問題概要

- N頂点の木が与えられる
- K本の辺を新たに追加し，総距離を最小化
 - 総距離=2点間の最短距離の総和
- $N=1000$
- OutputOnly

問題概要

- N 頂点の木が与えられる
- K 本の辺を新たに追加し，総距離を最小化
 - 総距離=2点間の最短距離の総和
- $N=1000$
- **OutputOnly !**

OutputOnly

- 入力を見て作業できる
 - 特徴が分かればその入力に特化してもよい
 - ビジュアライザもしばしば有効
 - 手で解いてしまってもよい
- 厳密解を出せなくてもよい
 - だいたい上手いきそうな貪欲・DPで高いスコアが出せることも
 - 初期状態を決めて山登り

OutputOnly

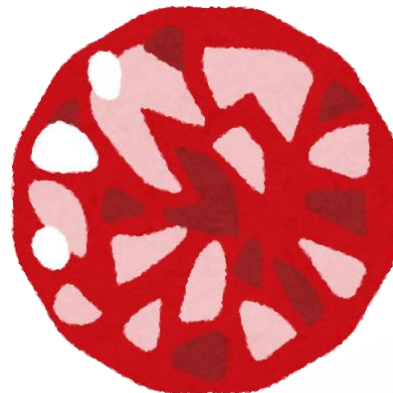
- 5時間中に終わられることが重要
 - さらに，5時間掛けるわけにもいかない
 - 終わりがないので最後に回す？
 - バックグラウンドで回すために序盤にやる？
 - なるべく短い時間でほどほどの得点を出し，残りの問題に時間をかけたい
 - 楽な方針をまずは探そう
- 何度も提出できる
 - 細かいパラメータ変更を行えるよう設計したい

OutputOnly

- その他，細かいこと
 - (今のパソコンはマルチコアなので) 例えば端末を複数開き同時実行
 - 実行時間制限はないので，山登りや探索をひたすら繰り返し，解が更新されるたび出力
 - 標準出力のバッファがあふれることも→リダイレクトしてファイルを開いて切り貼り

```
./a.out < in > out
```

- C++ではなくスクリプト言語を用いてもよい



方針

- 最小化したいのは全点間の最短距離の総和
- 全点間の最短距離の総和は，木について
 - 直線だと N^3 オーダー
 - ウニだと N^2 オーダー
- 1つの頂点に集中していると強い
 - "ハブ空港"を作るイメージ

方針

- 1つの頂点に集中していると強い
 - "ハブ空港"を作るイメージ
- 実は、無作為からの山登りも長い間続けると自然とハブを作りたがる
 - ただし、単純山登りしているとハブを作るまでが長すぎて難しい
- 単に頂点1-2, 1-3, ... 1-(K+1) と出力するだけで30点付近



方針

- ハブを決めたとして，どの頂点とハブを直接つなぐか？
- 総合的にどの頂点もハブに近いようにしたい
 - 単純な戦略の1つ→次数の高い頂点につなぐ
 - 他にも工夫の余地アリ
- ハブをN通り試し，最も良かったものを出力
- だいたい60点



方針

- ハブとつなぐ頂点の集合Xを，単に次数の大きい頂点にするのはさすがに雑
- この状態から山登りをしよう
- 山登りの一例:
 - Xから1点取り除き，1点追加する操作を1ステップで行ける近傍とする
 - 近傍のうち最もスコアのいいものに遷移
 - 近傍は $O(NK)$ 通り



方針

- 山登りで，評価関数を総距離とすると遅い; $O(N^2)$
- ハブから他の頂点への距離の総和で代用; $O(N)$
 - 実際に，すべての頂点対がハブを経由するなら大小を保つ
- これで近傍の広さと山登り回数が稼げる
- 初期状態を全ハブ試すのはやめて，その時点での評価
トップ10ぐらいに対して山登る→だいたい100点

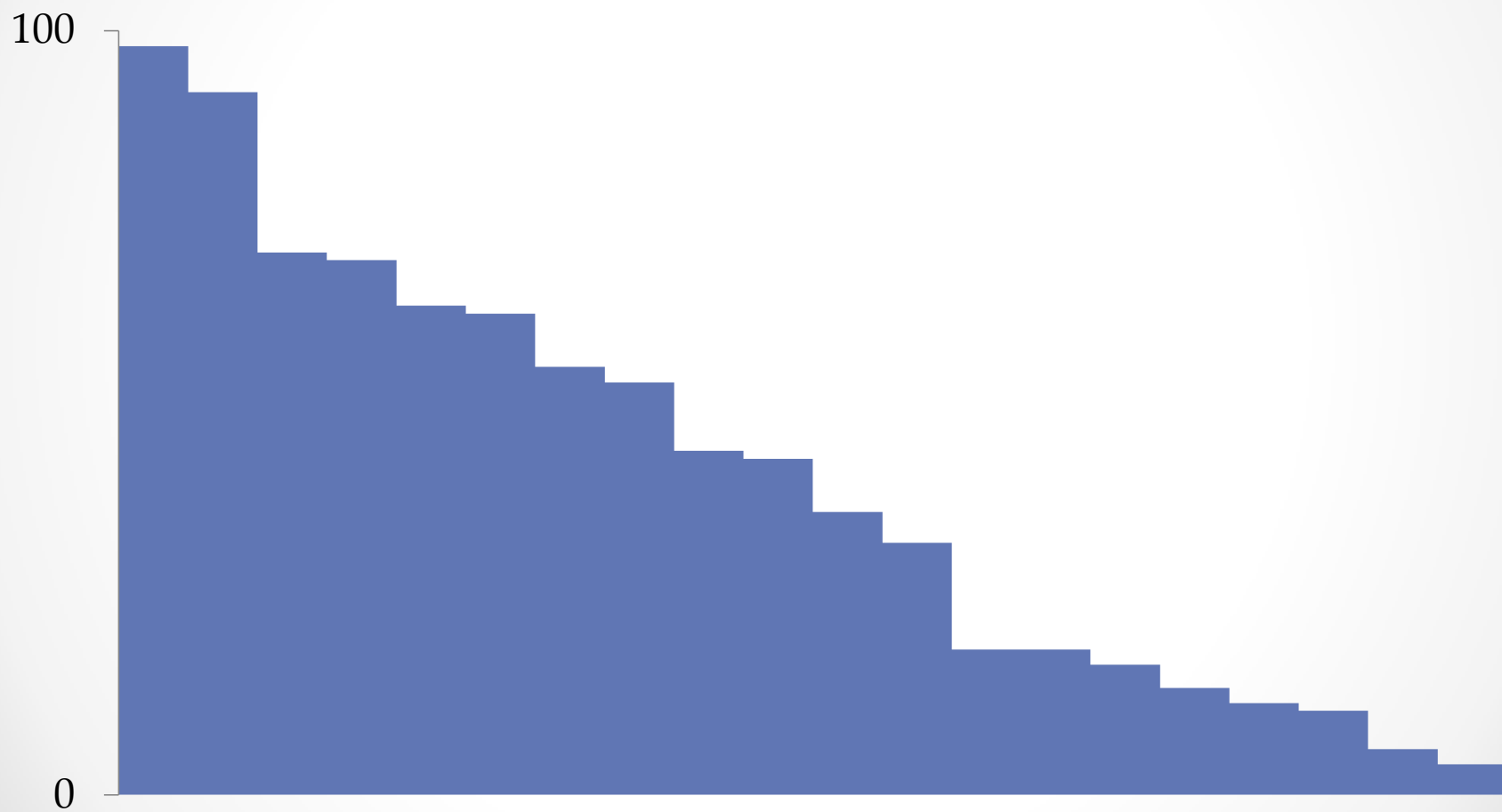


方針

- 次数の高い頂点につなぐ代わりに「最も総距離を減らす頂点に向けてハブから辺を貼る」をK回繰り返す
 - 山登りなしでいたい90点
 - 総距離の計算は先述の評価関数で近似



得点分布



Worst Reporter 3 解説

DEGWER

問題概要の前に.....

Worst Reporter(初代)

▶ JOI 2006-2007 本選 4

Worst Reporter(17/11)

- ▶ 舞台はクロアチア
- ▶ 一部の試合の結果から順位を勝手に推測する問題
- ▶ サイク要素: 試合結果を全部調べるのをサボった

4

最悪の記者

問題

あなたは JOI 新聞社の記者であり、スポーツ記事を担当している。

昨日までに、クロアチアでは、 n 個のサッカーチームによる総当りのリーグ戦が行われた。大会実行委員会は、試合結果と規定に基づき各チームに 1 位から n 位までの順位をつけたようである。あなたには、一部の試合の勝敗とともに、次の情報が伝えられた。

情報 1 引き分けの試合はなかった。

情報 2 全てのチームに異なる順位がついた。

情報 3 全ての $1 \leq a < b \leq n$ に対し、 a 位のチームと b 位のチームの試合において、必ず a 位のチームが勝利した。

あなたは記事を作成するために、一部の試合の勝敗と、伝えられた情報 1~3 をもとに、順位表を推測することにした。

入力として一部の試合の勝敗が与えられたとき、伝えられた情報に適合する順位表を 1 つ出力するプログラムを作れ。また、出力した順位表以外に、伝えられた情報に適合する順位表が存在するかどうか判定せよ。

ここで、順位表とは 1 位から n 位の順にチームを並べたもののことをいう。

Worst Reporter 2

▶ JOI 2015-2016 春合宿

Worst Reporter 2

- ▶ 不具合のある順位表に最も適合する順位表を求める問題
- ▶ サイアク要素:
むしろサイアクなのは
順位表を壊した運営では.....

最悪の記者 2 (Worst Reporter 2)

時は 21XX 年, 競技プログラミングはマインドスポーツの 1 つとして広く認知されており, テレビ, 新聞などのメディアで取り上げられることも多い.

あなたは JOI 新聞社の記者であり, 競技プログラミングの記事を担当している.

昨日, N 人の選手による国際的な競技プログラミングのコンテストが開催された. このコンテストについての記事を書くために, あなたには次の情報が与えられた.

- 国際情報オリンピックなどと同様, このコンテストにはいくつかの国から選手が参加した. 国には 1 から N までのいずれかの番号が付けられている. 一つの国から複数の選手が参加することもあり得る. また, 選手が参加しない国があるかもしれない.
- このコンテストの競技時間は 5 時間である.
- コンテスト中に選手の獲得した点数が, その後減らされることはない.
- コンテスト開始後 2 時間経過した時点において, 同点の選手はいなかった. その時点の順位表において, i 位 ($1 \leq i \leq N$) の選手は国 A_i の出身で, その選手の点数は B_i 点であった.
- コンテスト終了時点において, 同点の選手はいなかった. コンテスト終了時点の順位表において, i 位 ($1 \leq i \leq N$) の選手は国 C_i の出身で, その選手の点数は D_i 点であった.

しかしながら, 記事を書く段階になって, 順位表の出身国の表示に不具合があったことが判明した. 選手の出身国の情報が間違っ表示されていた可能性がある. 表示されていた選手の点数は正しいことが分かっている.

そこで, あなたは, 与えられた情報にできるだけ少ない修正を加えることで, 順位表の情報として矛盾のない (同じ選手の出身国がコンテスト中に変わったり, 選手の獲得した点数がコンテスト中に減少したりしない) ものを推測することにした. すなわち, $2N$ 個の値 $A_1, \dots, A_N, C_1, \dots, C_N$ のうちのできるだけ少ない箇所を変更することで, 次の条件を満たすようにしたい:

- $1, 2, \dots, N$ のある並び替え $x_1, x_2, \dots, x_N$ であって, 各 $i = 1, 2, \dots, N$ に対して $A_i = C_{x_i}$ かつ $B_i \leq D_{x_i}$ が成り立つものが存在する.

あなたは, 与えられた情報に, 最少で何箇所の修正を加える必要があるだろうか.

Worst Reporter 3

▶ JOI 2017-2018 春合宿

問題概要

- ▶ $N + 1$ 人の参加者が IOI ちゃんを先頭にして行進している
- ▶ IOI ちゃんは 1 秒に 1 すすむ
- ▶ 他の参加者は、前の人との距離が一定以上になったら前の人との距離を 1 まで詰める

Worst Reporter 3(今回)

- ▶ あなたは写真を撮らなければならなかったが、熟睡していた(サイアク)
- ▶ あなたは会場の写真を撮り、参加者の絵を描いて誤魔化すことにした(もっとサイアク)
 - ▶ なぜバレないと思ったのか

Worst Reporter 3(今回)

- ▶ 以下のクエリ Q 個に答えよ:
- ▶ 時刻 T に、区間 $[l, r]$ に人は何人いるか？
- ▶ $1 \leq N, Q \leq 500000$
 - ▶ 選手 0 人とかにはならない
- ▶ 座標とか諸々は大きい

サンプル 1

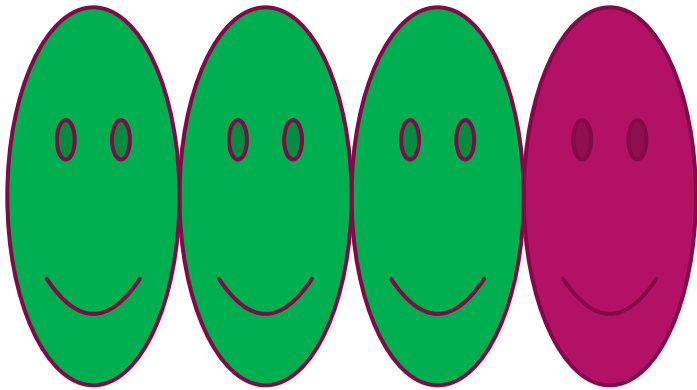
呑気さ

3

5

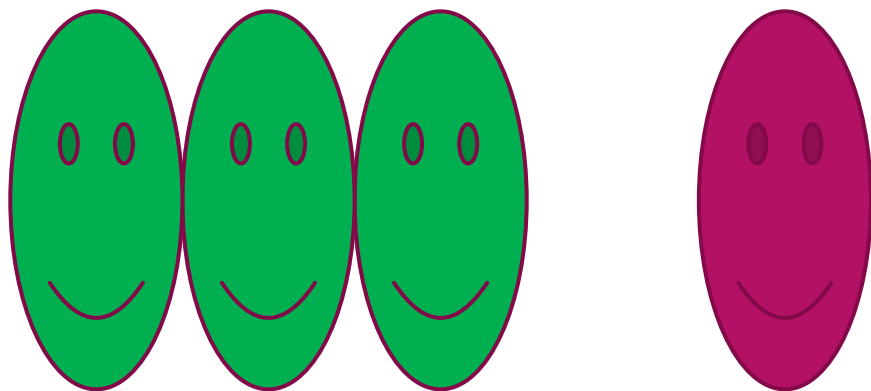
2

101



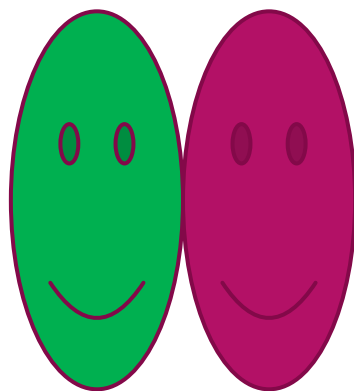
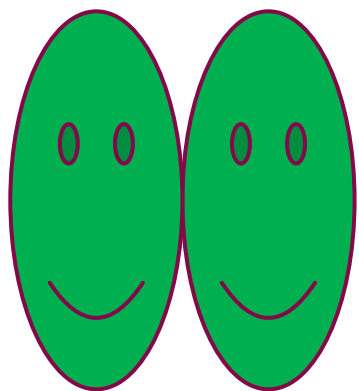
時刻 0

サンプル 1



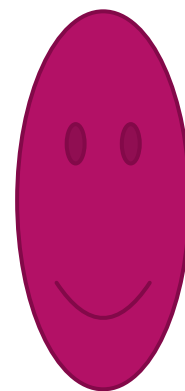
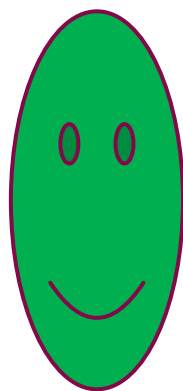
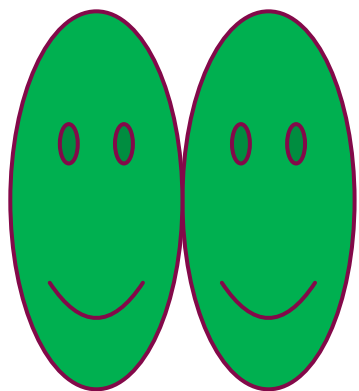
時刻 1

サンプル 1



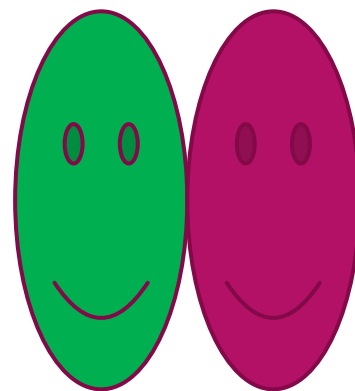
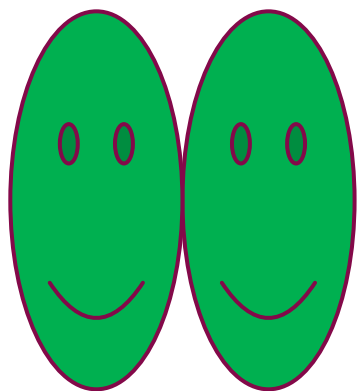
時刻 2

サンプル 1



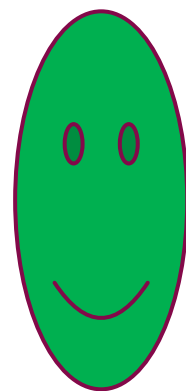
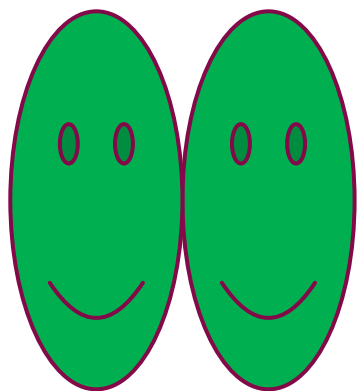
時刻 3

サンプル 1

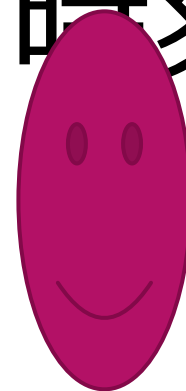


時刻 4

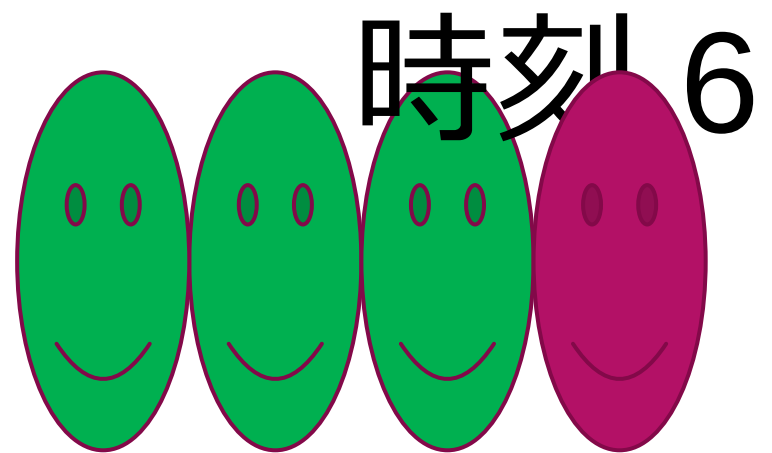
サンプル 1



時刻 5



サンプル 1



小課題 1

- ▶ 全ての選手の呑気さは 1
 - ▶ めっちゃピリピリしてそう
- ▶ つまり、全選手は前の人にぴったりくっついて進む
- ▶ 時刻 t における i 番目の選手の座標は $t - i$
- ▶ よって、各クエリに対して $[t - N, t]$ と $[l, r]$ の共通部分の長さを求めれば OK
- ▶ 7 点が得られる

小課題 2

- ▶ いろいろ小さい(雑)
- ▶ 愚直シミュレーションができるので、やってください
- ▶ 12 点が得られる

とりあえず、簡単のために.....

- ▶ 「前の人から距離 1 の位置まで進む」はわかりにくい
- ▶ 最初に i 番目の人の座標に i を足しておけば、前の人と同じ位置まで進むことができる
 - ▶ 最初はみんな位置 0 にいる

満点解法

- ▶ 各参加者は、どのタイミングで前に進むか？
- ▶ 実は、進み方はかなり規則的になっている
- ▶ 各参加者は時間間隔 X_i を持っており、 X_i の倍数の時刻に距離 X_i だけ進む、ということがわかる
 - ▶ 以下これを示す
 - ▶ とりあえず、IOI ちゃんについては $X_i = 1$

満点解法

- ▶ i 番目の選手について、 $X_{i-1} \geq D_i$ のとき
 - ▶ 0 番目の選手は IOI ちゃんとする
- ▶ X_i の倍数の時刻に、 i 番目の選手と前の選手との間の距離が D_i より大きくなるので、 $X_i = X_{i-1}$ となる

満点解法

- ▶ i 番目の選手について、 $X_{i-1} < D_i$ のとき
- ▶ 前の人が $\text{ceil}(D_i/X_{i-1})$ 回の距離 X_{i-1} の移動を行って初めて、 i 番目の選手と前の選手との間の距離が D_i より大きくなるので、 $X_i = X_{i-1} \times \text{ceil}(D_i/X_{i-1})$ となる
- ▶ 特に、 $X_i \geq 2X_{i-1}$ となる

満点解法

- ▶ X_i の値は、 i が増えると変わらないか、2 倍以上になる
- ▶ よって、 X_i が変化する回数は高々 $O(\log \text{呑気さ})$ 回
- ▶ 人を、 X_i の値で $O(\log \text{呑気さ})$ 個のグループに分類できる！

満点解法

- ▶ これがわかってしまえば、あとは小課題 1 と同じ
 - ▶ 座標を戻して考えれば、各ブロックに属する人の存在するのはある連続した区間になる
- ▶ $O(N + Q \log \text{呑気さ})$ 時間でこの問題が解けた

満点解法

- ▶ これがわかってしまえば、あとは小課題 1 と同じ
 - ▶ 座標を戻して考えれば、各ブロックに属する人の存在するのはある連続した区間になる
- ▶ $O(N + Q \log \text{呑気さ})$ 時間でこの問題が解けた

得点分布

100

19

* 航空路線図
(Airline Route Map)

井上卓哉

@yokozuna_57

- * Alice はラベルの付いたグラフを渡される.
- * Alice は Bob にラベルの付いたグラフを送る.
- * グラフのラベルがシャッフルされる.
- * Bob は送られたグラフから Alice が受け取ったもののグラフを復元する.

* 問題概要

- * Alice はラベルの付いたグラフを渡される.
- * Alice は Bob にラベルの付いたグラフを送る.
- * グラフのラベルがシャッフルされる.
- * Bob は送られたグラフから Alice が受け取ったもののグラフを復元する.
- * シャッフルされても壊れない情報を使いたい

* 問題概要

* N : 頂点数, M : 辺数 (渡されるグラフ)

* $1 \leq N \leq 1000$

* $0 \leq M \leq \frac{N(N-1)}{2}$

* V : 送るグラフの頂点数

* $1 \leq V \leq 1500$

* 制限

* $N \leq 10$

* 何をやってもいいから送りたい

* シャッフルされても壊れない情報

* 連結成分の個数, 次数, 葉の数とか

* 小課題1

- * 連結成分ごとの頂点数を使う
- * C 個の連結成分からなるグラフを用いて, C bit 分の情報を送ることを考える.
- * 連結成分の大きさを $S_1 \leq S_2 \leq \dots \leq S_C$ として, $S_1 - 1, S_2 - S_1, \dots, S_C - S_{C-1}$ で各 bit を表す.
- * 例えば, 001011 を送りたいければ, 連結成分の大きさは小さい順に 1, 1, 2, 2, 3, 4 とする.

* 小課題1

* 各頂点間に辺があるかをbit列に直し,
 $\frac{N(N-1)}{2}$ 個の連結成分を持つグラフでその
情報を送る.

* (i, j) ($1 \leq i < j \leq N$)を辞書順に並べ,
 (i, j) は $ord_{i,j}$ 番目にあるとする.

* 頂点 i と頂点 j の間に辺がある

$\Leftrightarrow ord_{i,j}$ 番目のbitは1

となるようにする.

* 小課題1

* 頂点の数は高々

$$1 + 2 + \dots + \frac{N(N-1)}{2} \leq 1035$$

となるのでAC

* 22点

* 小課題1

* $N \leq 40$

* もう少し賢く送りたい

* もとのグラフはそのまま送りたい

* どうやってラベルを付けるか

* 小課題2

- * ラベル付を次数1の頂点を使って行う
- * 頂点 i には $i + 2$ 個の次数1の頂点をつけておく
- * 頂点の数は

$$N + 2 + 3 + \dots + (N + 1) = \frac{N(N+5)}{2}$$

* AC 37点

* 小課題2

*追加の制限はない

*といいつつ, 採点基準の項を見ると,

$$V - N \leq 12$$

でなければ満点にはならない

*小課題3

* 賢くラベル付けしたい

* 2進数を使いたい

* 1, 2, 4, ..., 512 を表すラベル付け用の
頂点を用意する

* 小課題3

* 頂点の種類はどう区別するか？

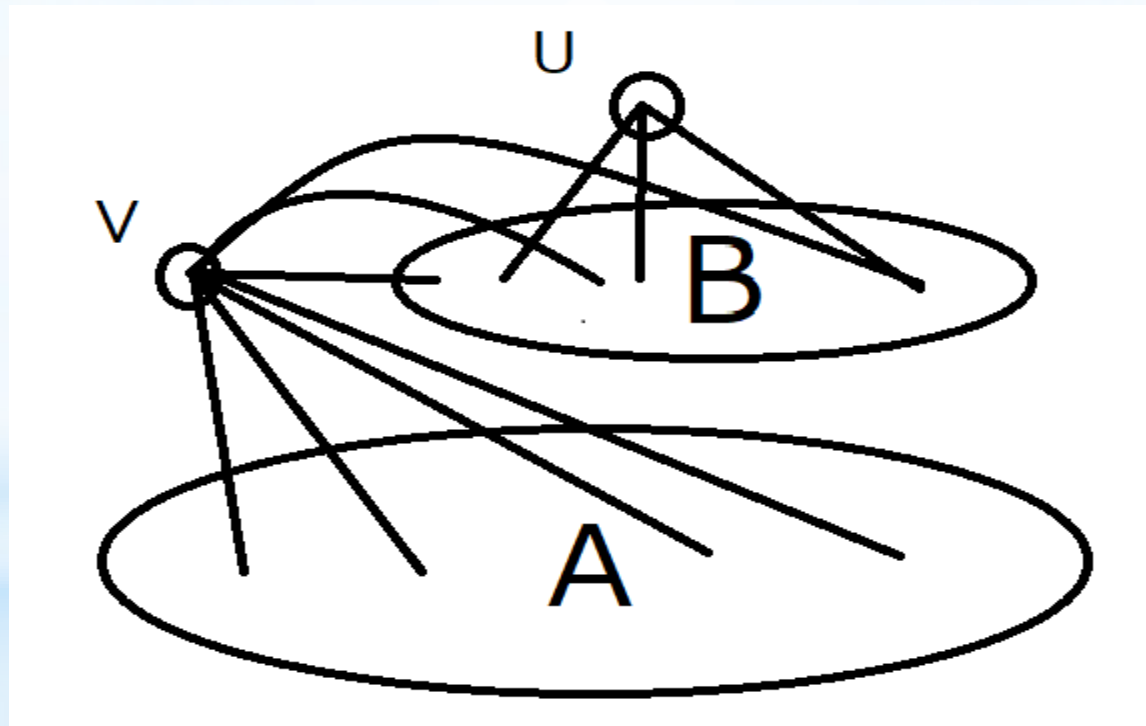
* 小課題2では各頂点の次数で区別した

* やっぱり次数を使いたい

* もとのグラフを表す頂点の集合を A ,
2 進数でラベルを表すための頂点の
集合を B とする.

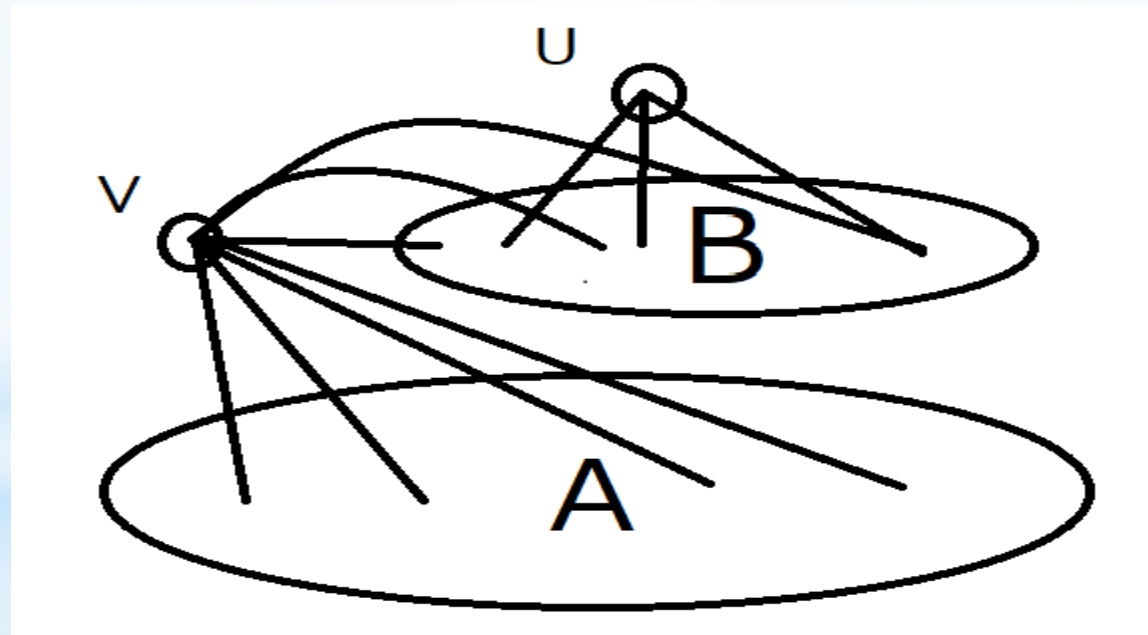
* 小課題3

- * 2つの頂点 V, U を用意する.
- * V は U 以外のすべての頂点と辺を張る
- * U は B に含まれる頂点すべてと辺を張る

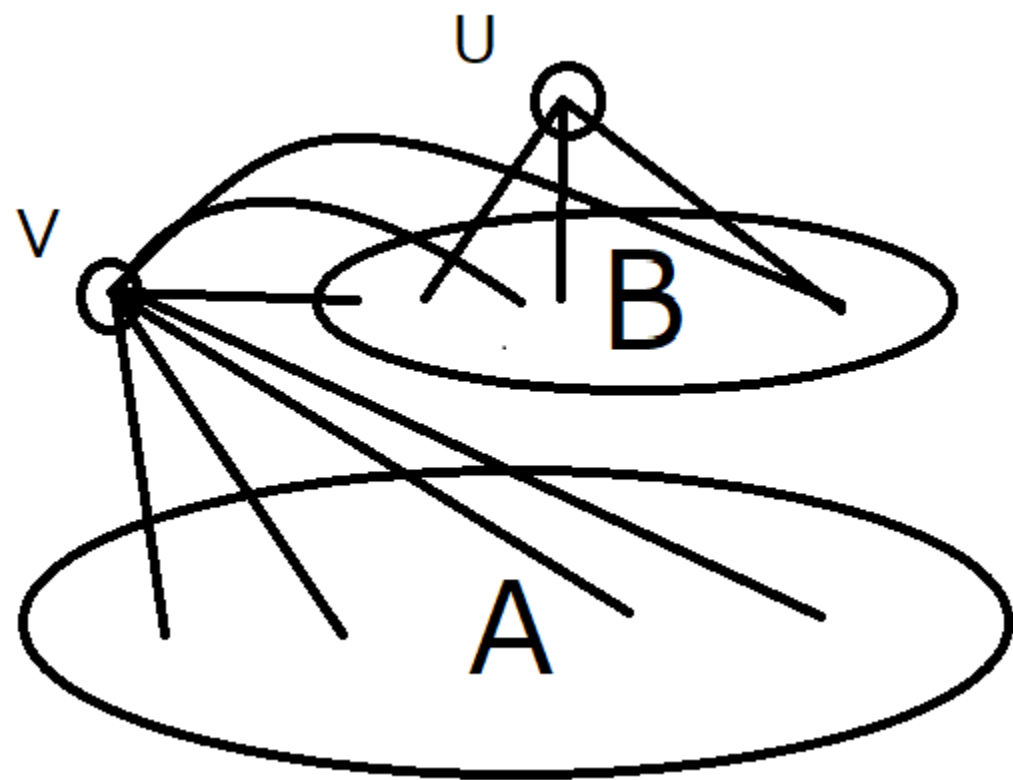


題3

- * V の次数は $N + 10$, U の次数は 10
- * A に含まれる頂点の次数は, 高々 $N + 9$
- * B に含まれる頂点の次数はより小さい
(後述)



題3



- * シャッフル後のグラフでも各頂点の次数は保存されるため, V は区別できる
- * V と直接つながっていない頂点として U が区別され, U と直接つながっているかで B に含まれる頂点が区別される.

* 小課題3

* A と B との間の辺は, A の頂点のものと
のグラフでの頂点番号を2進表示し
て, 対応する B の頂点と結ぶ.

* 例えば, 頂点 $57 = 1 + 8 + 16 + 32$ は,
 B の頂点のうち, $1, 8, 16, 32$ を表す頂
点と結ぶ.

* 小課題3

- * B の頂点同士の区別も必要
- * $1 - 2, 2 - 4, \dots, 256 - 512$ の間にそれぞれ辺を張る
- * 両端がどちらであるかは, 次数を見ればわかる. (1 の次数は $\left\lfloor \frac{N}{2} \right\rfloor + 3$, 512 の次数は $N - 509$)
- * ほかの B の頂点の次数も $\left\lfloor \frac{N}{2} \right\rfloor + 4$ 以下 ($N + 10$ より小さい)

* 小課題3

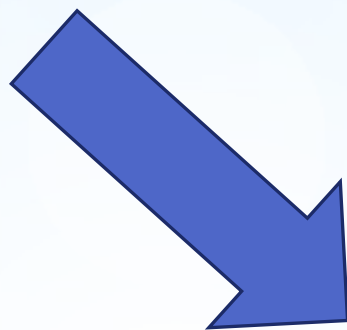
- * 乱択すると $V - N \leq 11$ にすることができる.
- * $P_1, P_2, \dots, P_N$ を適当に 0 か 1 に定める.
(これはAliceとBobで共有しておく.)
- * $Edge_{i,j}$ をもとのグラフの隣接行列として, $Edge_{i,j} \wedge P_i \wedge P_j$ を隣接行列とするグラフを小課題 3 と同じ方法で送る.

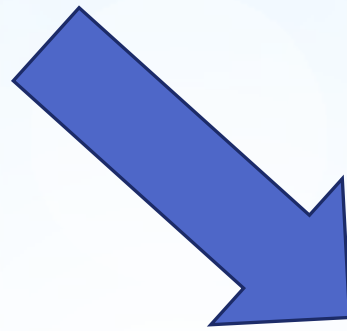
* 別解

* 小課題 3 では A と B とを区別するのに 2 つの頂点を用いたが, この場合は 1 つで区別できる. (A に含まれる頂点の次数が小さくなるため)

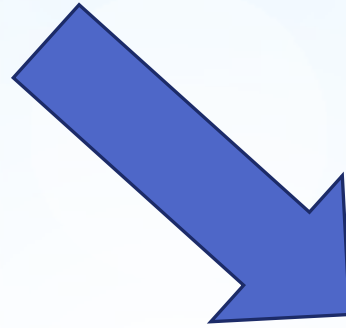
* A のすべての頂点とのみ辺でつながっている 1 つの頂点を使って区別できる. (N が小さい場合は別のアルゴリズムを用いる必要がある.)

* 別解

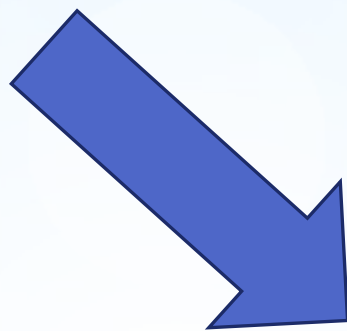




南東



な ん と う



なんと！

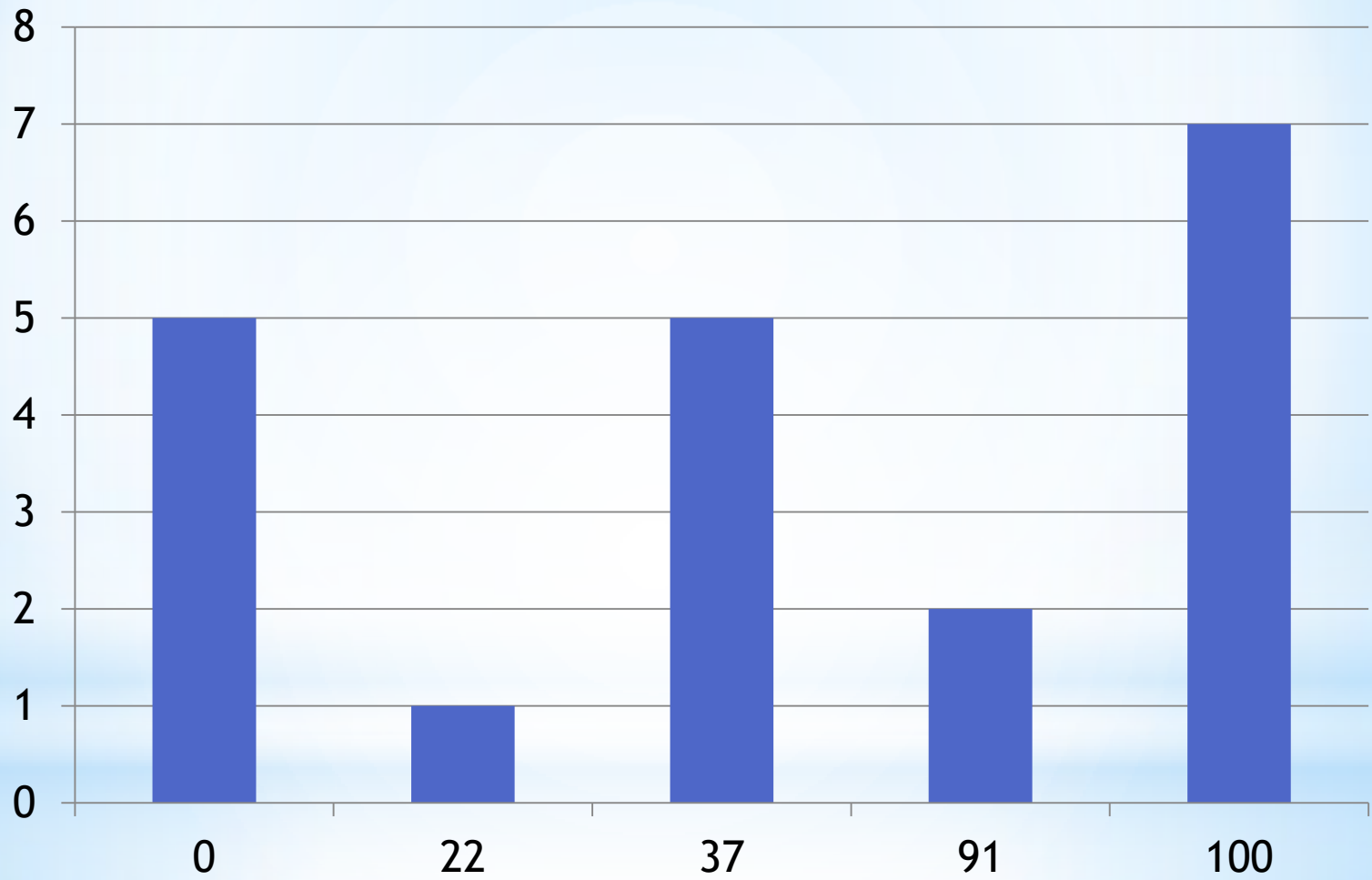
*有向グラフが送れます!

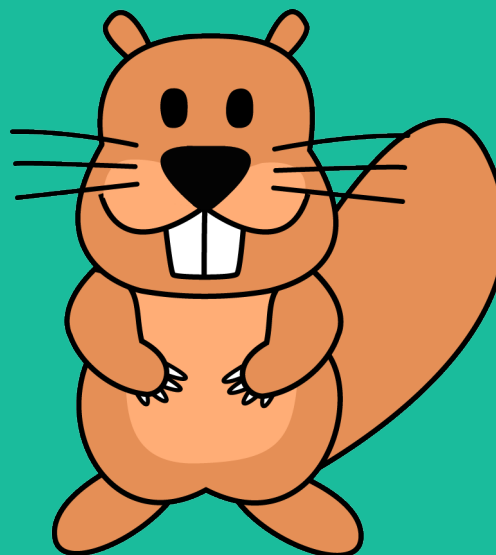
*草

* $V - N \leq 1$ で解ける

*十分大きい N では
 $\sqrt{\frac{2}{3}} N + \text{定数}$ で解ける

得点分布



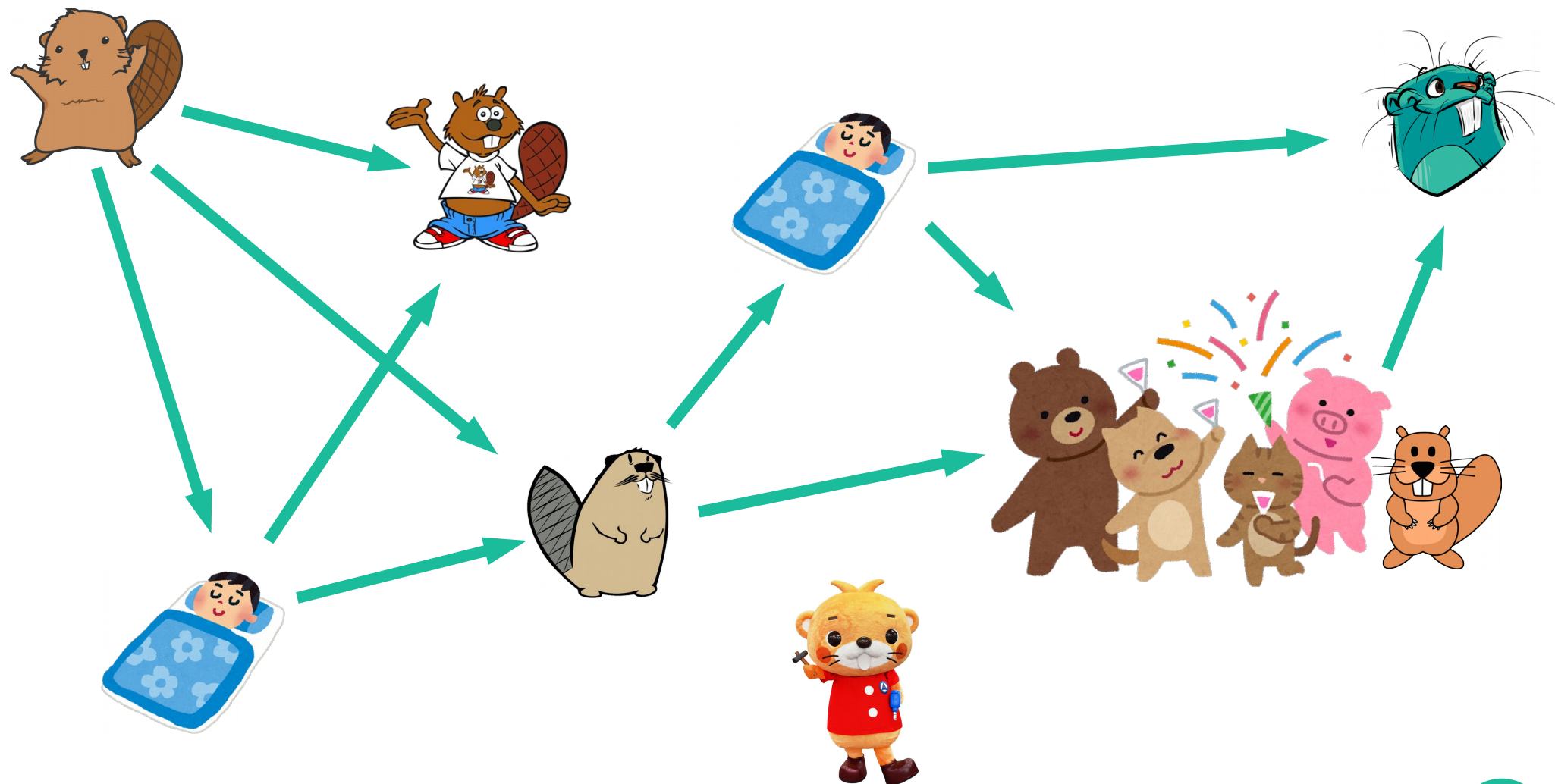


ビ太郎のパーティー (Bitaro's Party)

2018 年 3 月 22 日
情報オリンピック 2017-2018 春季トレーニング合宿 競技日 3

チューター：河原井 啓 (@gotoloop1)

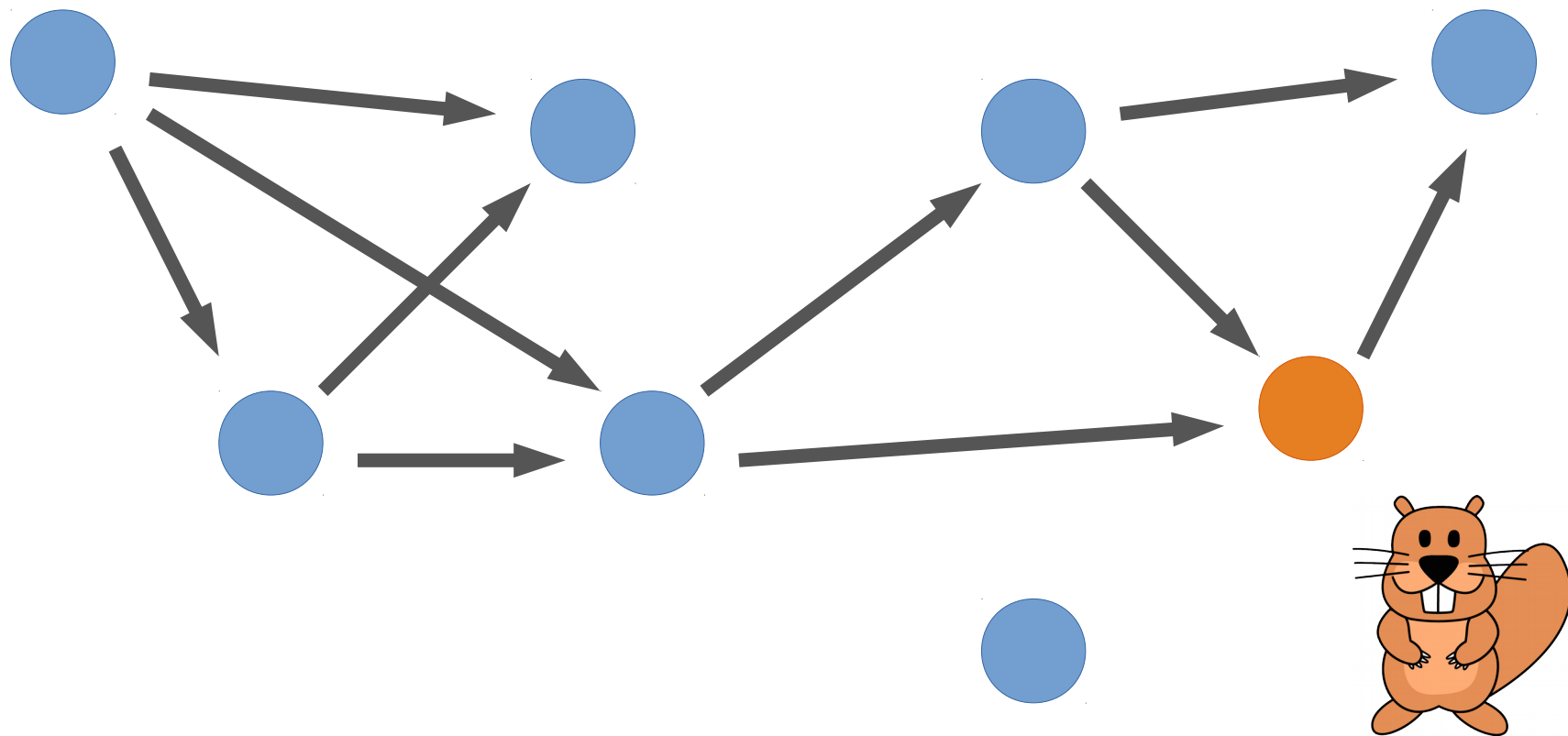
問題概要



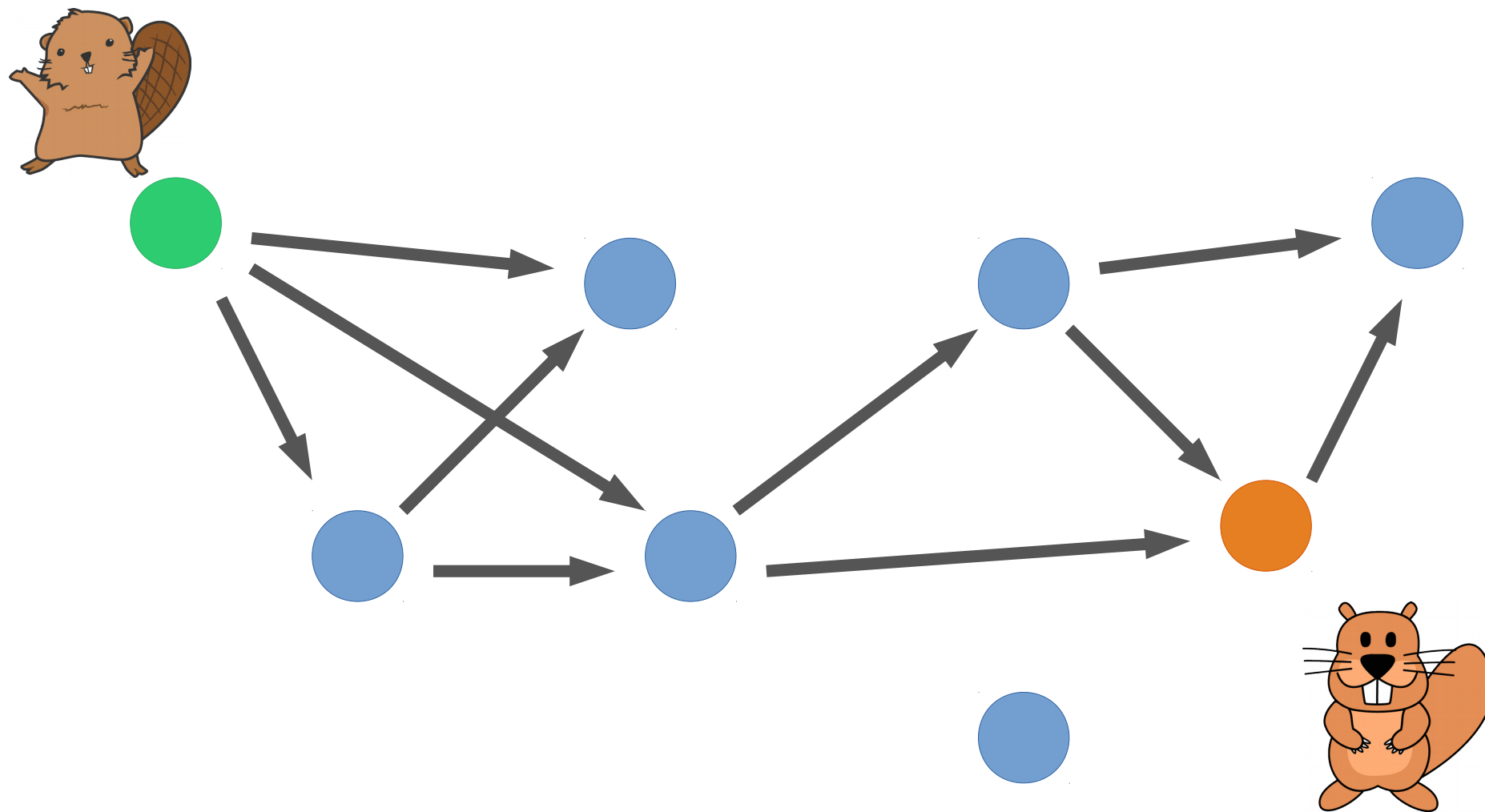
問題概要

- **N** 頂点 **M** 辺の **DAG** がある
 - トポロジカル順序も与えられている
 - 有効な頂点たちから、ある頂点に行くとき、考えられる最長の経路の長さを求めるクエリが **Q** 個来る
-
- **$N \leq 100000$**
 - **$M \leq 200000$**
 - **$Q \leq 100000$**

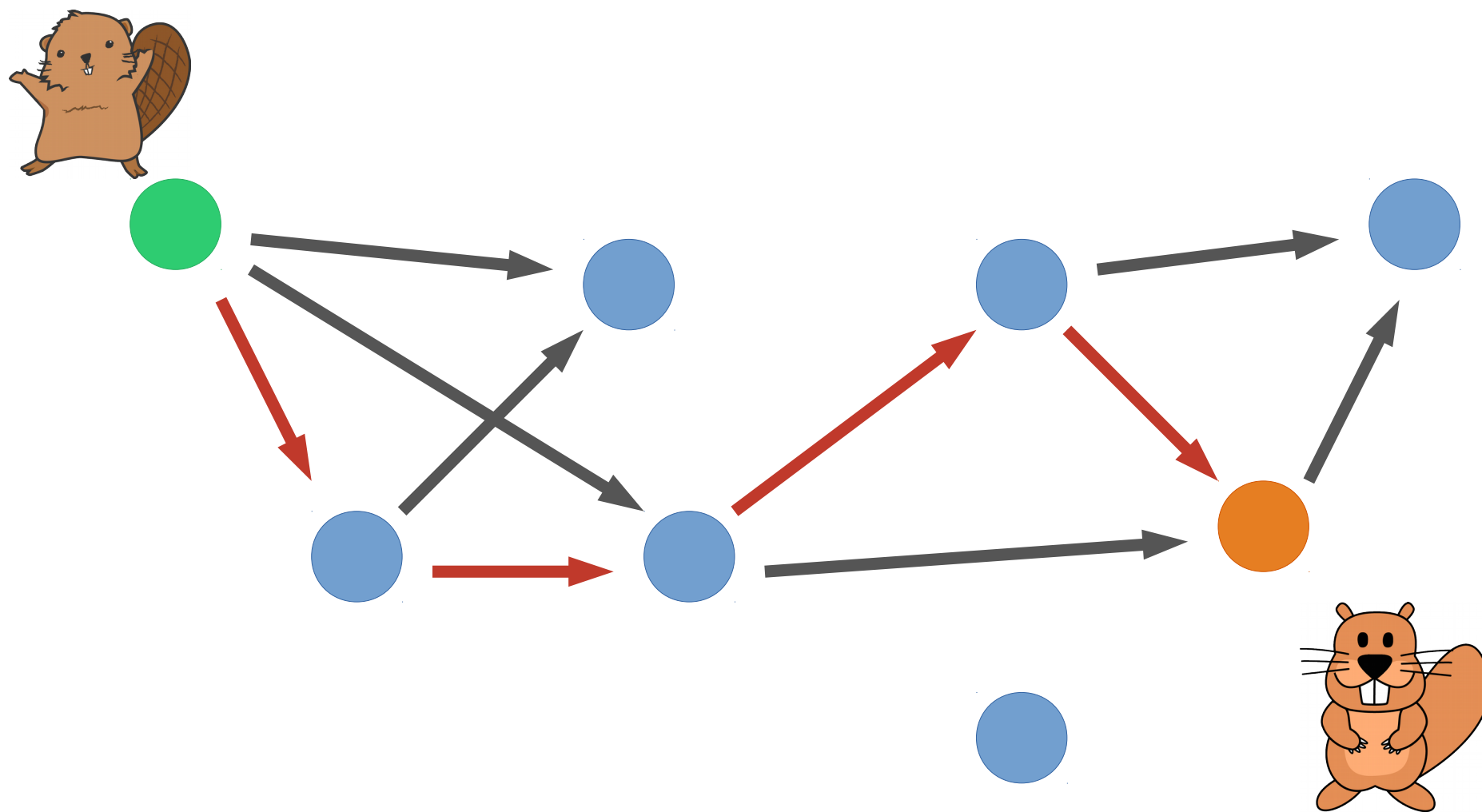
例



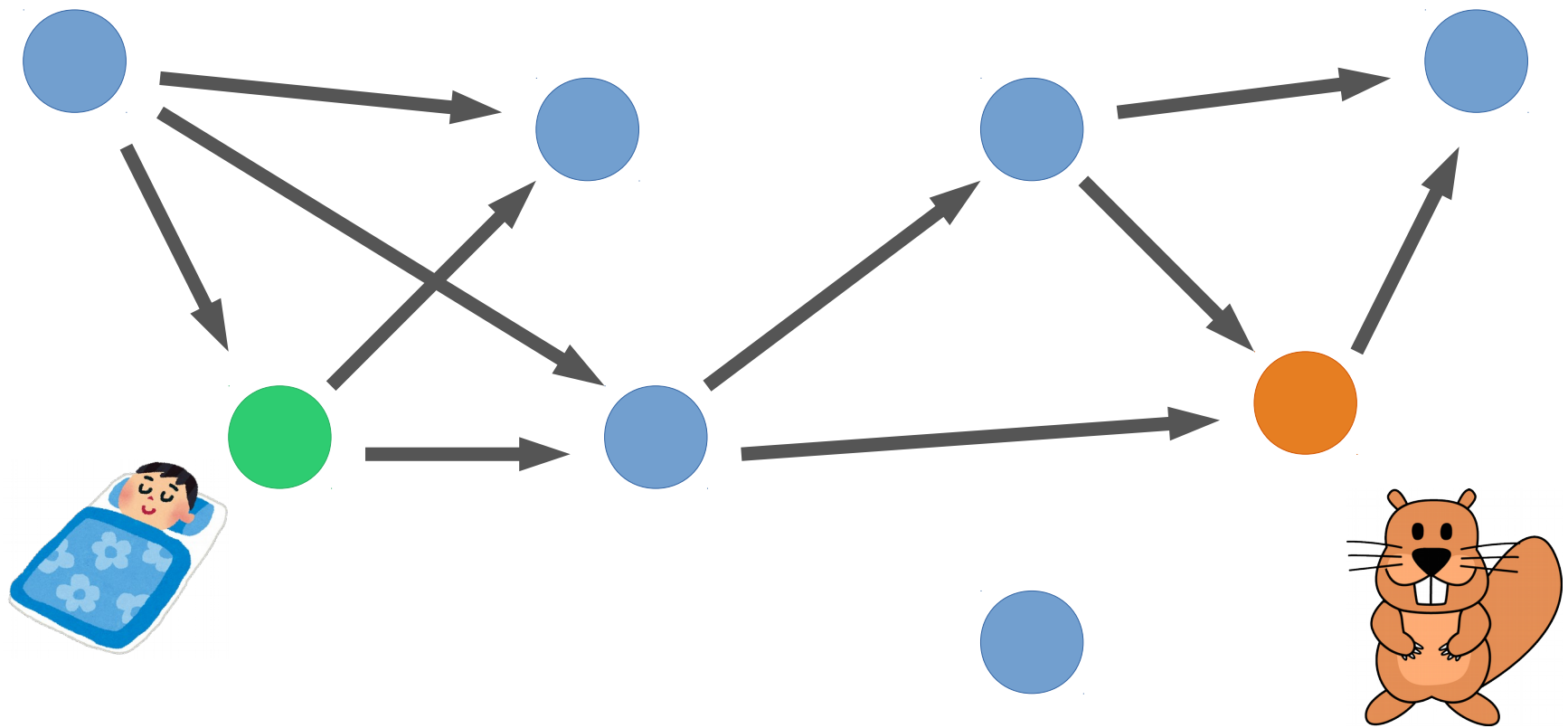
例



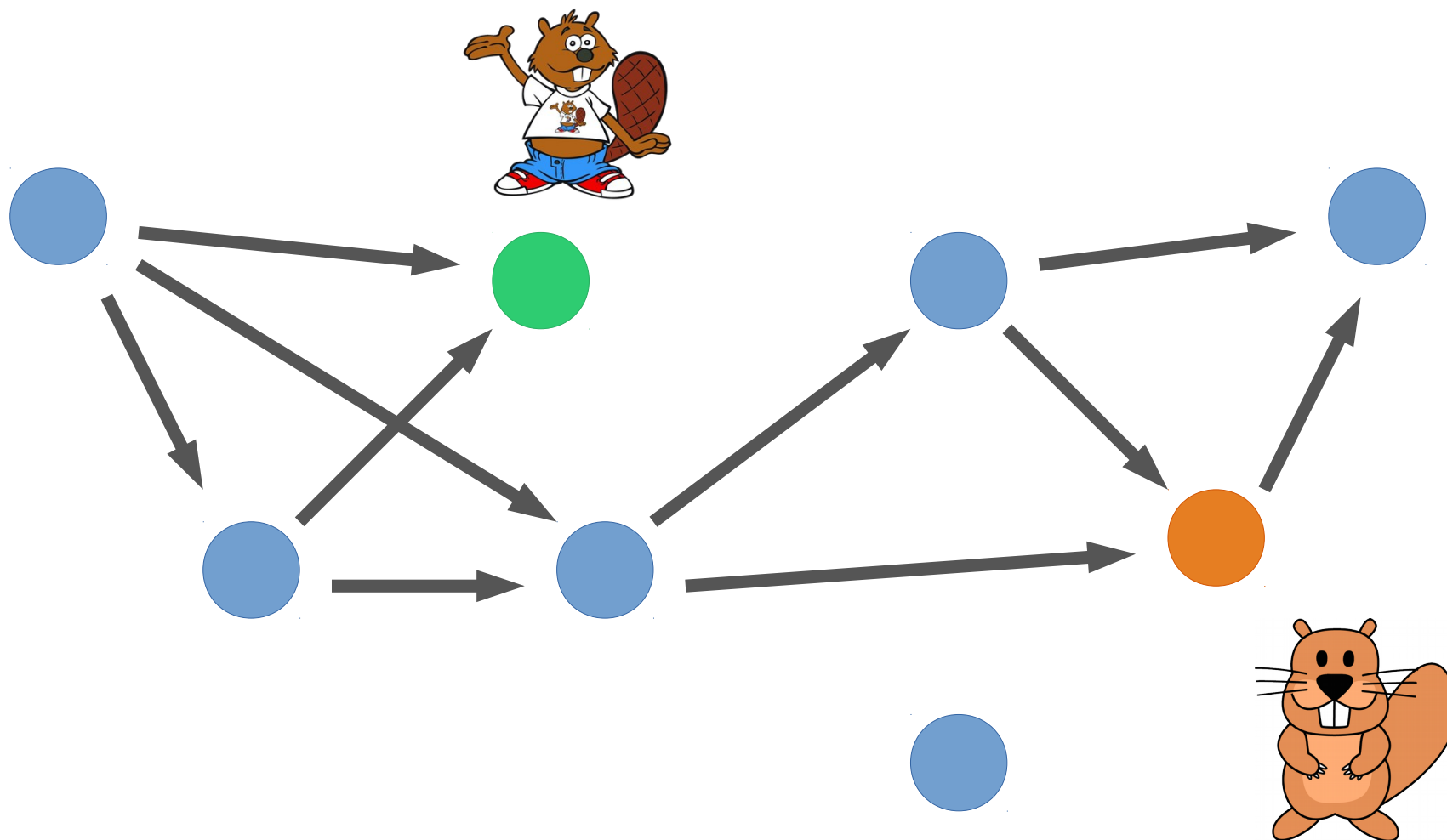
例



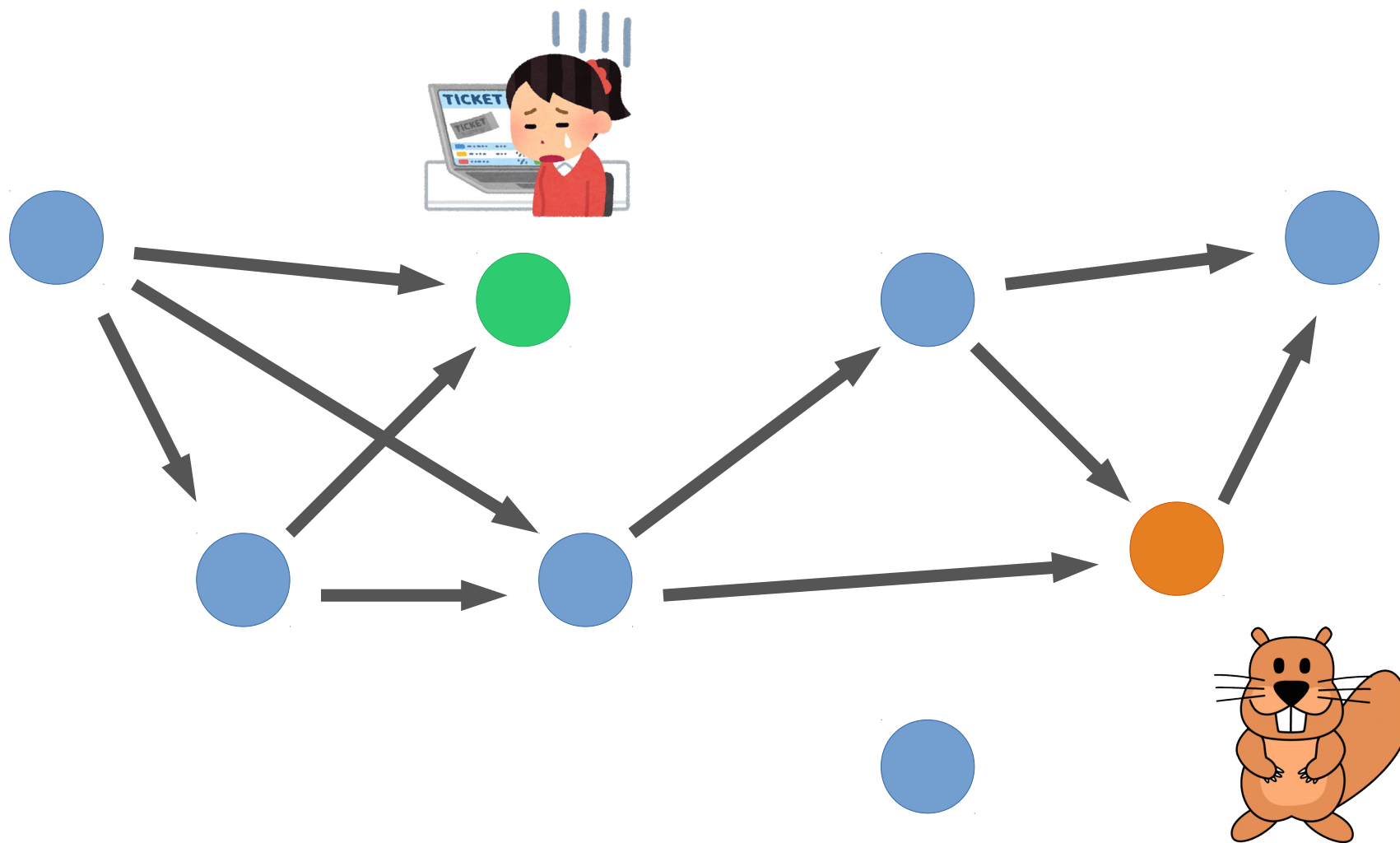
例



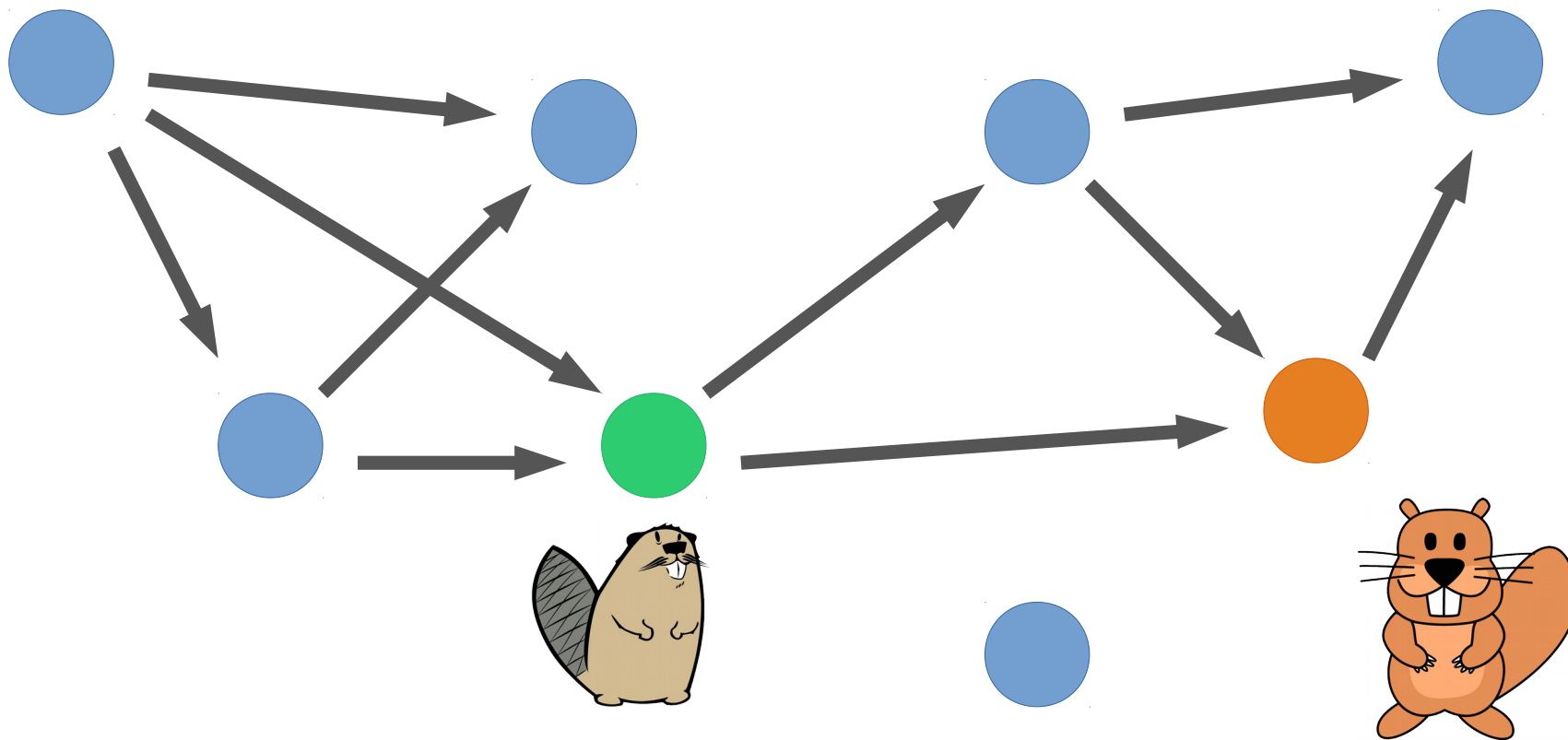
例



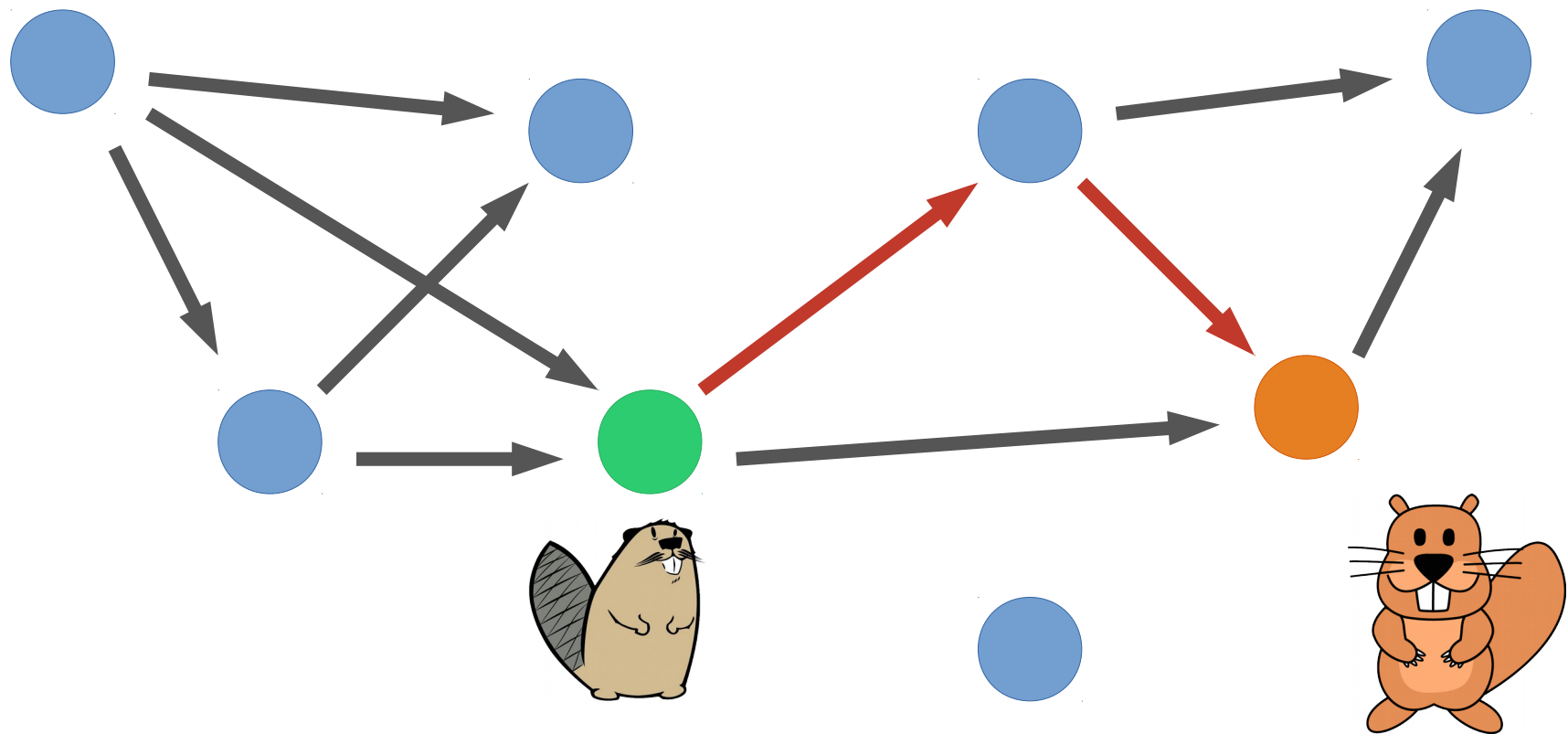
例



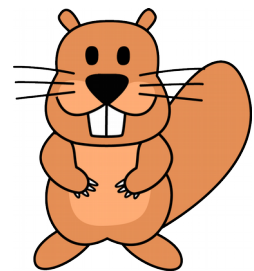
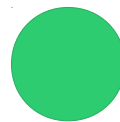
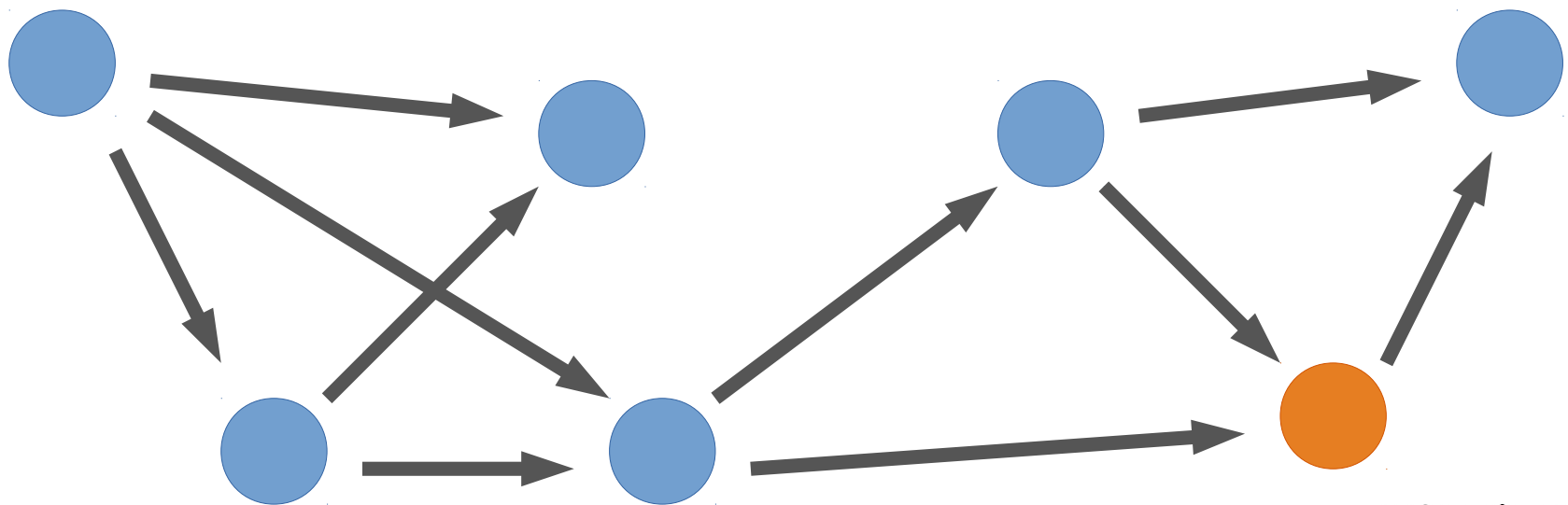
例



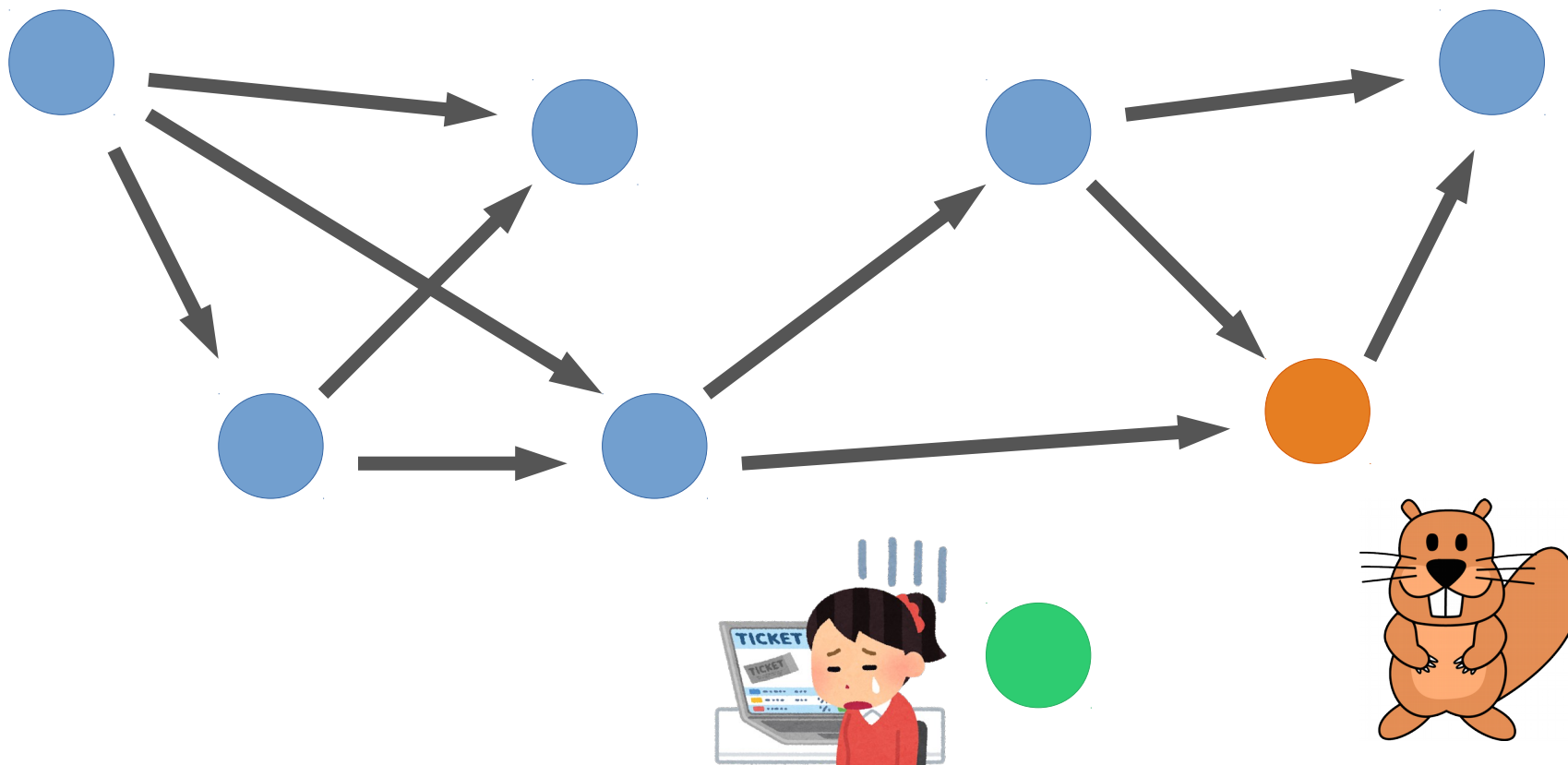
例



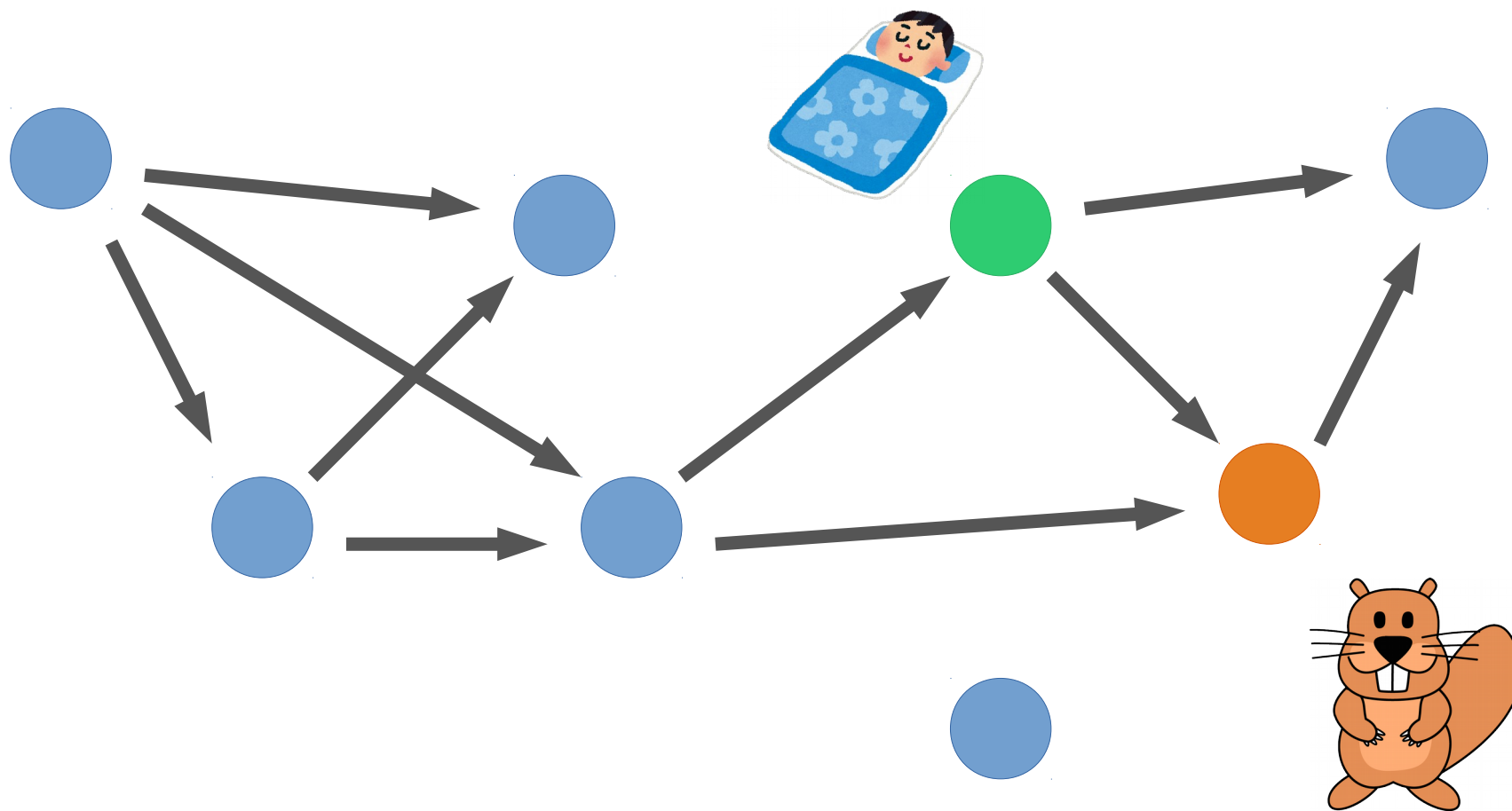
例



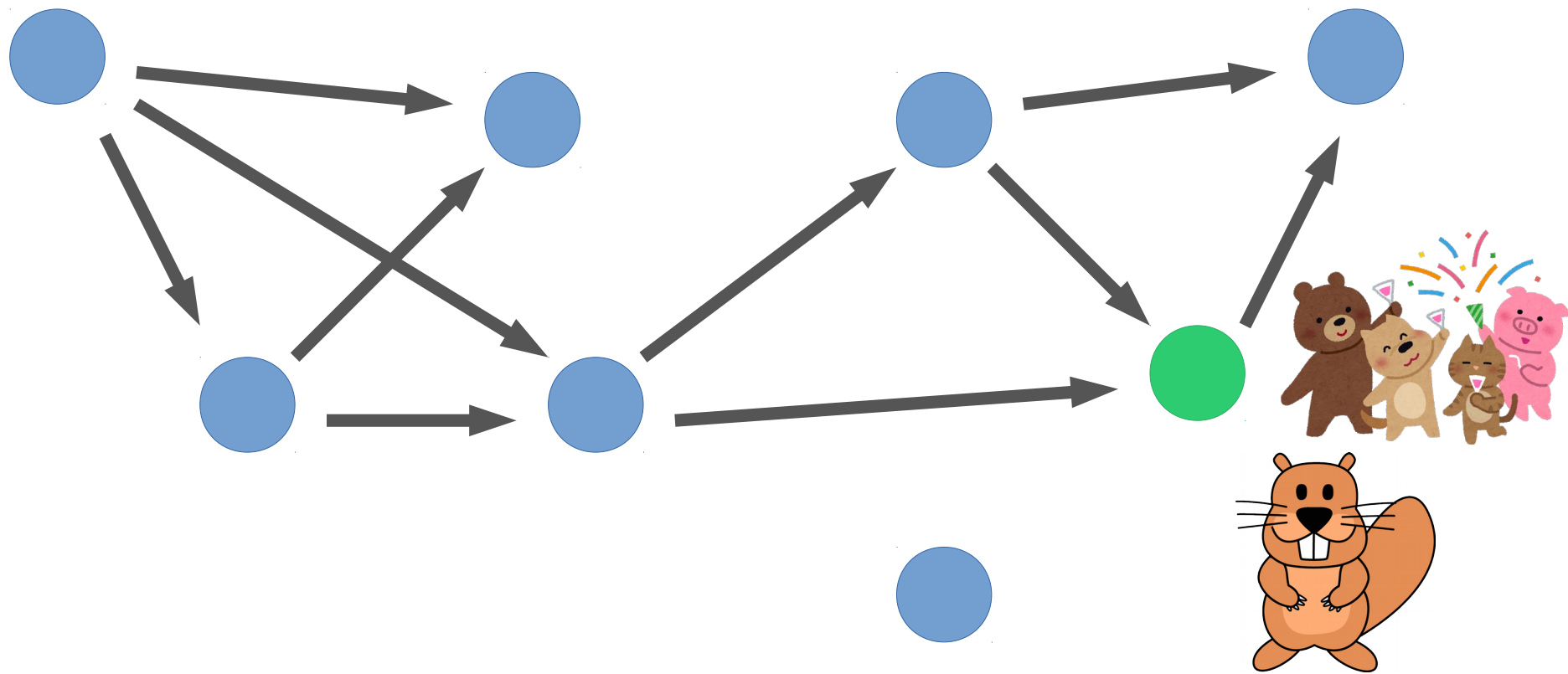
例



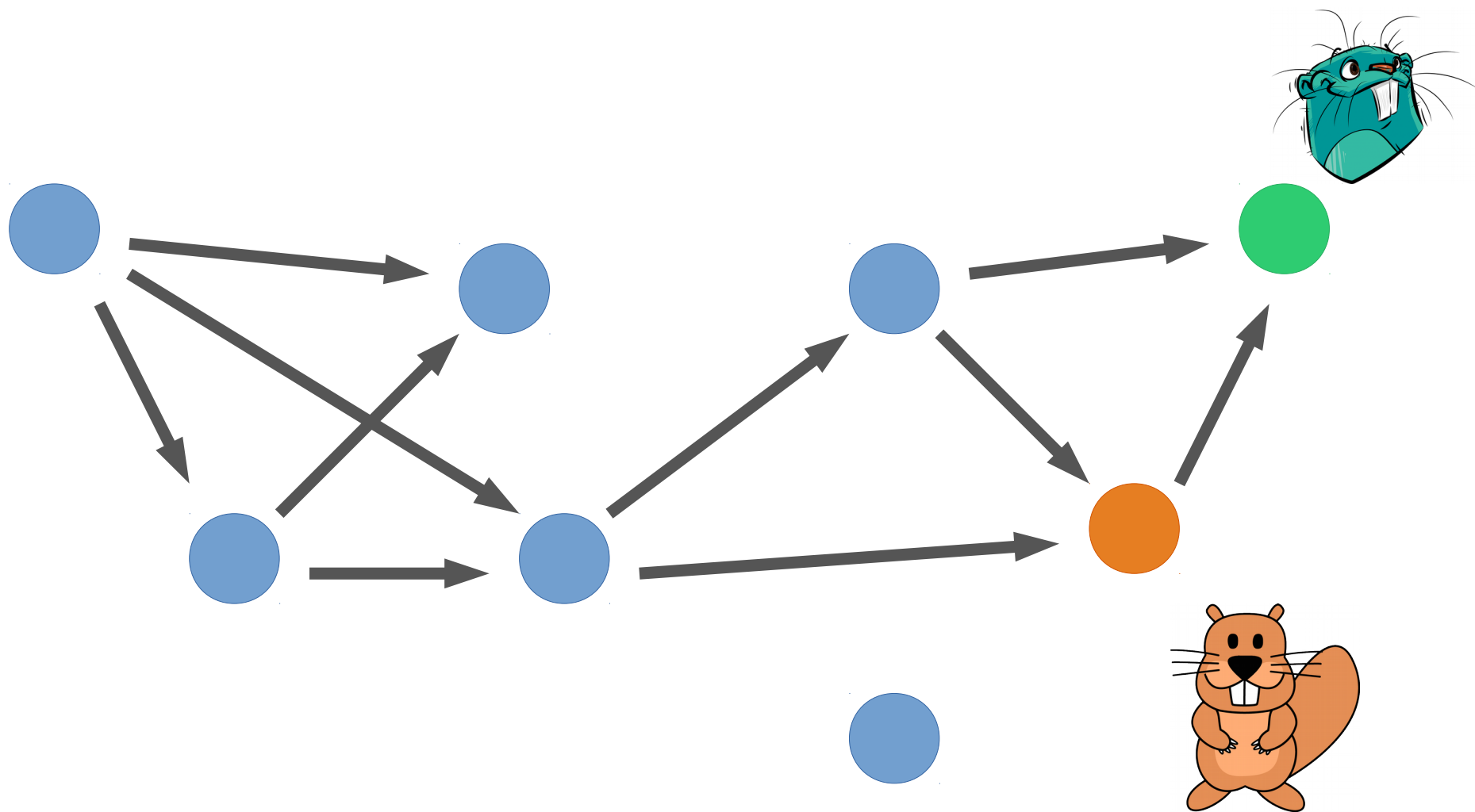
例



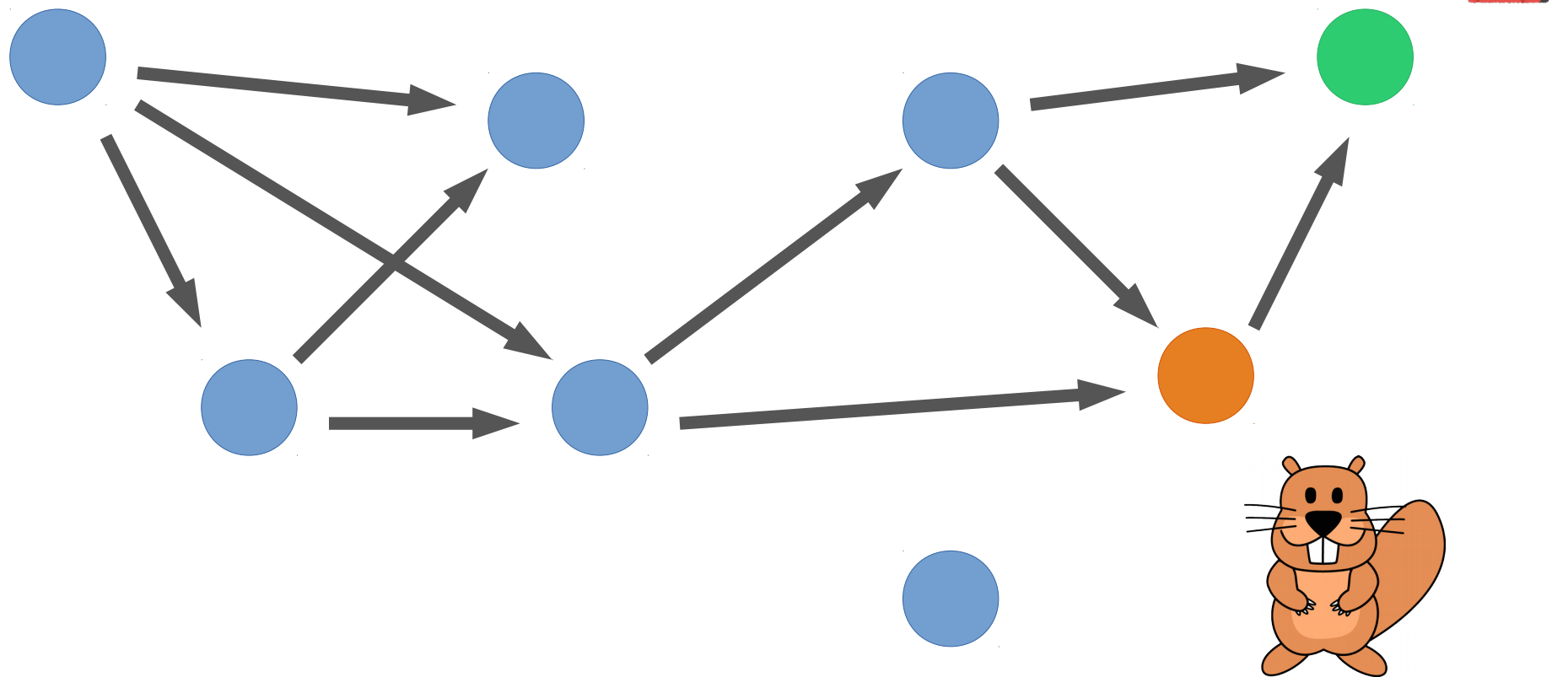
例



例



例



小課題 1 (7 点)

- $N \leq 1000$
 - $M \leq 2000$
 - $Q = 1$
-
- $O(NMQ)$ が通りそう

小課題 1 (7 点)

- すべての頂点 i について、有効な頂点からの考えられる最長の経路の長さ $\text{max_len}(i)$ を求める
- これは始点 i 、終点 j の辺について、
 $\text{max_len}(j) < \text{max_len}(i) + 1$
のとき、
 $\text{max_len}(j) = \text{max_len}(i) + 1$
と更新することを繰り返すことで求められる
- 初期値は有効なら 0 、無効なら $-\text{inf}$ など

小課題 1 (7 点)

- $\text{max_len}(i) \leq N$ なので、更新されるのは 1 本の辺につき高々 N 回
- それが M 本あるので、計算量は $O(NMQ)$

小課題 2 (7 点)

- $Q = 1$
- $O((N+M)Q)$ なら通りそう

小課題 2 (7 点)

- 小課題 1 のやり方は無駄が多い
- → トポロジカル順序で考える
- ある頂点よりトポロジカル順序が後の頂点は、その頂点に到達できない
- そのため、ある頂点 i の $\text{max_len}(i)$ は、
 - 有効な頂点なら $\max(0, \max(\text{max_len}(j)) + 1)$
 - 無効な頂点なら $\max(\text{max_len}(j)) + 1$
 - j は辺 $j \rightarrow i$ がある頂点

小課題 2 (7 点)

- すべての辺を $O(1)$ 回見ることになる
- 全体としては $O((N+M)Q)$

小課題ではない (0 点)

- クエリ i において $Y_i=1$ であるとする
- これを $O(N+M+Q)$ で解きたい
- どうしよう

小課題ではない (0 点)

- 各頂点において、有効な頂点からの考えられる最長の経路の長さと、**2** 番目に長い経路の長さを求める
- どの頂点から出発したかの情報を持っておけば、重複等なく経路の長さを求められる
- するとこの問題は **$O(N+M+Q)$** で解ける

小課題 3 (86 点)

- $Q \leq 100000$
- $\text{sum}(Y_i) \leq 100000$

小課題 3 (86 点)

- Y_i は最大で **100000** となる
- “小課題ではない”の方針でとこうとすると、**N** 番目に長い経路の長さまで持っていないと行けない
- $\rightarrow O(N(N+M)+Q)$
- 無理

小課題 3 (86 点)

- $B \leq N$ とする
 - 妥協して、 B 番目に長い経路まで持つ
 - $Y_i \geq B-1$ のときは小課題 2 の方針で解く
 - $\rightarrow O(B(N+M)+100000+ \text{何か})$
-
- “何か” ってなんだ☆

小課題 3 (86 点)

- “何か” が起こるのは $O(100000/B)$ 回
- そのたびに $O(N+M)$ の処理
- よって、全体では
$$O(B(N+M) + 100000 + 100000(N+M)/B)$$
- $B = 100000^{0.5}$ とすると
$$O(100000^{0.5}(N+M) + 100000)$$
- 解けた

ビ太郎とは

- ・ 小中高生を対象とした国際情報科学コンテスト
「ビーバーチャレンジ」の日本でのマスコットキャラクター
- ・ それぞれの国で違う



エジプト



イスラエル

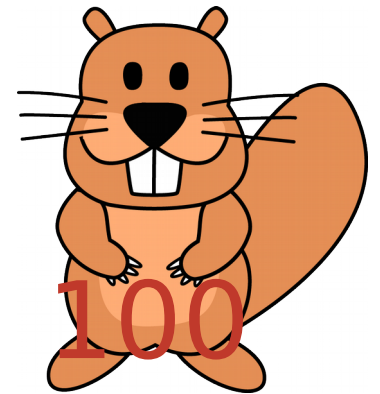
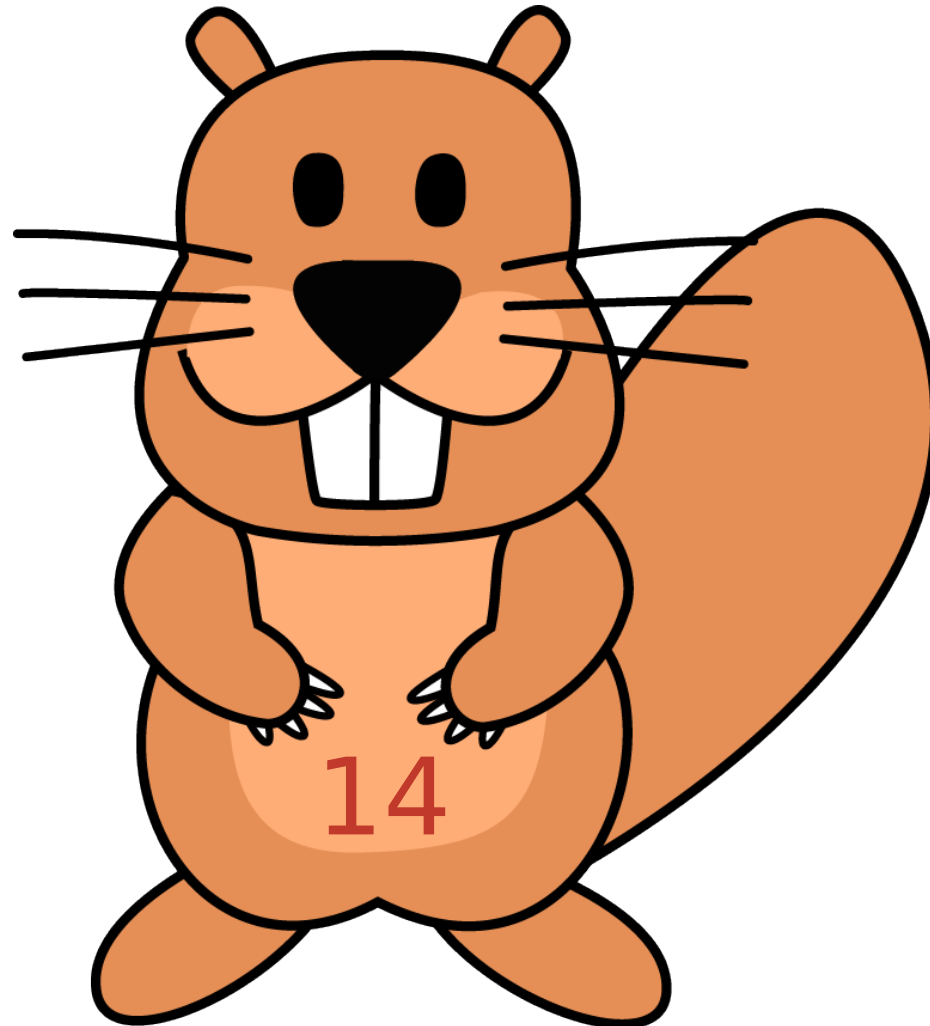
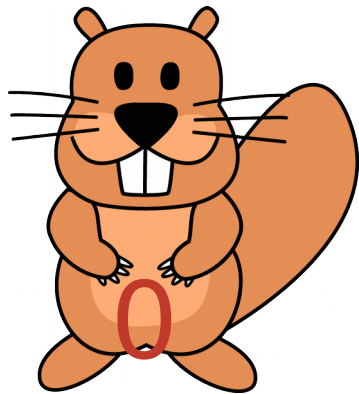


ラトビア



エストニア

得点分布



解説「防犯ゲート」

maroon

問題概要

- ‘(’ ‘)’ ‘x’ からなる文字列が与えられる
- ‘x’ を ‘(’ または ‘)’ に変更してできる文字列のうち、次の条件を満たすものは何通りか？
- 条件：ある連続する区間の ‘(’ と ‘)’ を入れ替えたとき、文字列はvalidな括弧列である

小課題1

- $N \leq 100$
- 'x' の個数 ≤ 4
- 'x' を '(' か ')' に変えてできる文字列は高々 2^4 通り
- ある文字列が条件を満たすか $O(N^3)$ くらいで判定できれば良さそう

小課題1

- 条件を満たすか否かの判定
- 反転する区間は $O(N^2)$ 通り
- Validな括弧列であるかどうかは $O(N)$ で判定できる
- $O(N^3)$ で出来て、4点

小課題2

- $N \leq 100$
- 'x' の個数 ≤ 12
- 'x' を '(' か ')' に変えてできる文字列は高々 2^{12} 通り
- ある文字列が条件を満たすか $O(N^2)$ くらいで判定できれば良さそう

小課題2

- ある連続区間を反転した後、文字列は以下のようになっている
(反転してない)(反転した)(反転してない)
- $DP[i][j][k]$ = i 文字目までを見たときに、かっこ列の高さが j で、 i 文字目が上の図での k 番目の位置にあることは可能か否か でDPすればいい
- 状態数 $O(N^2)$ 、遷移 $O(1)$ 、合計 $O(N^2)$ で出来て、12点

小課題3

- $N \leq 100$
- 'x' の個数 ≤ 20
- 'x' を '(' か ')' に変えてできる文字列は高々 2^{20} 通り
- ある文字列が条件を満たすか $O(N)$ くらいで判定できれば良さそう

小課題3

- 小課題2のDPをbitsetで高速化して30点

小課題3

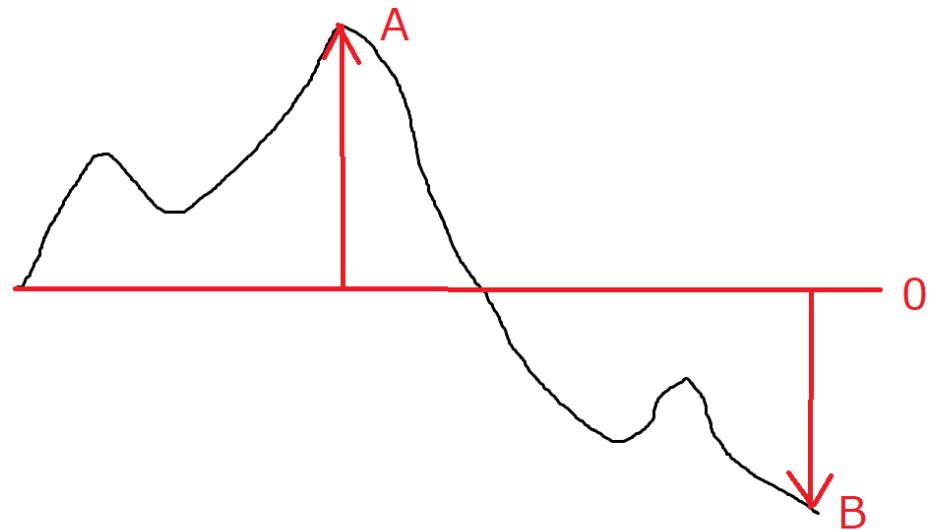
- 場合分けをする
- ① 何もしなくても valid
- ② 片側から見た時だけ invalid(括弧列の高さが途中で負になる)
- ③ どちら側から見ても invalid

小課題3

- ① 何もしなくてもvalid
- $O(N)$ で普通にできる

小課題3

- ② 片側から見た時だけ invalid
(一般化して、左から見たときにinvalidとする)
- 左から見て、高さが負にならないところまでの最大の高さをA、終了地点の高さをBとする
- この時、高さAの地点と、そこから右にある中で最も近い高さ $A+B/2$ の地点で反転するのが最適な操作
- $O(N)$ で出来る



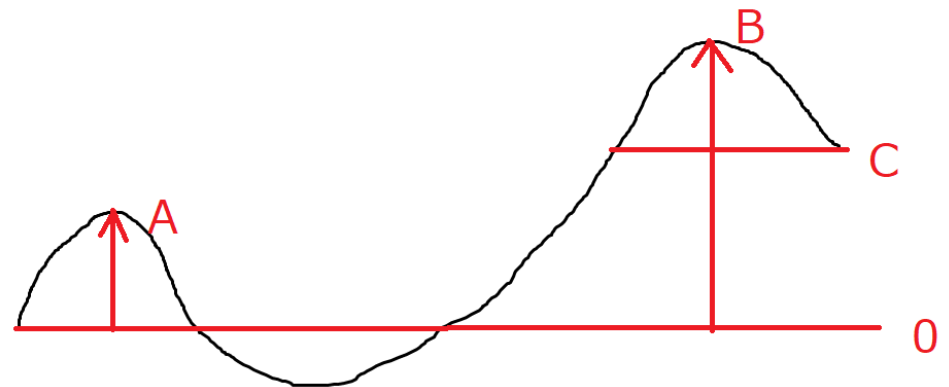
小課題3

- ③ どちら側から見ても invalid
- 終了地点の高さをC、左から見て高さが負になるまでの最大の高さをA、右から見たとき高さがC-1になるまでの最大の高さをBとする
- $A+C/2 \leq B$ の時、高さAの地点と、BもしくはBより左にある中で最も近い高さ $A+C/2$ の地点で反転するのが最適



小課題3

- ③ どちら側から見ても invalid
- $A+C/2 > B$ のとき、先ほどと同様にできる
- $O(N)$ で出来る



小課題4

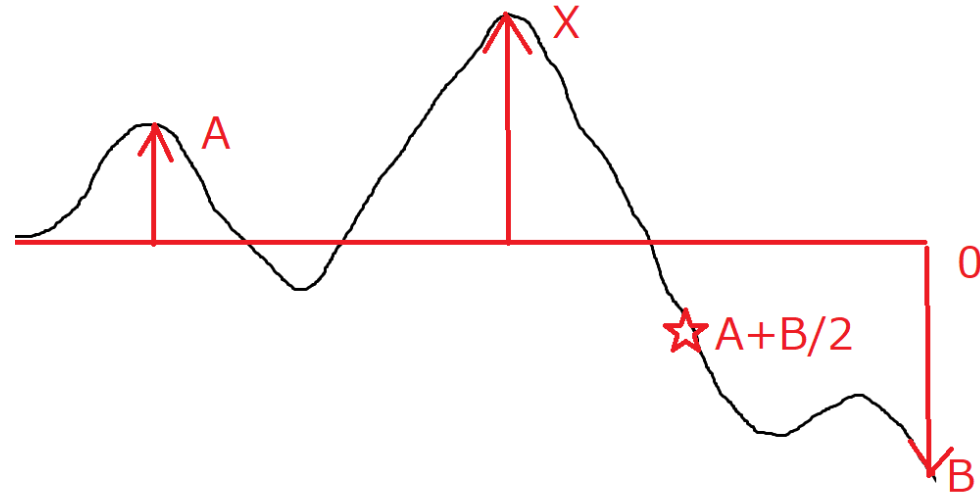
- ‘x’の個数に関する制約がない
- 小課題3と同じ場合分けをする

小課題4

- ① 何もしなくてもvalid
- 適当にDPをすると $O(N^2)$

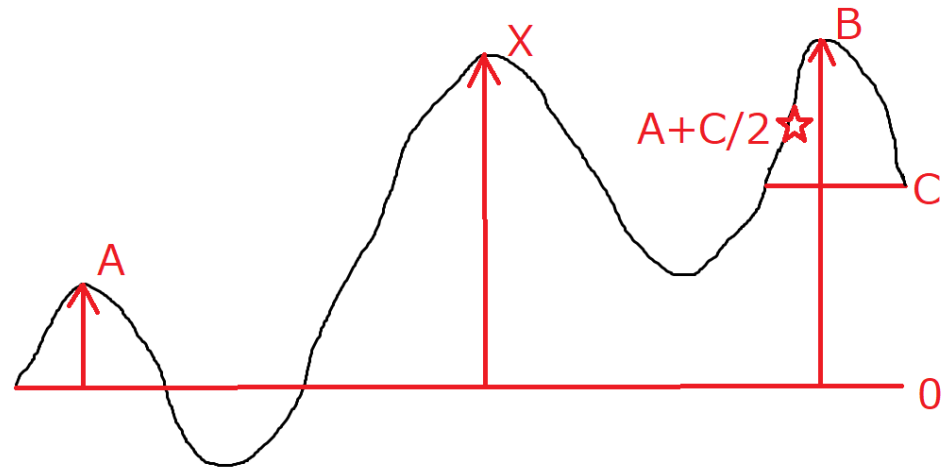
小課題4

- ② 片側から見た時だけ invalid
- 反転する区間(Aの地点と $A+B/2$ の地点の間)の高さの最大値を X としたとき、反転後の括弧列が $\text{valid} \Leftrightarrow X \leq 2A$
- A, B を固定すると、条件を満たすように丁寧な遷移すれば $O(N^2)$ のDPができる
- A, B の組をすべて試せば、合計 $O(N^4)$



小課題4

- ③ どちら側から見ても invalid
- $A+C/2 \leq B$ の時を考える
- 反転する区間(Aの地点と $A+B/2$ の地点の間)の高さの最大値を X としたとき、反転後の括弧列が $\text{valid} \Leftrightarrow X \leq 2A$
- A と C を固定すると、 $O(N^2)$ で DP できる
- A と C の組をすべて試せば、合計 $O(N^4)$



小課題4

- ③ どちら側から見ても invalid
- $A+C/2 > B$ の時を考える
- 先ほどと同様に $O(N^4)$ で出来る
- ①、②、③全部合わせても $O(N^4)$ で73点

小課題5

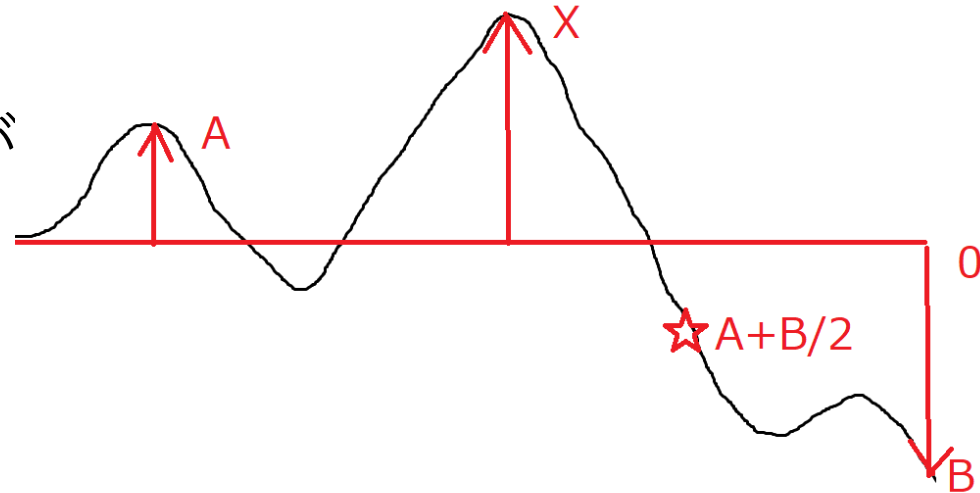
- 小課題5のDPを早くする

小課題5

- ① 何もしなくてもvalid
- そもそも $O(N^2)$ なのでOK

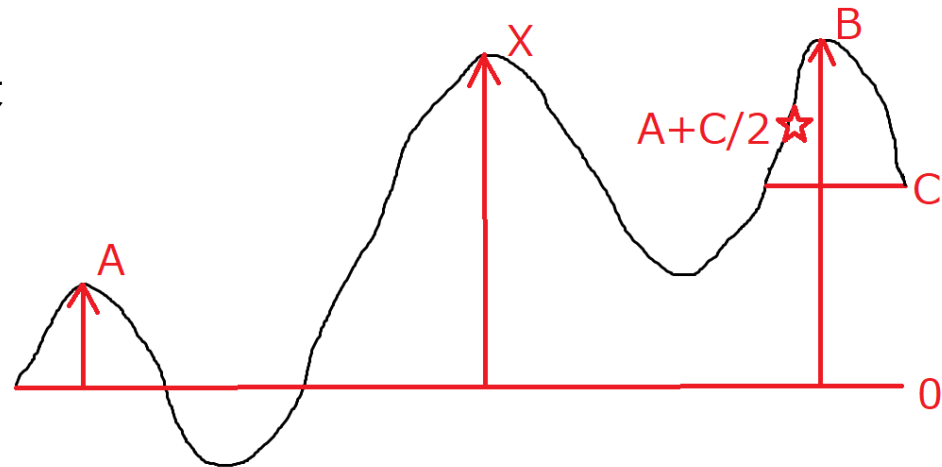
小課題5

- ② 片側から見た時だけ invalid
- A, B を固定すると、高さが A になる地点より右で初めて高さが 0 になる地点、の左右の遷移は、それぞれ $A, A-B/2$ にしか依存しない
- 左から進めたDPの結果と右から進めたDPの結果を使えば、 A, B を固定した際の答えが $O(N)$ で求まり、合計 $O(N^3)$ で出来る



小課題5

- ③ どちら側から見ても invalid
- $A+C/2 \leq B$ の時を考える
- A, C を固定すると、終了地点ではない中で最も右にある高さ C の地点、の左右の遷移は、それぞれ $A, B-C$ にのみ依存している
- ②と同じように、左からと右からのDPの結果を持っておけば、 A, C を固定した際の答えが $O(N)$ で求まる



小課題5

- ③ どちら側から見ても invalid
- $A+C/2 > B$ の時を考える
- 先ほどと同様に $O(N^3)$ で出来る
- ①、②、③全部合わせても $O(N^3)$ で100点

得点分布

[illegible]

Candies 解説

坂部



問題内容

- ・長さ N の自然数列 $A[i]$ がある。
- ・ $1 \leq K \leq N/2$ (切上げ)の全ての K について、 $A[i]$ の中から K 個を選んだ和を最大化したい。
- ・ただし、隣り合う2数を選んではいけない。

例

このようなおいしさの飴があるとき

3	5	1	7	6
---	---	---	---	---

例

このようなおいしさの飴があるとき

3	5	1	7	6
---	---	---	---	---

1つ選ぶ

3	5	1	<u>7</u>	6
---	---	---	----------	---

7美味しい

2つ選ぶ

3	<u>5</u>	1	<u>7</u>	6
---	----------	---	----------	---

12美味しい

3つ選ぶ

<u>3</u>	5	<u>1</u>	7	<u>6</u>
----------	---	----------	---	----------

10美味しい

例

このようなおいしさの飴があるとき

3	5	1	7	6
---	---	---	---	---

これを出力

1つ選ぶ

3	5	1	<u>7</u>	6
---	---	---	----------	---

7美味しい

2つ選ぶ

3	<u>5</u>	1	<u>7</u>	6
---	----------	---	----------	---

12美味しい

3つ選ぶ

<u>3</u>	5	<u>1</u>	7	<u>6</u>
----------	---	----------	---	----------

10美味しい

小課題1(8点)

$N \leq 2,000$

- そんなに大きくない
- $O(N^2)$ が通りそう。

小課題1(8点)

DP[i][j] :

A[0] ~ A[i - 1]からj個選ぶ時の最大値

としてDP

小課題1(8点)

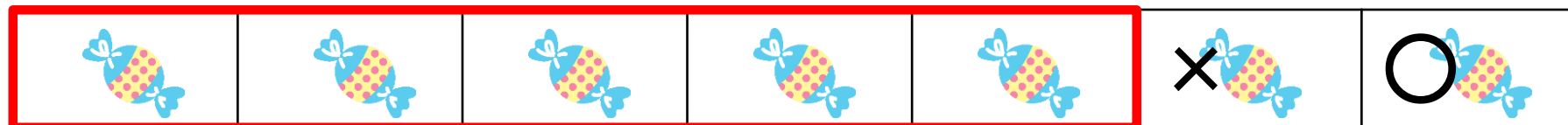


○ この中から3個選ぶとき

小課題1(8点)



- この中から3個選ぶとき



- 赤枠の中から2個と右端

もしくは



- 赤枠の中から3個

小課題1(8点)

DP[i][j] :
A[0] ~ A[i + 1]からj個選ぶ時の最大値

としてDP $\rightarrow O(N^2)$ で解ける

小課題1(8点) 解法2

K個選ぶときの最適解と、K+1個選ぶときの最適解を見比べてみる

K = 1	3	5	1	<u>7</u>	6
K = 2	3	<u>5</u>	1	<u>7</u>	6
K = 3	<u>3</u>	5	<u>1</u>	7	<u>6</u>

小課題1(8点) 解法2

K + 1個選ぶときの最適解は、K個選ぶときの選び方と比較して

①両隣がまだ選ばれていない飴を新たに選ぶ

② $\times \bigcirc \times \dots \bigcirc \times \rightarrow \bigcirc \times \bigcirc \dots \times \bigcirc$

のいずれか。

(どちらでもない場合、①で新たに選ばれる飴があるにも関わらず、その他の飴の位置がいくつか変わっており、このときK+1個の選び方の最適性より、K個の選び方の最適性に矛盾。)

小課題1(8点) 解法2

このとき、下の形は、

	A	<u>B</u>	C	
--	---	----------	---	--

小課題1(8点) 解法2

以降下の2パターンにしかないので

	A	<u>B</u>	C	
--	---	----------	---	--

	<u>A</u>	B	<u>C</u>	
--	----------	---	----------	--

小課題1(8点) 解法2

(B点の差はあるが)右の形として考えられる。

A	<u>B</u>	C
---	----------	---

 $=$

$A + C - B$

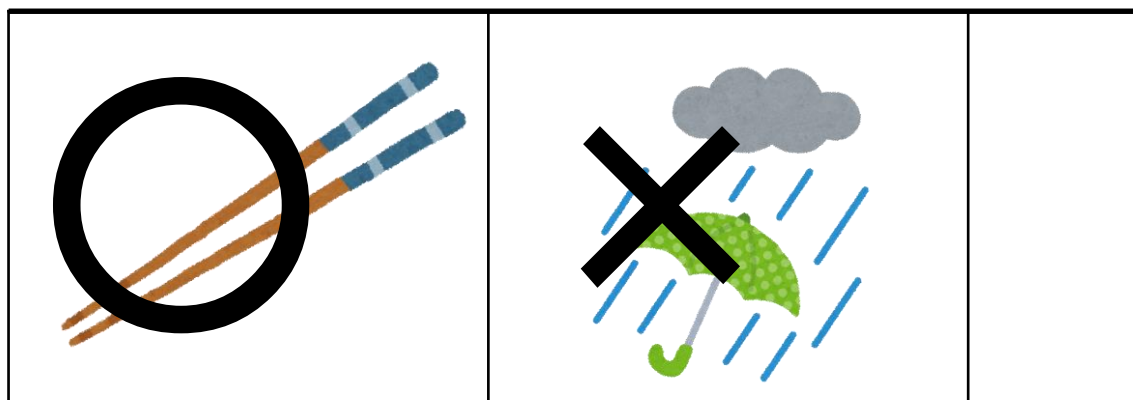
<u>A</u>	B	<u>C</u>
----------	---	----------

 $=$

<u>$A + C - B$</u>

小課題1(8点) 解法2

また、端の飴を選んだ場合、その部分はい降変更されない
ので、考えなくて良い。



小課題1(8点) 解法2

K = 1	3	5	1	6	<u>7</u>
-------	---	---	---	---	----------

7おいしい

K = 2	3	<u>5</u>	1
-------	---	----------	---

12おいしい

K = 3	<u>-1</u>
-------	-----------

11おいしい

小課題1(8点) 解法2

これを愚直に実装して、 $O(N^2) \rightarrow$ 解けた

小課題2(累計100点)

$N \leq 200,000$

- ・大きい。
- ・ $O(N^2)$ はTLE

小課題2(累計100点)

さっきの方法は、更新されていない所の価値の大きを何度も比較していて、効率が悪い。

小課題2(累計100点)

priority_queueなどを用いて無駄な計算を省く。

※ただし、priority_queueから出てきた情報が古い可能性があり、区間管理などが必要。

小課題2(累計100点)

一例(区間を管理する配列を持つ)

$A[i]$	<u>$A[i + 1]$</u>	$A[i + 2]$
i	$i + 1$	$i + 2$



複雑	-INF	-INF
$i + 2$	$i + 1$	i

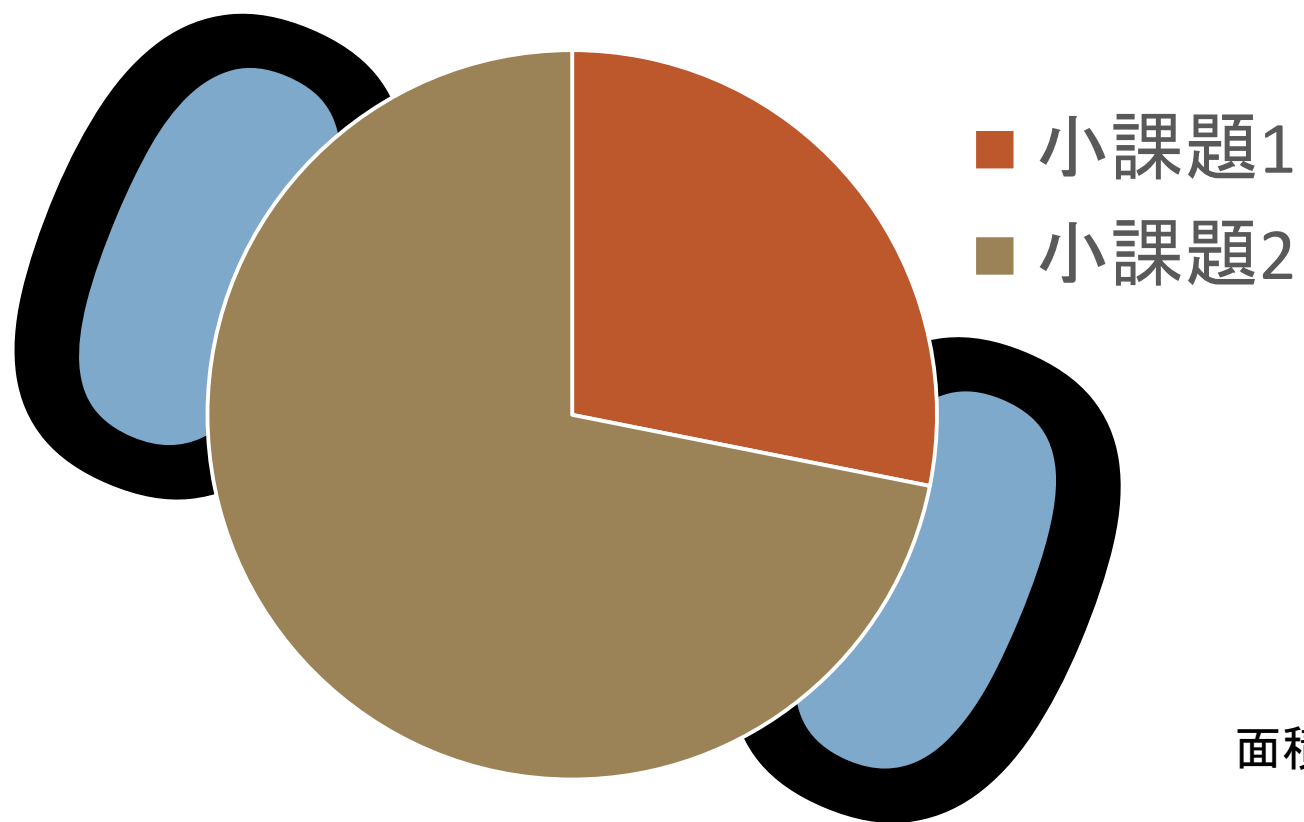
小課題2(累計100点)

$O(N\log N)$ → 解けた

小課題2(累計100点)

※分割統治解など別解あり

得点分布



面積は、総得点の量を表す。

図書館



問題概要

reactiveな問題

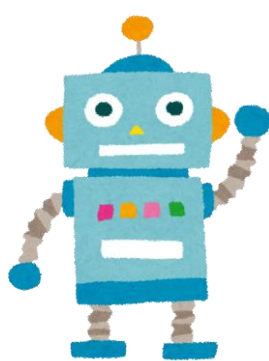
- 番号1から番号NまでのN冊の□がある。 $(1 \leq N \leq 1000)$
- 1列の本棚にそれらの□が無作為に並んでいる。
- クエリを投げて□の並び順を特定したい。(左右は逆でもよい)



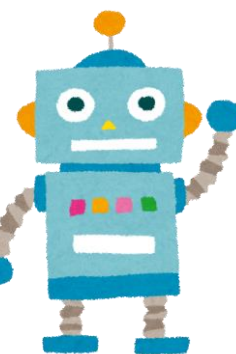
問題概要

reactiveな問題

- 本棚からは連続して並んでいる1冊以上の□を同時に取り出せる。
- 空でない□の集合Mとともにクエリを投げると、クエリで指定した□を取り出すために必要な操作の回数を教えてもらえる。
- 20000回以下のクエリで□の並び順を特定したい



1回



2回

例

(1, 5, 2, 3, 4)

Query(1,1,1,0,0) -> 2

Query(1,1,1,0,1) -> 1

Query(0,0,1,1,0) -> 1

Query(0,0,0,0,0) -> Wrong Answer

クエリの読み替え

Query(M)は指定した本の集合Mを
すべて取り出すための最小の作業回数が返ってくる

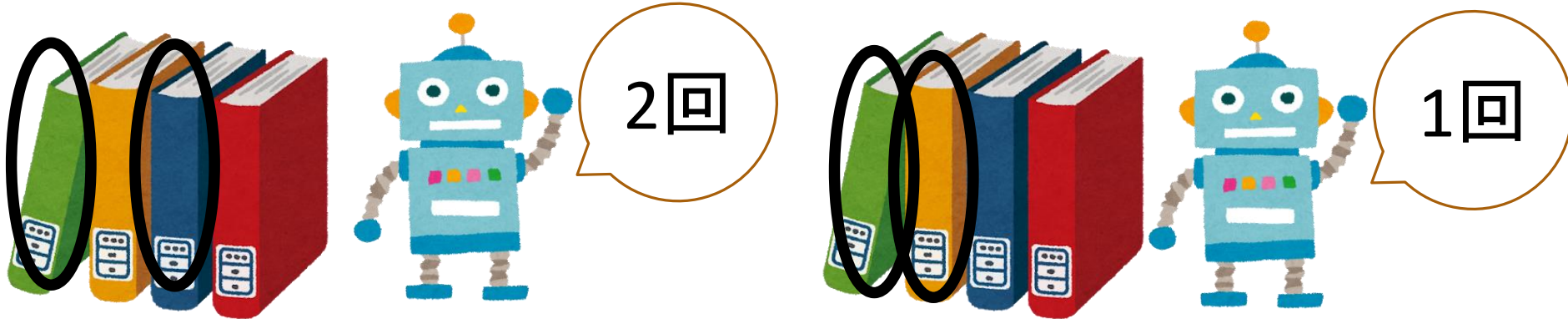
このクエリは状態を変えるわけではないので情報を
得ることに意味がある

しかしこれは一見何がわかるのかわかりづらい

クエリの読み替え

作業回数は孤立した本の数が増えると大きくなる。

また、同じ本の数でも、作業回数は隣接する本が1組あるたびに1回減ることを意識する



クエリの読み替え

つまり、

Query(M)=

Mで指定した本の数-Mで指定した本同士で隣接する本の組の数

であることがわかる



クエリの読み替え

つまり、クエリから隣接する□の組の数がわかる。

なので、クエリを投げるとMの中で、隣接する組の数が返ってくると考える(これをAdjacent(M)とする。)

(つまり、 $\text{Adjacent}(M) = |M| - \text{query}(M)$)

例

(1,5,2,3,4)

Adjacent(1,1,1,0,0) -> 1(Query=2)

Adjacent(1,1,1,0,1) -> 3(Query=1)

Adjacent(0,0,1,1,0) -> 1(Query=1)

基本事項・自明な考察

- 質問できる回数は20000回
- 隣接する□の組は $N-1$ 組
- 指定した□の集合 M が1冊なら自明に $\text{Adjacent}(M)=0$
- 1組 $\log N$ 回で求められればできそう
- どの□が端にあるかは簡単にわかる(全集合 A と任意の□ x に対し $\text{Adjacent}(A-x)$ は x が端にあるなら $N-2$, そうでないなら $N-3$ となり、異なるため)

小課題1 ($N \leq 200$)

全探索

(全ての2つの□の組み合わせを投げる)

すると各□に隣接する□がわかるので並び順が一意に定まる

小課題1 ($N \leq 200$)

全探索

(全ての2つの□の組み合わせを投げる)

nC_2 回クエリを投げればよいので、たか
だか19900回でできる

考察

指定する本の集合 M (x を含まない) へ要素を追加したときの挙動を考えると

$$\text{Adjacent}(M \cup x) - \text{Adjacent}(M)$$

$$= |\{x\}| \text{ に隣接する本の数}$$

となる

考察

例

$$\text{Adjacent}(1, 1, 0, 0, 1) = 3$$

$$\text{Adjacent}(1, 1, 1, 0, 1) = 4$$

このとき{□□1, □□2, □□5}のうち

□□3に隣接する□□の数は上2つの式の差から1つであることがわかる。

考察

これを応用して二分探索すれば、 x に隣接する頂点の1つが $O(\log N)$ 回のクエリでわかる

1. まず x に隣接する頂点の候補をすべて列挙し、その集合を M とする。
2. M を A と B におよそ均等に分割し、Queryを投げて $C = \text{Adjacent}(x \cup A) - \text{Adjacent}(x)$ を求める。
3. C が1以上なら A を、そうでないなら B を M として2を再び行う。

M のサイズが1になったとき、その M に含まれる頂点が x に隣接することがわかる。

満点解法

二分探索

まず端にある $\square a_1$ を1つ求める

その $\square a_1$ に隣接する $\square a_2$ を二分探索で求める。

その $\square a_i$ に隣接する $\square a_{i+1}$ を二分探索で求める...

を繰り返せばおわり

満点解法

操作回数は

a_1 を求めるのにN回

$2\log N$ 回の二分探索をN回で $2N\log N$ 回

合計で $N+2N\log N$ 回(ん?)

注意点

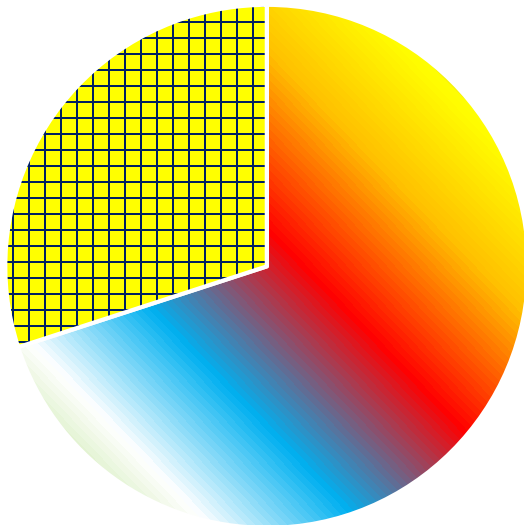
雑にやると $N+2N\log N$ 回で怪しいが、適当に節約すれば大丈夫

隣接する□□の候補が1つになったとき、2回ではなく1回で識別するとか

すでに並び順がわかっている□□を省いて探索とか

そもそも、1つの□□を選んでその□□の両隣を見つけて同様のことをすれば
 $2N\log N$ 回でよい

得点分布



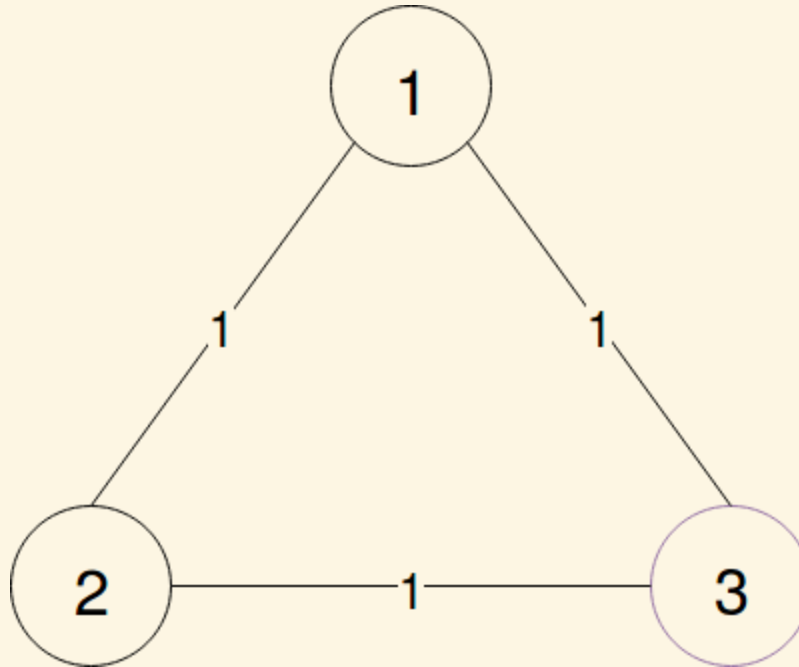
イノシシ (Wild Boar)

joisino

課題の説明

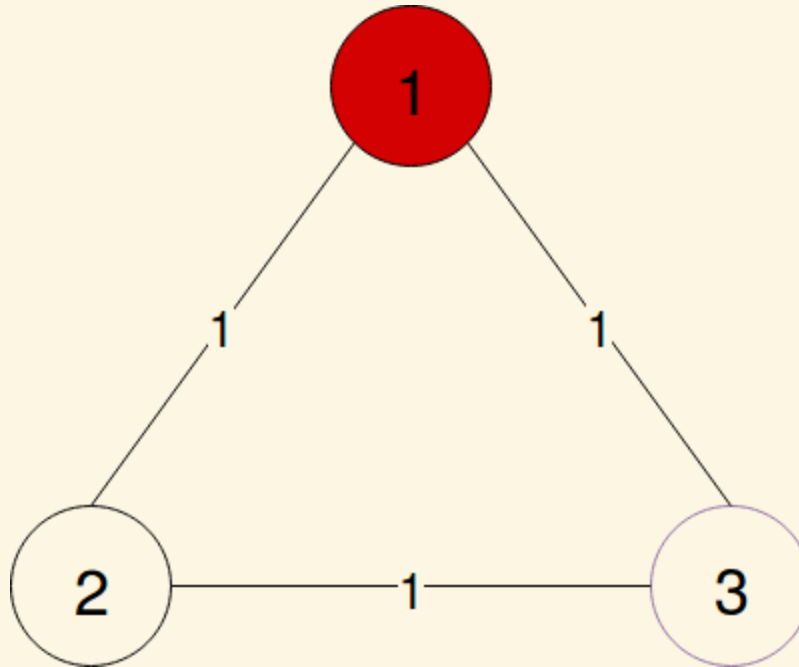
- 重み付き無向グラフと頂点列が与えられる
- 頂点列をこの順に訪れる最小合計コストを求める
- ただし、辺を通ったあとすぐにその辺を折り返すことはできない
- 頂点列の一つを変更するクエリが与えられるので、変更するたびに最小合計コストを求める

サンプル 1



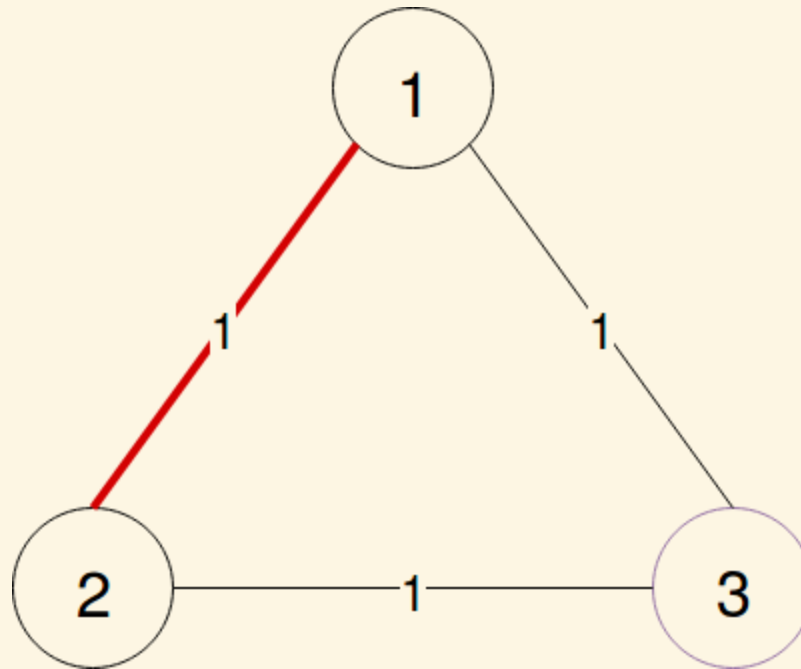
- 実行する補給計画: 1 -- 2 -- 1
- 時間: 0

サンプル 1



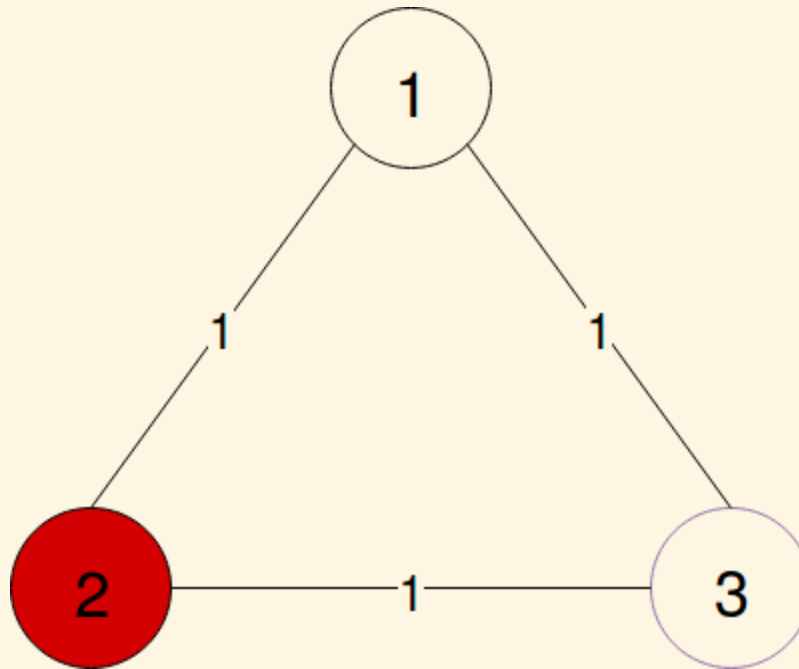
- 実行する補給計画: **1** -- 2 -- 1
- 時間: 0

サンプル 1



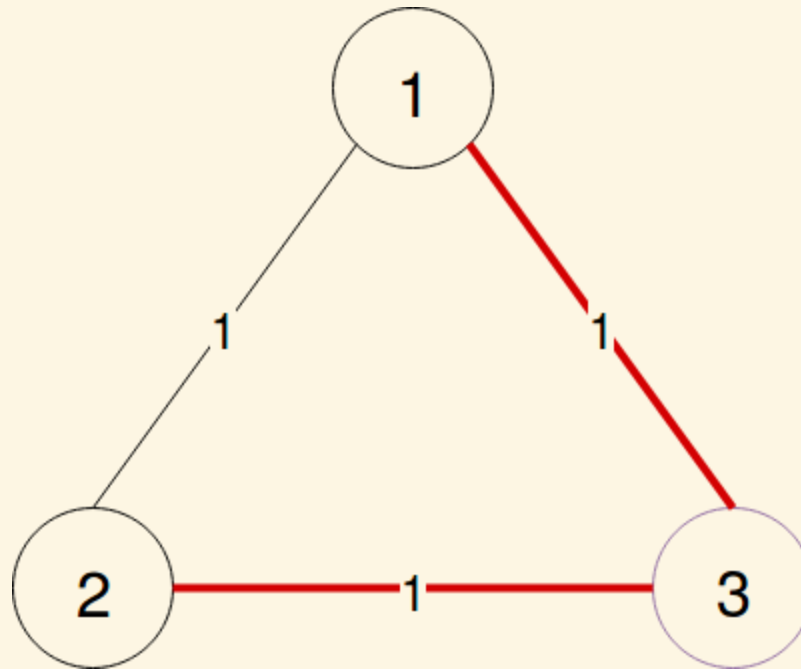
- 実行する補給計画: 1 -- 2 -- 1
- 時間: 1

サンプル 1



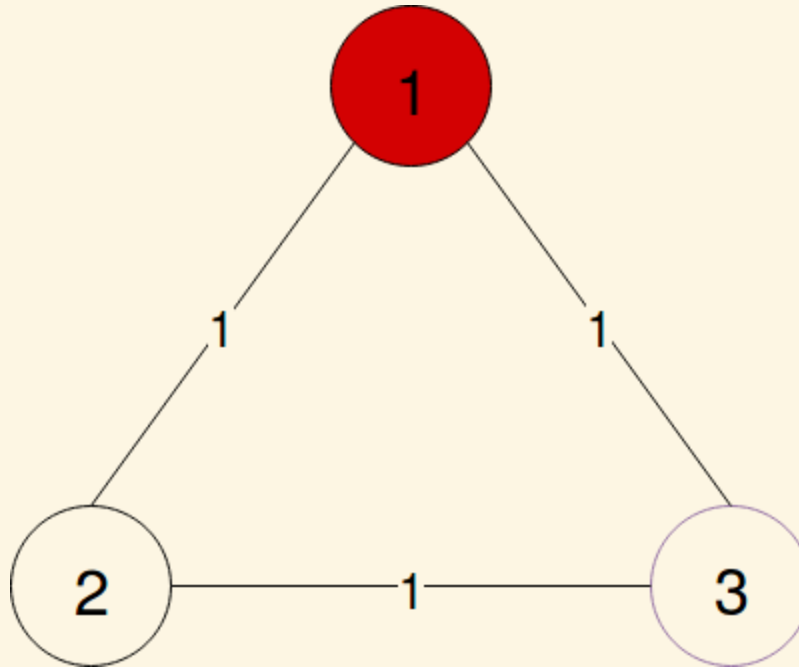
- 実行する補給計画: 1 -- **2** -- 1
- 時間: 1

サンプル 1



- 実行する補給計画: 1 -- 2 -- 1
- 時間: 3

サンプル 1



- 実行する補給計画: 1 -- 2 -- **1**
- 時間: 3

サンプル 1

- 先に損する逆回りでも OK

小課題 1 (12 点)

- $N \leq 10$
- $M \leq 10$
- $T = 1$
- $L \leq 10$
- $C_i \leq 10$

全てがとても小さい。

解法

- 全てがとても小さい
- 例えば(何番目の補給地点まで訪れたか, 現在の頂点, 直前の頂点) を状態として dijkstra すれば解ける
 - 頂点数: $O(LN^2)$
 - 辺数: $O(LNM)$
 - 時間計算量 $O(LNM \log (LN^2))$ など
- これ以外にも最短路問題 or DP に帰着できれば大抵 OK

小課題 2 (35 点, 合計 47 点)

- $N \leq 500$
- $M \leq 500$
- $T = 1$

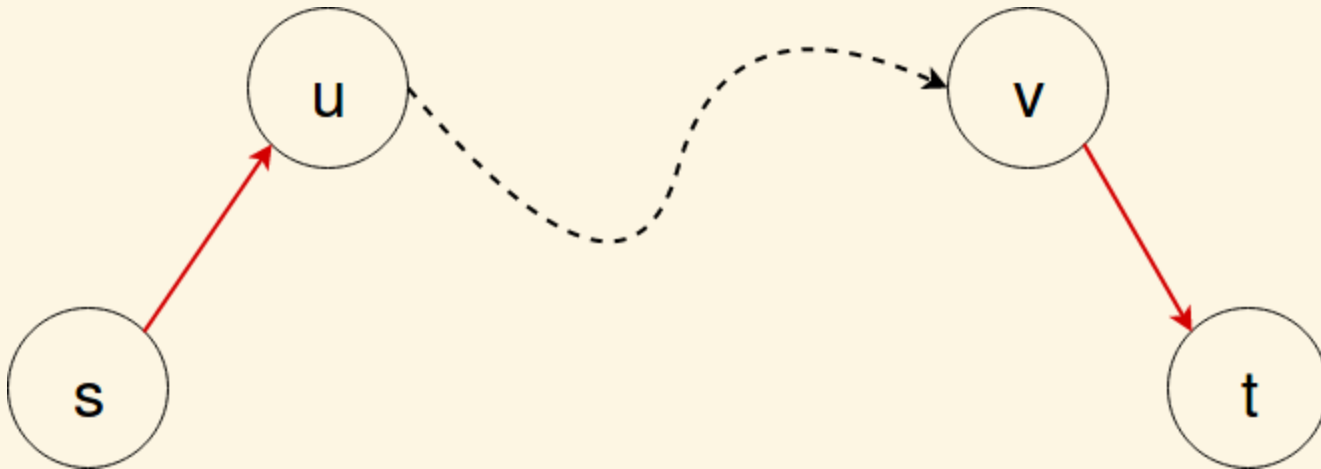
頂点数と辺数は小さい。クエリは 1 回。

考察

- 補給地点は多い ($L \leq 100,000$) -> 補給地点の数にできるだけ依らないようにしたい
- 補給地点間のパスは毎回計算しなくてもあらかじめやっておけばよさそう

考察

- 全ての頂点組 (s, t) と (s, t) に接続する辺 $(s \dashrightarrow u, v \dashrightarrow t)$ について, 最初に $s \rightarrow u$ と辺を使って最後に $v \rightarrow t$ と辺を使う時の最短路長を計算する

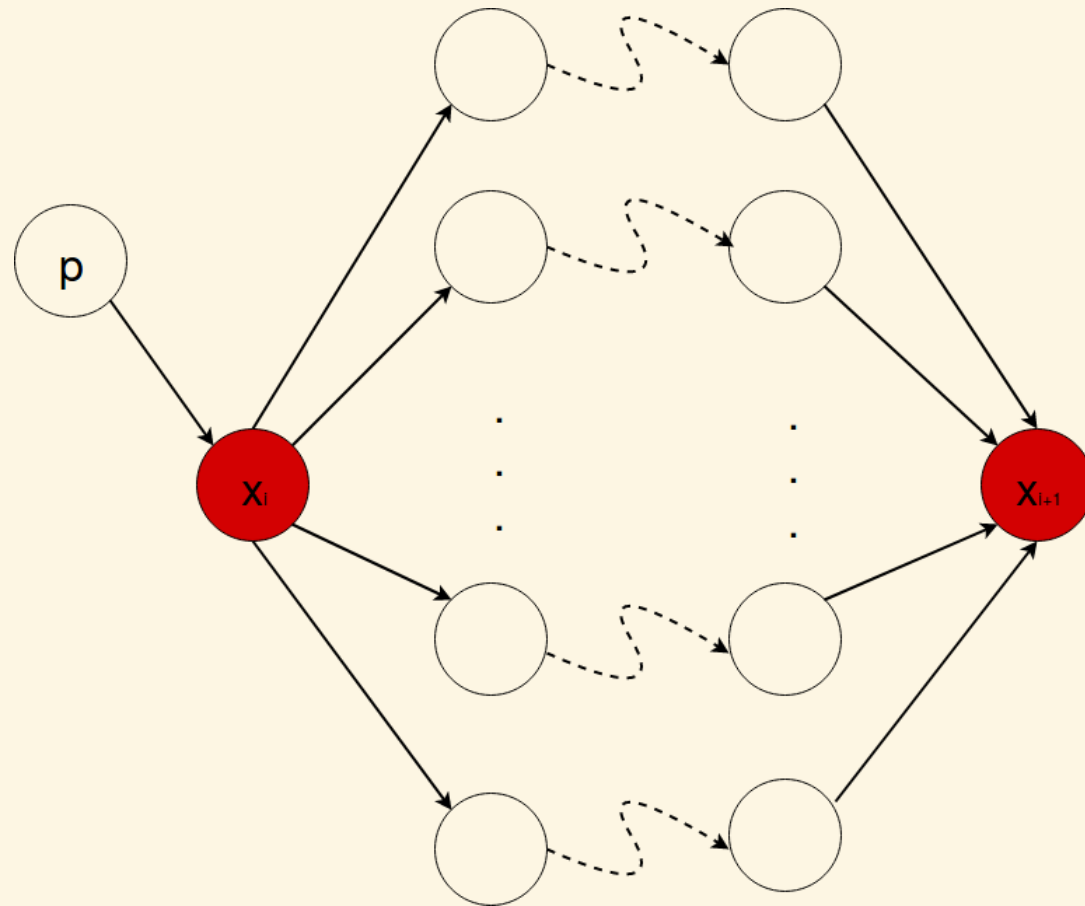


考察

- (現在の頂点, 一つ前の頂点) を状態として dijkstra すれば、
- 頂点数: $O(N^2)$
- 辺数: $O(NM)$
- 一回あたりの dijkstra の計算量は $O(NM \log N)$
- 辺の個数回だけ回せばよいので合計 $O(NM^2 \log N)$

考察

- (何番目の補給地点まで訪れたか, 直前の頂点) を状態にして DP すれば解けそう



考察

- 状態数: $O(NL)$
- 遷移回数: $O(NM^2L)$
- $L \leq 100,000$, $N \leq 500$, $M \leq 500$ -> やばい

考察

- 遷移数が多すぎてつらい
- だいたい $s \rightarrow t$ 最短パスを使う
 - 最短パスを使うと直前の頂点に引き返してしまうときや、その後の移動で最短パスを阻害してしまうときには最短パスを使わないかもしれない
- 本当に全ての (u, v) について調べる必要がある？

考察

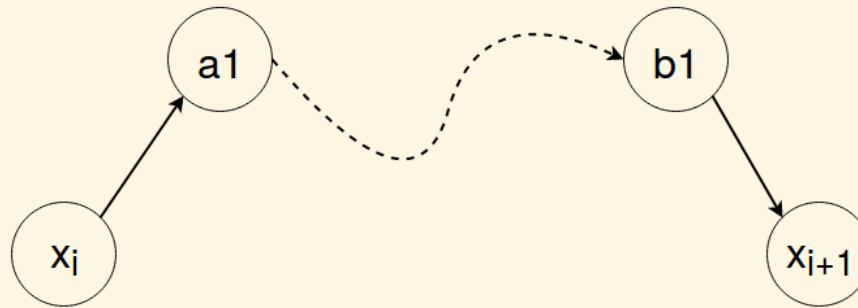
- 答えのパスが

$x_1 \rightarrow \dots \rightarrow p \rightarrow x_i \rightarrow \dots \rightarrow x_{i+1} \rightarrow q \rightarrow \dots \rightarrow x_L$

というとき、どんな p, q であってもしっかりと $x_i \rightarrow x_{i+1}$ パス
が選択できるようにしつつできるだけ絞りたい

考察

- 最短路 $s \rightarrow a1 \rightarrow \dots \rightarrow b1 \rightarrow t$ だけのとき

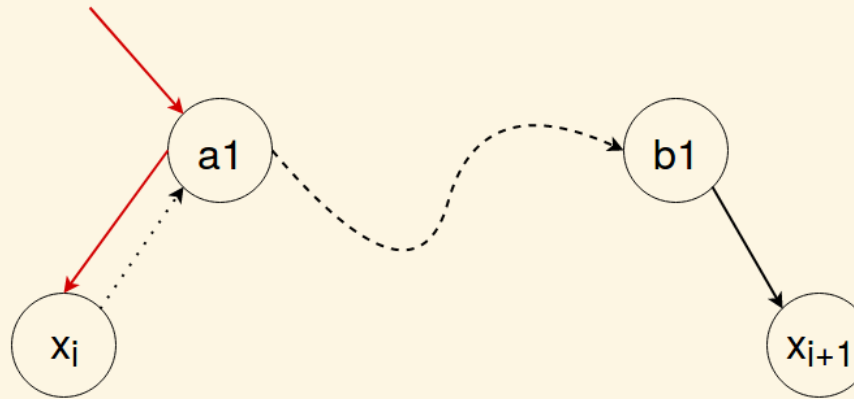


•

•

考察

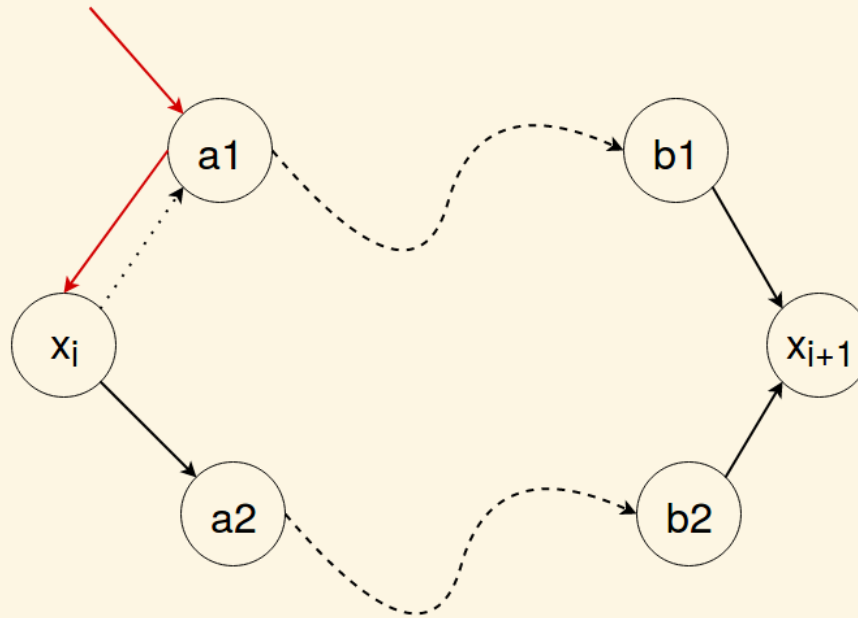
- 最短路 $s \rightarrow a1 \rightarrow \dots \rightarrow b1 \rightarrow t$ だけのとき



- $p = a1$ のとき死
- $a \neq a1$ となる最短経路も必要

考察

- 最短路 $s \rightarrow a2 \rightarrow \dots \rightarrow b2 \rightarrow t$ ($a2 \neq a1$) も追加した

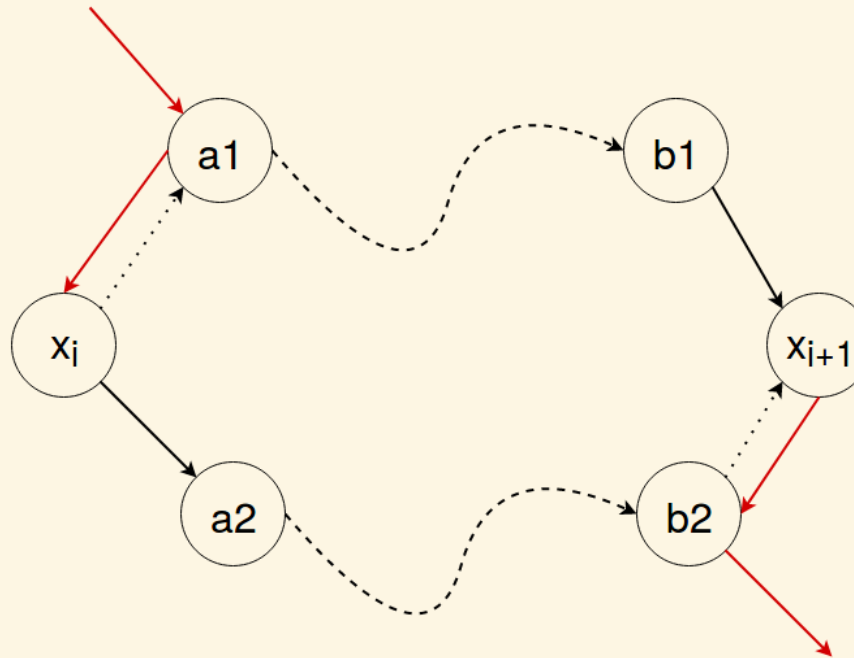


•

•

考察

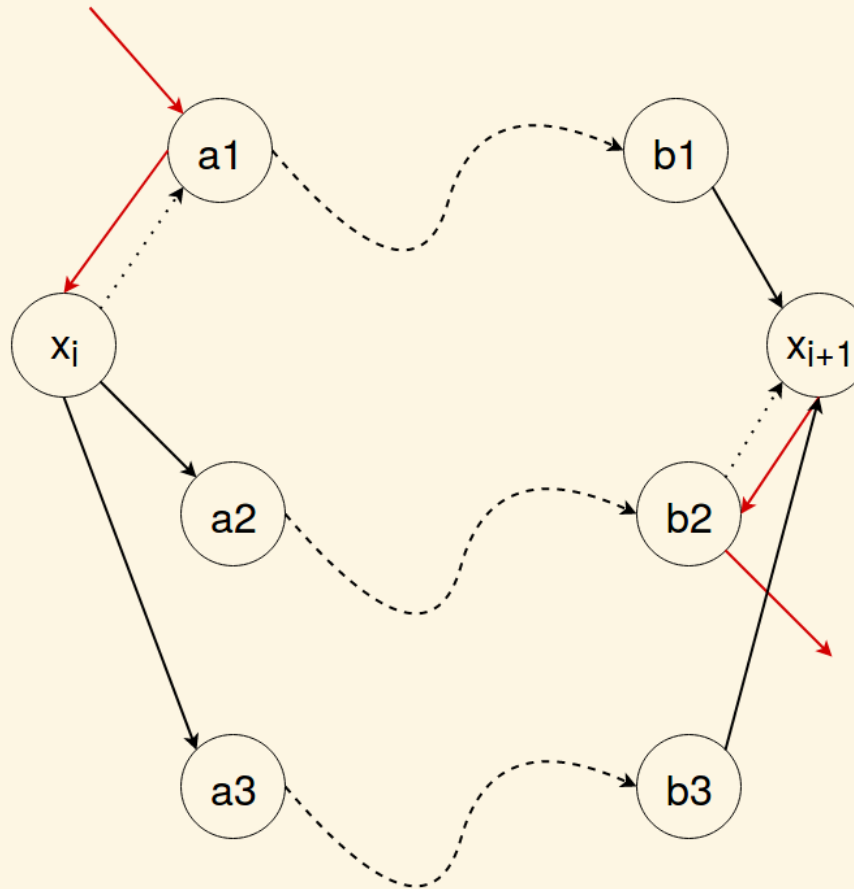
- 最短路 $s \rightarrow a2 \rightarrow \dots \rightarrow b2 \rightarrow t$ ($a2 \neq a1$) も追加した



- $p = a1, q = b2$ のとき死
- $a \neq a1, b \neq b2$ となる最短経路も必要

考察

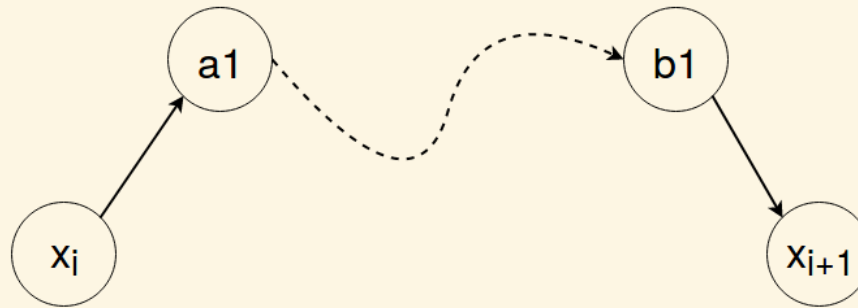
- 最短路 $s \rightarrow a3 \rightarrow \dots \rightarrow b3 \rightarrow t$ ($a3 \neq a1, b3 \neq b2$)



- $p = a1$ のときは OK になった
- $b \neq b1$ となる経路も必要

考察

- 最短路 $s \rightarrow a1 \rightarrow \dots \rightarrow b1 \rightarrow t$ だけのとき

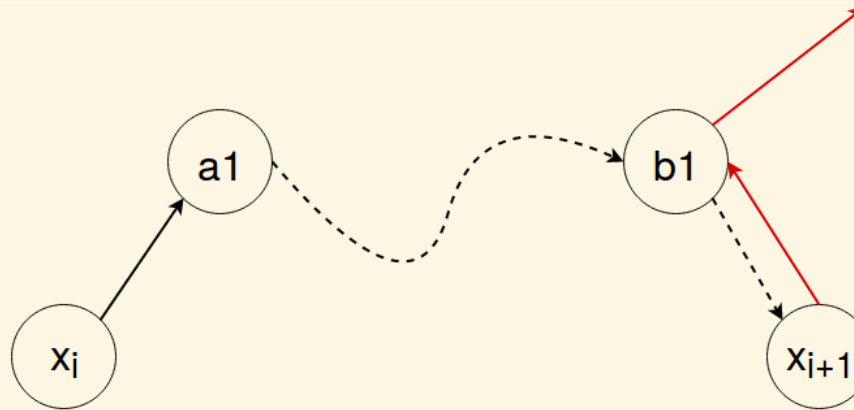


•

•

考察

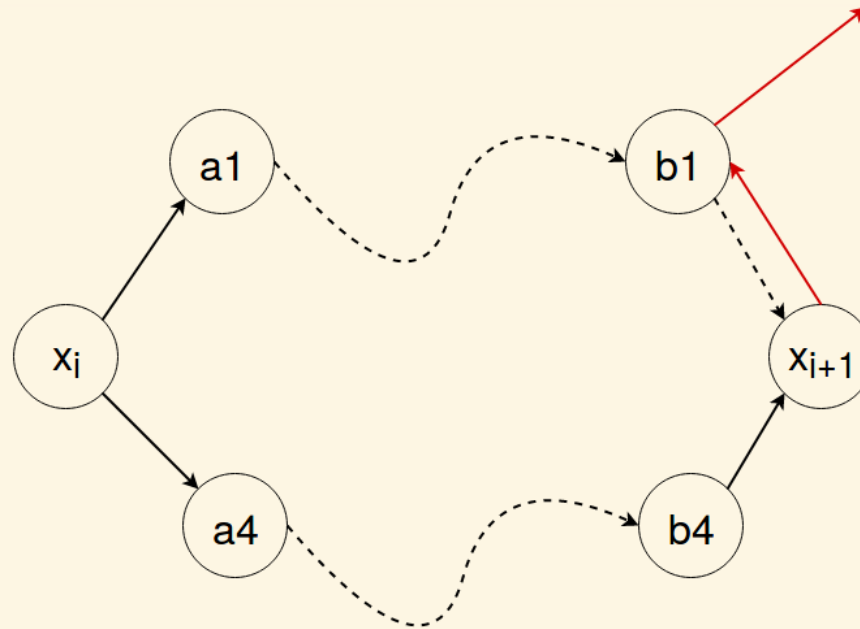
- 最短路 $s \rightarrow a1 \rightarrow \dots \rightarrow b1 \rightarrow t$ だけのとき



- $q = b1$ のとき死
- $b \neq b1$ となる経路も必要

考察

- 最短路 $s \rightarrow a4 \rightarrow \dots \rightarrow b4 \rightarrow t$ ($b4 \neq b1$)

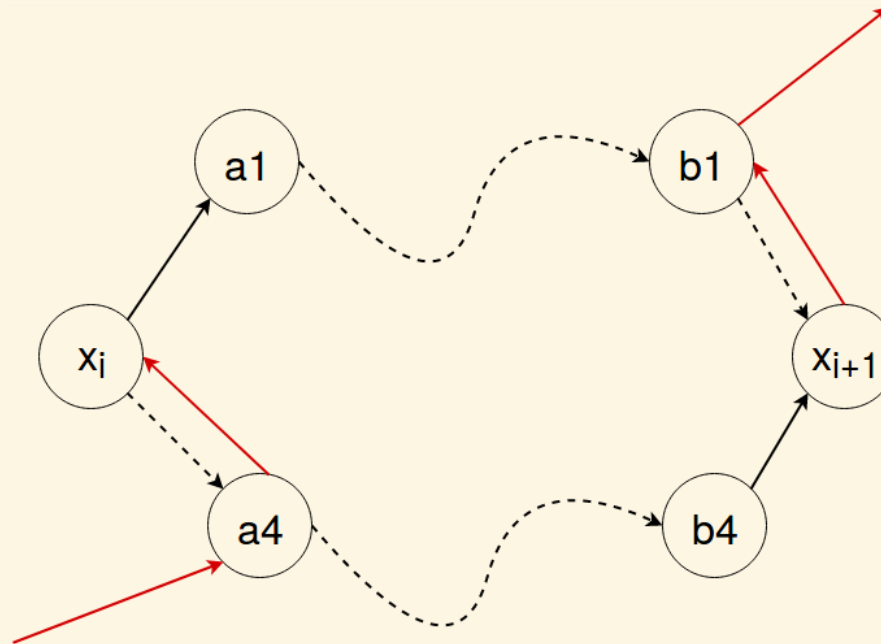


•

•

考察

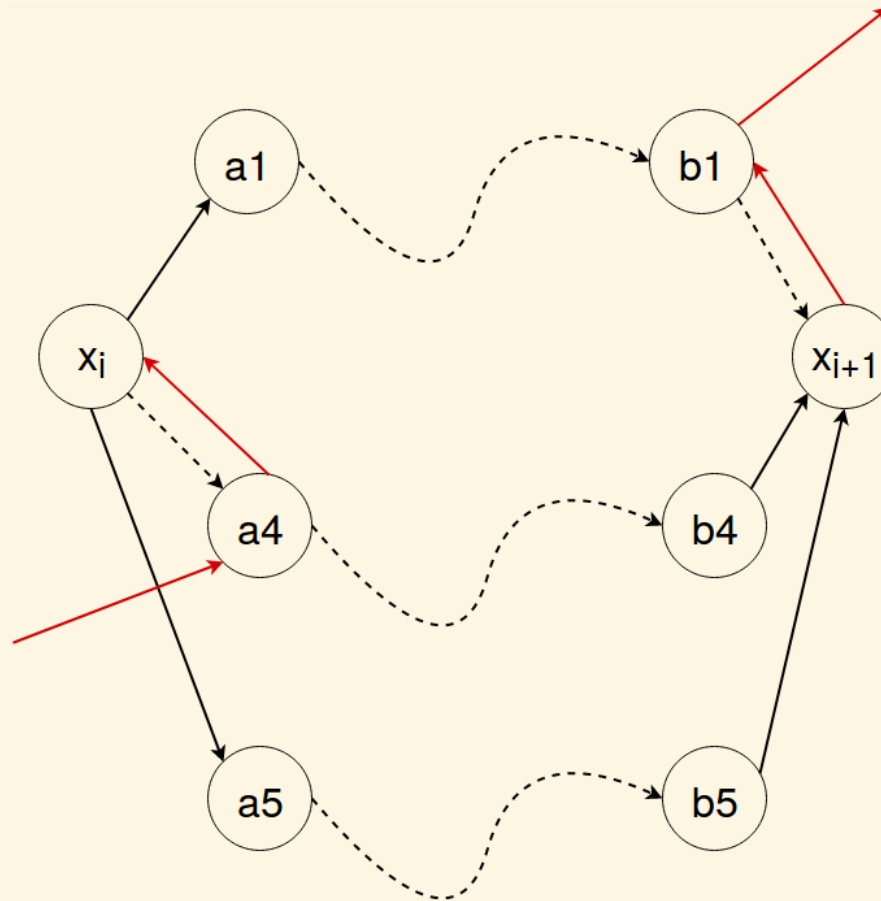
- 最短路 $s \rightarrow a4 \rightarrow \dots \rightarrow b4 \rightarrow t$ ($b4 \neq b1$)



- $p = a4$, $q = b1$ のとき死
- $a \neq a4$, $b \neq b1$ となる経路も必要

考察

- 最短路 $s \rightarrow a5 \rightarrow \dots \rightarrow b5 \rightarrow t$ ($a5 \neq a4$, $b5 \neq b1$)



- これで OK
- 合計 5 通り

考察

- どんな答えにおいても、 $x_i \rightarrow x_{i+1}$ パスとして考えられるのは 5 通りしかない

考察

- グッと睨むと

- `s -> a1 ->...-> b1 -> t`
- `s -> a2 ->...-> b2 -> t (a2 != a1, b2 != b1)`
- `s -> a3 ->...-> b3 -> t (a3 != a1, b3 != b2)`
- `s -> a4 ->...-> b4 -> t (a4 != a2, b4 != b1)`

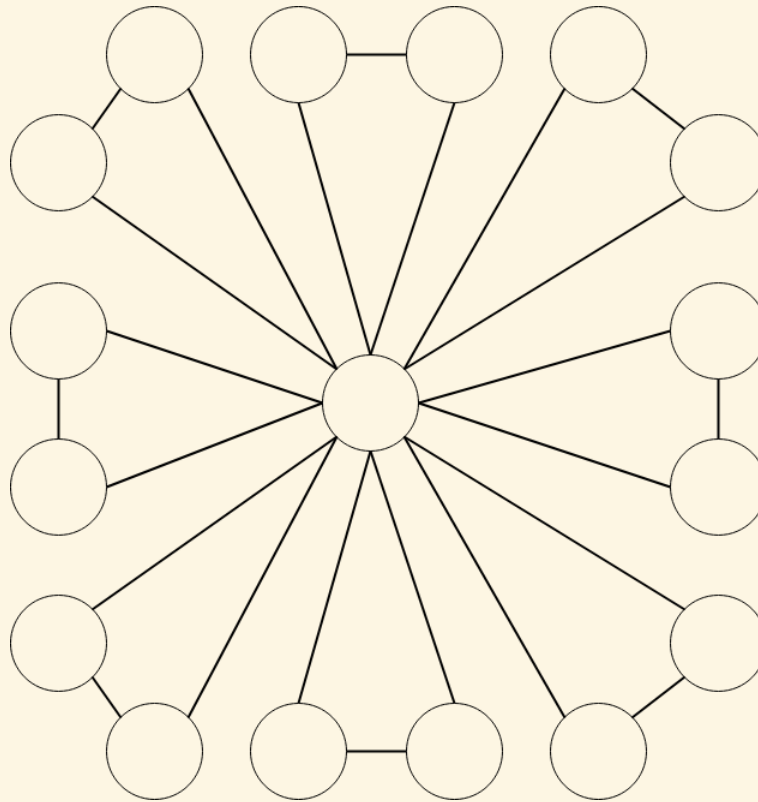
の 4 通りだけでもよいことがわかる

- いずれにせよ、必要なパスを定数個に絞る (以降は 5 通りに絞ったとする。定数は適宜読み替えてほしい。)

解法

- 補給地点間のパスは 5 (定数) 通りだけ考えればよい
- 前述の (何番目の補給地点まで訪れたか, 直前の頂点) を状態にして DP すれば
- 状態数: $O(NL)$
- 遷移回数: $O(NL)$
- 通る

別解(?)



- こういうグラフがテストケースに無いらしく、普通に毎回 dijkstra でも通るらしい

小課題 3 (15 点, 合計 62 点)

- $T = 1$

考察

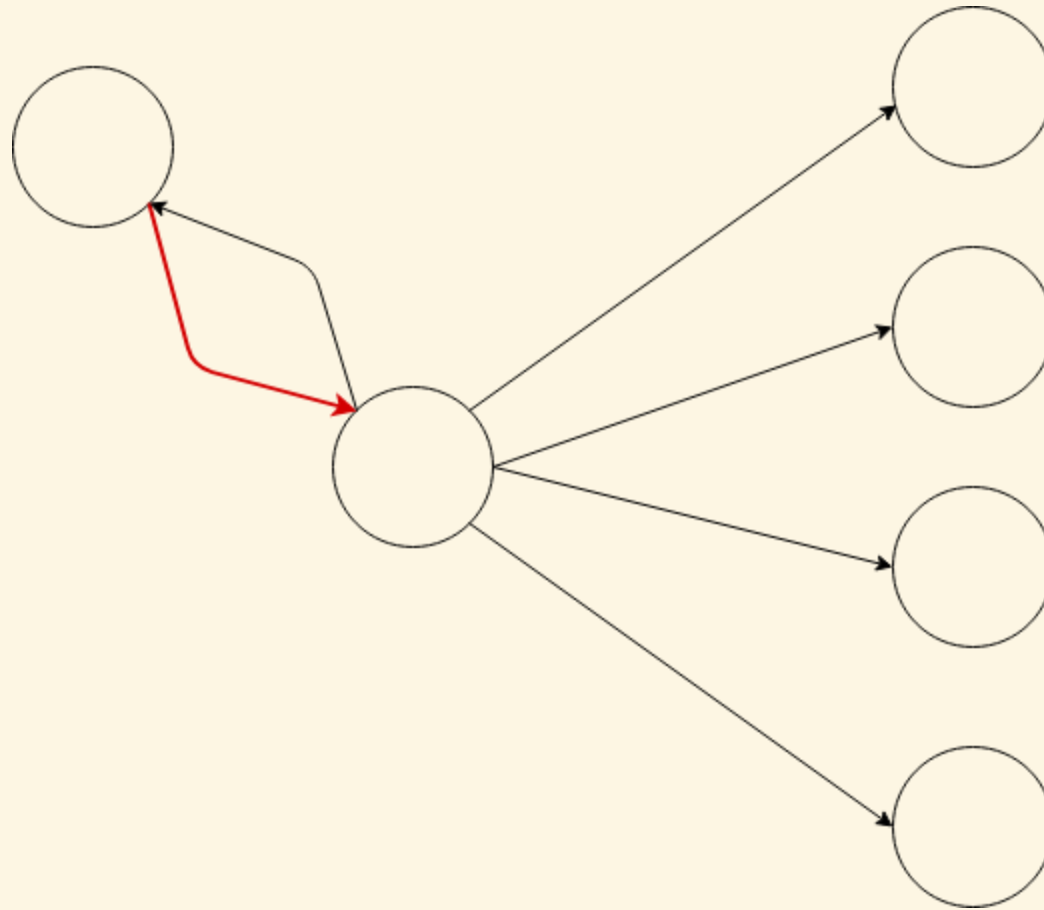
- 頂点数、辺数も $N \leq 2000, M \leq 2000$ とそこそこ多い
- DP 部分の計算量は $O(NL)$ なので OK
- $O(NM^2 \log N)$ かかっていた dijkstra の前計算を高速にする必要がある

考察

- (この辺をこの向きに通るまでの最小時間) を dijkstra で解きたい

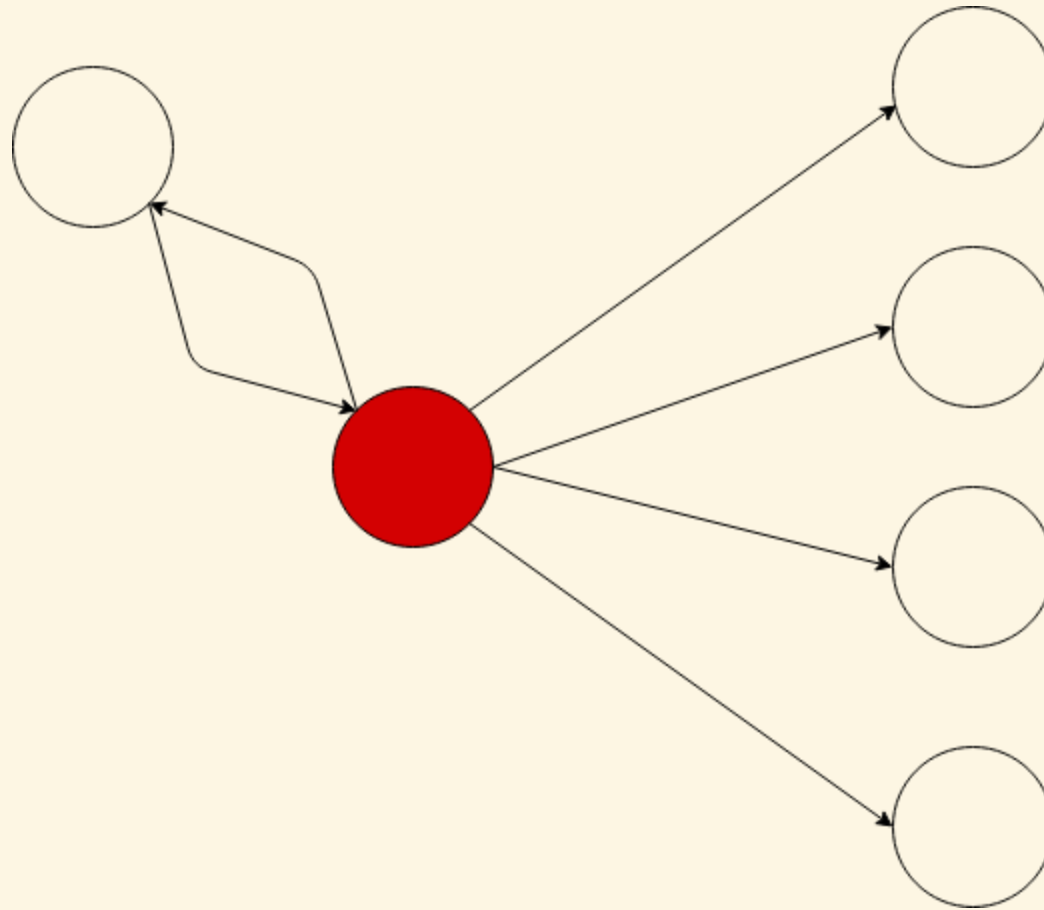
考察

- 頂点を初めて訪れた時、直前の辺の逆辺以外は全部更新する



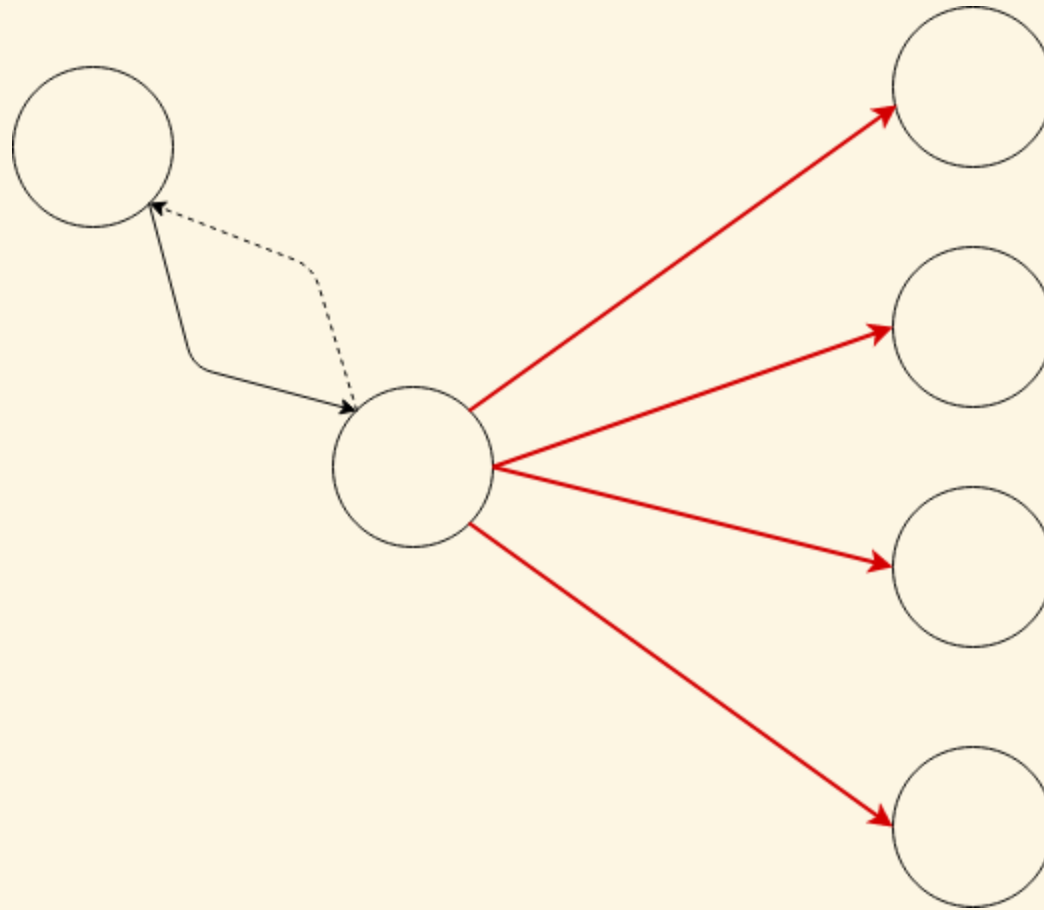
考察

- 頂点を初めて訪れた時、直前の辺の逆辺以外は全部更新する



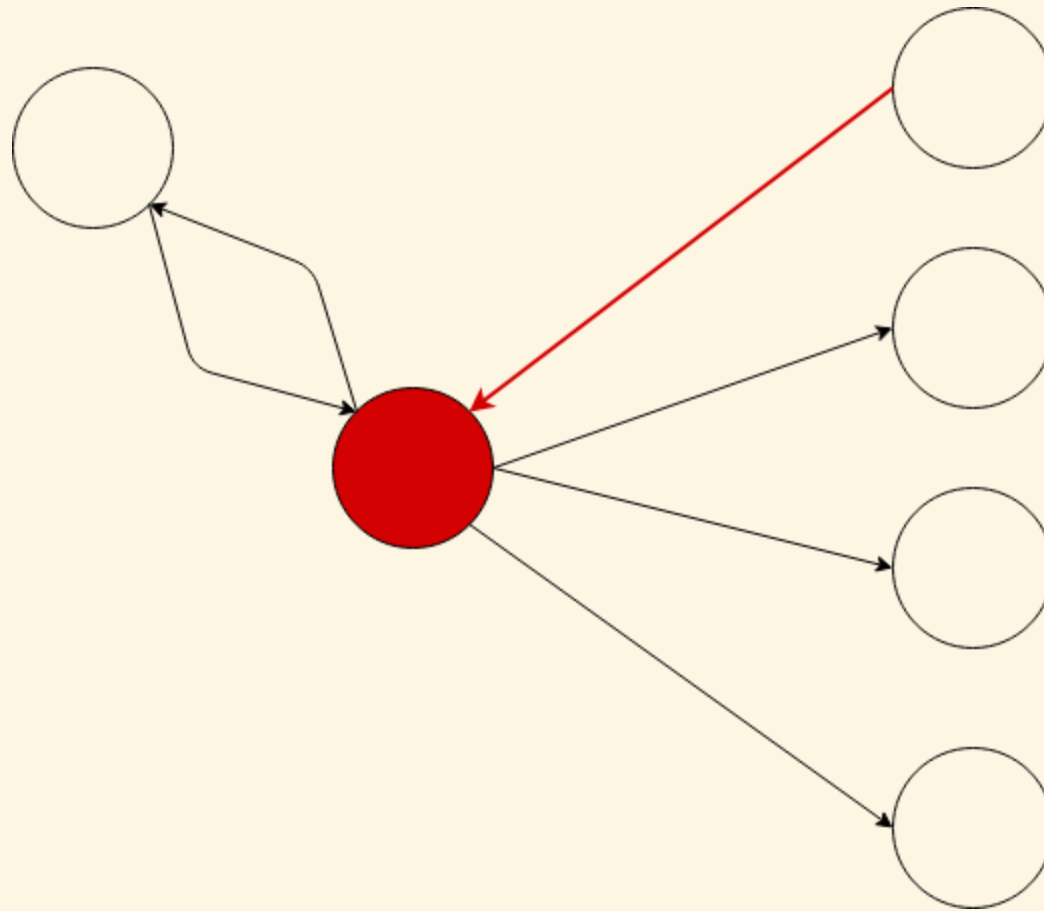
考察

- 頂点を初めて訪れた時、直前の辺の逆辺以外は全部更新する



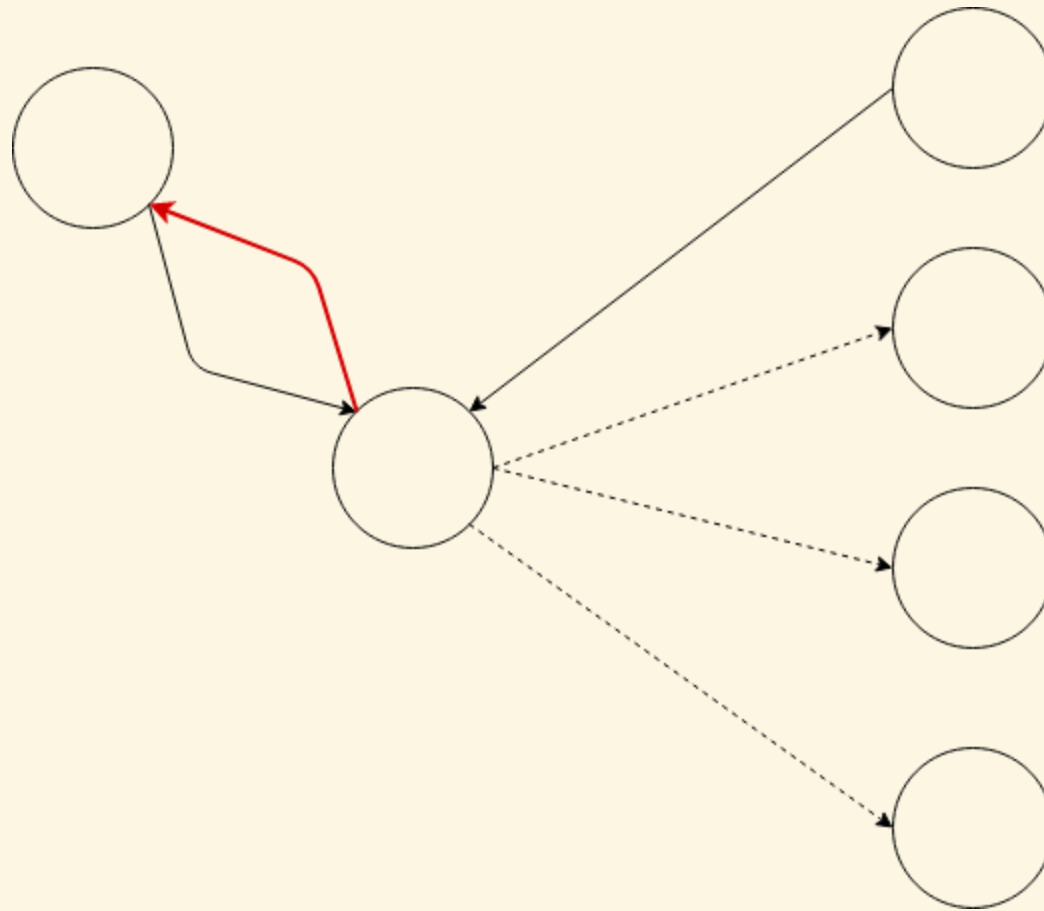
考察

- 次別の方向から来ると、初めて来た方向のみ更新すれば十分



考察

- 次別の方向から来ると、初めて来た方向のみ更新すれば十分



解法

- 辺を状態として dijkstra
- 頂点に (まだ訪れていない) or (最初どの辺から来たか) をもっておく
- 初めて来たら、直前に使った辺の逆辺以外は全部更新する
- 次別の方向から来ると、初めて来た方向のみ更新する
- 一回の dijkstra が $O(M \log M)$
- これを $2M$ 回やれば良いので $O(M^2 \log M)$
- 間に合う

小課題 4 (38 点, 合計 100 点)

- 追加の制約は無い

クエリが $T \leq 100,000$ 回来る

考察

- dijkstra の $O(M^2 \log M)$
- 毎回 DP に $O(NL)$ かけると合計 $O(TNL)$ で間に合わないので
高速に処理する必要がある

考察

- 変更される補給地点の数は 1 日ごとに高々 1 箇所
- ほとんど同じ計算を T 回することになっている
- 補給計画の変更しない区間は使いまわせないか

こういう一点更新クエリたくさんみたいなものを
処理するテクといえは...?

Segment tree

解法

- 区間の状態として必要なのは最初使う辺と最後使う辺(間で何が使われているかは範囲外には影響しない)
- 先ほど考察したように、補給地点間のパスとして考えられるのは高々 5 通り
- Segtree の $[1, r)$ に対応するノードに、 5×5 行列 $\text{node}[i, j]$ $:=$ 1 番目の補給地点から $r-1$ 番目の補給地点に行くのの最小値、ただし最初のパスとして、 $x[1] \rightarrow x[1+1]$ 間の i 番目のパスを使い $x[r-2] \rightarrow x[r-1]$ 間の j 番目のパスを使うを格納する

解法

- $[1, r)$, $[r, s)$ をマージするとき、 5^4 通りのパスを考えて同様の 5×5 行列を構築すればよい

解法

- 前計算の dijkstra: $O(M^2 \log M)$
- Segtree の初期化 $O(L)$
- クエリの回答: $O(T \log L)$

考察

- 定数倍が若干重い 5^4
- 実は 5×5 通りのパスを保存する必要はない
- さきほど補給地点間のパスとして 5 本しか考えなくてよいと考察したことと同様の考察をすると各ノードに 5 通りのパスしか保存しなくてもよい
- これで定数倍も安心
- 最初からパスを 4 通りに絞っているとここで頑張らなくても安心

得点分布

100:

62: ##

47: ##

12: #####

0 : #####

ご清聴ありがとうございました